

Apresentação do componente.**DESENVOLVIMENTO PARA SERVIDORES II (4)** ³⁰

Objetivos gerais. Desenvolver um site completo de *e-commerce* ou outro tipo de negócio na Internet usando uma linguagem apropriada a servidores, banco de dados e padrões de projeto.

Objetivos específicos. Implementar softwares do lado servidor e com uso de uma linguagem de programação e de padrões de projetos mais usuais como MVC, DAO, Composite, Singleton, entre outros.

Ementa. Conceitos e evolução das tecnologias de programação de servidores. Recursos da linguagem escolhida para servidores na Internet. Padrões de projetos. Integração com sistemas (Google Maps API, Twitter, entre outros)

Bibliografia básica

DEITEL, H; DEITEL, P. *Java – Como Programar*. São Paulo: Prentice-Hall do Brasil, 2010.

GAMMA, E et al. *Padrões de projeto*. Porto Alegre: Bookman, 2005.

MELO, A. A; LUCKOW, D. H. *Programação Java para a web*. São Paulo: Novatec, 2011.

Bibliografia complementar

DAIGNEAU, R. *Service design patterns*. Harlow (UK): Addison Wesley, 2011.

FREEMAN, E; FREEMAN, E. *Use a Cabeça! Padrões de Projetos*. 2, ed. Rio de Janeiro: Starlin Alta Consult, 2007.

HARTL, M. *Ruby on rails 3 tutorial*. Harlow (UK): Addison Wesley, 2011.

IERUSALIMSKY, R; CELES, W; FIGUEIREDO, L. H. *LUA programming gems*. Rio de Janeiro: LUA. Org, 2008.

RUBY, S; THOMAS, D; HANSSON, D. *Agile web development with rails*. New York: Oreilly & Assoc. 2010.

1-) Uso de celulares:

Para o bom andamento das aulas, recomendo que utilizem os celulares em *vibracall*, para não atrapalhar o andamento da aula.

2-) Material das aulas:

A disciplina trabalha com NOTAS DE AULA que são disponibilizadas ao final de cada aula, iremos utilizar o GitHub como repositório, que está neste local: <https://github.com/marcossousa33dev/FATECSRDSII202601>

3-) Prazos de trabalhos e atividades:

Toda atividade solicitada terá uma data limite de entrega, de forma alguma tal data será postergada, ou seja, se não for entregue até a data limite a mesma receberá nota 0, isso tanto para atividades entregues de forma impressa ou enviadas ao e-mail da disciplina. Qualquer problema que tenham, me procurem com antecedência para verificarmos o que pode ser realizado.

4-) Qualidade do material de atividades:

- Impressas ou manuscritas:

Muita atenção na qualidade do que será entregue, atividades sem grampear, faltando nome e número de componentes, rasgadas, amassadas, com rebarba de folha de caderno e etc. serão desconsiderados por mim.

- Digitais:

Ao enviarem atividades para o e-mail da disciplina, SEMPRE no assunto deverá ter o nome da atividade que está sendo enviada, e no corpo do e-mail deverá ter o(s) nome(s) do(s) integrante(s) da atividade, sem estar desta forma a atividade será DESCONSIDERADA.

5-) Critérios de avaliação:

Cada trimestres teremos as seguintes formas de avaliação:

- Práticas em Laboratório;
- Assiduidade;
- Outras que se fizerem necessário.

1. Introdução

Nessa fase do curso iremos abordar a ideia da programação no lado do servidor, o que podemos chamar de **backend**, para isso, utilizaremos a linguagem PHP, nossa preocupação não será o **frontend**, e sim entendermos o comportamento dessa linguagem dentro da programação Web, com o andamento do curso iremos refinando e melhorando os conceitos.

Mas antes vamos imaginar a internet na qual você pode apenas consumir conteúdos, como se fosse um jornal, uma revista, ou ainda, um programa na televisão. Chato, né? Mas quando se aprende as linguagens da web, como HTML e CSS, podemos montar sites que são como revistas e servem apenas para leitura, sem permitir interação com os internautas.

O segredo da famosa web 2.0 é a capacidade de interação entre as pessoas e os serviços online. Mas, para que esta interação seja possível, é necessário que os sites sejam capazes de receber informações dos internautas e também de exibir conteúdos personalizados para cada um, ou de mudar seu conteúdo automaticamente, sem que o desenvolvedor precise criar um novo HTML para isso.

Estes dois tipos de sites são chamados de estático e dinâmico, respectivamente.

Você já parou para pensar em tudo o que acontece quando você digita um endereço em seu navegador web? A história toda é mais ou menos assim:

- O navegador vai até o servidor que responde no endereço solicitado e pede a página solicitada.
- O servidor verifica se o endereço existe e se a página também existe em seu sistema de arquivos e então retorna o arquivo para o navegador.
- Após receber o arquivo HTML, o navegador começa o trabalho de renderização, para exibir a página para o usuário. É neste momento que o navegador também requisita arquivos de estilos (css), imagens e outros arquivos necessários para a exibição da página.

Quando se desenvolve páginas estáticas, este é basicamente todo o processo necessário para que o navegador exiba a página para o usuário. Chamamos de estáticas as páginas web que não mudam seu conteúdo, mesmo em uma nova requisição ao servidor.

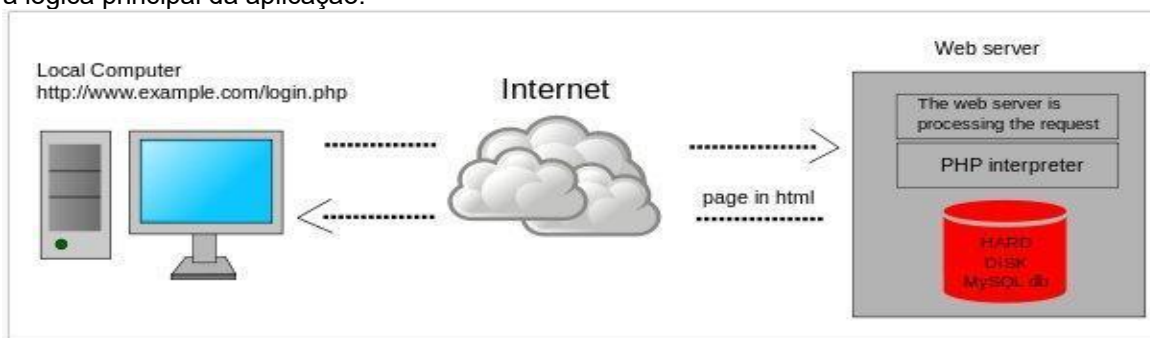
E como funciona uma página dinâmica?

O processo para páginas dinâmicas é muito parecido com o das páginas estáticas. A diferença é que a página será processada **no servidor** antes de ser enviada para o usuário. Este processamento no servidor é usado para alterar dinamicamente o conteúdo de uma página, seja ele HTML, CSS, imagens ou outros formatos.

Pense, por exemplo, em um site de um jornal. Em geral, este tipo de site contém algumas áreas destinadas às notícias de destaque, outras áreas para notícias gerais e ainda outras áreas para outros fins. Quando o navegador solicita a página para o servidor, ele irá montar o conteúdo antes de enviar para o navegador. Este conteúdo pode ser conseguido de algumas fontes, mas a mais comum é um banco de dados, onde, neste caso, as notícias ficam armazenadas para serem exibidas nas páginas quando necessário.

2. Scripting no lado do servidor (server-side)

A linguagem de servidor, ou *server-side scripting*, é a linguagem que vai rodar, fornecendo a lógica principal da aplicação.



Do lado do servidor significa toda a regra de negócio, acesso a banco de dados, a parte matemática e cálculos estarão armazenadas, enquanto que *local computer* será na máquina do usuário/cliente. Para a segurança e integridade dos dados, do lado do servidor geralmente é melhor, pois o usuário final não será capaz de ver ou manipular os dados que são enviados para o mesmo, e muito menos visualizar a regra de negócio, ou a forma de acesso ao banco de dados.

3. Scripting no lado do cliente (client-side)

As linguagens *client-side* são linguagens onde apenas o seu NAVEGADOR vai entender. Quem vai processar essa linguagem não é o servidor, mas o seu browser (Chrome, IE, Firefox,

etc...). Significa "lado do cliente", ou seja, aplicações que rodam no computador do usuário sem necessidade de processamento de seu servidor (ou host) para efetuar determinada tarefa.

Basicamente, ao se falar de aplicações client-side na web, estamos falando de *JavaScript*, e AJAX (*Asynchronous Javascript And XML*).

Existem vantagens e desvantagens ao utilizar o JavaScript e AJAX.

A principal vantagem está na possibilidade de você economizar bandwidth (largura de banda), e dar ao usuário uma resposta mais rápida de sua aplicação por não haver processamento externo.

Outra vantagem ao utilizar, agora o AJAX, seria o apelo visual de sua aplicação e rapidez de resposta. O que o AJAX faz é o processamento externo (server-side) parecendo ser interno (*client-side*). O usuário não percebe que houve um novo carregamento de página, pois ele busca informações no servidor e mostra rapidamente em um local específico da página através do JavaScript.

A principal desvantagem do *JavaScript* atualmente é que o usuário pode desativá-lo em seu navegador. Se a sua aplicação basear-se exclusivamente em JavaScript, nesse caso, ela simplesmente não vai funcionar.

4. A diferença entre o lado do cliente (*client-side*) e do lado do servidor (*server-side*) Scripting

Ao escrever aplicações para a web, você pode colocar os programas, ou scripts, ou no servidor da Web ou no navegador do cliente, a melhor abordagem combina uma mistura cuidadosa dos dois. Endereços de scripts do lado do servidor de gerenciamento e segurança de dados, enquanto que o script do lado do cliente se concentra principalmente na verificação de dados e layout de página.

Um servidor web é um computador separado e software com a sua própria conexão à Internet. Quando o navegador solicita uma página, o servidor recebe o seu pedido e envia o conteúdo do navegador. Um script de programa que executa no servidor web gera uma página com base na lógica do programa e envia para o navegador do usuário. O conteúdo pode ser texto e imagens padrão, ou pode incluir scripts do lado do cliente. Seu navegador executa os scripts do lado do cliente, o que pode animar imagens da página web, solicitar os dados do servidor ou executar outras tarefas.

Para um website ter uma sessão, onde você faz *login*, fazer compras e outros pedidos, o servidor precisa para identificar o computador. Milhares de usuários podem ser registrados em, ao mesmo tempo, o servidor tem de distingui-los. *Scripting* do lado do servidor mantém o controle da identidade de um usuário através de alguns mecanismos diferentes, tais como variáveis de sessão. Quando você fizer *login*, o script do servidor cria uma identificação de sessão exclusiva para você. O script pode armazenar informações em variáveis que duram o tempo que você ficar conectado.

Muitas páginas da web têm formulários para você preencher com seu nome, endereço e outras informações. Para certificar que os dados vão corretamente, os scripts de validação são capazes de verificar que as datas e códigos postais contêm apenas números e os estados têm certas combinações de duas letras. Este processo é mais eficaz quando o script é executado no lado do cliente. Caso contrário, o servidor tem que receber os dados, verificá-lo, e enviar-lhe uma mensagem de erro. Quando o navegador faz isso, você envia os dados de volta para o servidor somente uma vez.

Quando uma sessão de web envolve peneirar grandes quantidades de dados, um *script* do lado do servidor faz este trabalho melhor. Por exemplo, um banco pode ter um milhão de clientes. Quando você entrar, ele deve buscar o seu registro a partir deste arquivo grande. Ao invés de enviá-lo por toda a sua conexão de Internet para o seu navegador, o servidor web solicita informações de um servidor de dados perto dele. Além de aliviar a Internet do tráfego de dados desnecessários, isso também melhora a segurança.

Podemos encontrar uma maior variedade de linguagens de programação em servidores do que em navegadores. Os programadores fazem mais *scripting* do lado do cliente com a linguagem Javascript. No lado do servidor, você pode escrever em linguagens como PHP, VBScript ou ColdFusion. Enquanto alguns programadores escrever scripts do lado do cliente para executar fora do navegador, isso é arriscado, uma vez que pressupõe que o computador sabe que a linguagem.

5. Mantendo o histórico do código

Quando ingressamos no mercado de trabalho, veremos que sempre existirá uma pressão por entregas rápidas de novas funcionalidades nos sistemas que desenvolveremos. Mas, apesar da pressa, será necessário prestar atenção no que estamos fazendo, mesmo se a alteração for pequena.

Ao mexermos em um código existente é importante tomarmos cuidado para não quebrar o que já funciona. Por isso, queremos mexer o mínimo possível no código.

Temos medo de remover código obsoleto, não utilizado ou até mesmo comentado, mesmo que mantê-lo já nem faça sentido. Não é incomum no mercado vermos código funcional acompanhado de centenas de linhas de código comentado. Sem dúvida, é interessante manter o histórico do código dos projetos, para entendermos como chegamos até ali. Mas manter esse histórico junto ao código atual, com o decorrer do tempo, deixa nossos projetos confusos, poluídos com trechos e comentários que poderiam ser excluídos sem afetar o funcionamento do sistema. Seria bom se houvesse uma maneira de navegarmos pelo código do passado, como uma máquina do tempo para código.

5.1. Trabalhando em equipe

Mas, mesmo que tivéssemos essa máquina do tempo, temos outro problema: muito raramente trabalhamos sozinhos. Construir um sistema em equipe é um grande desafio, temos feito bastante práticas com isso em nossas aulas.

Nosso código tem que se integrar de maneira transparente e sem emendas com o código de todos os outros membros da nossa equipe. Como podemos detectar que estamos alterando o mesmo código que um colega? Como mesclar as alterações que fizemos com a demais alterações da equipe?

E como identificar conflitos entre essas alterações? Fazer isso manualmente, com cadernetas ou planilhas e muita conversa, parece trabalhoso demais e bastante suscetível a erros e esquecimentos. Seria bom que tivéssemos um robô de integração de código, que fizesse todo esse trabalho automaticamente.

5.2. Sistemas de controle de versão

Existem ferramentas que funcionam como máquinas do tempo e robôs de integração para o seu código. Elas nos permitem acompanhar as alterações desde as versões mais antigas.

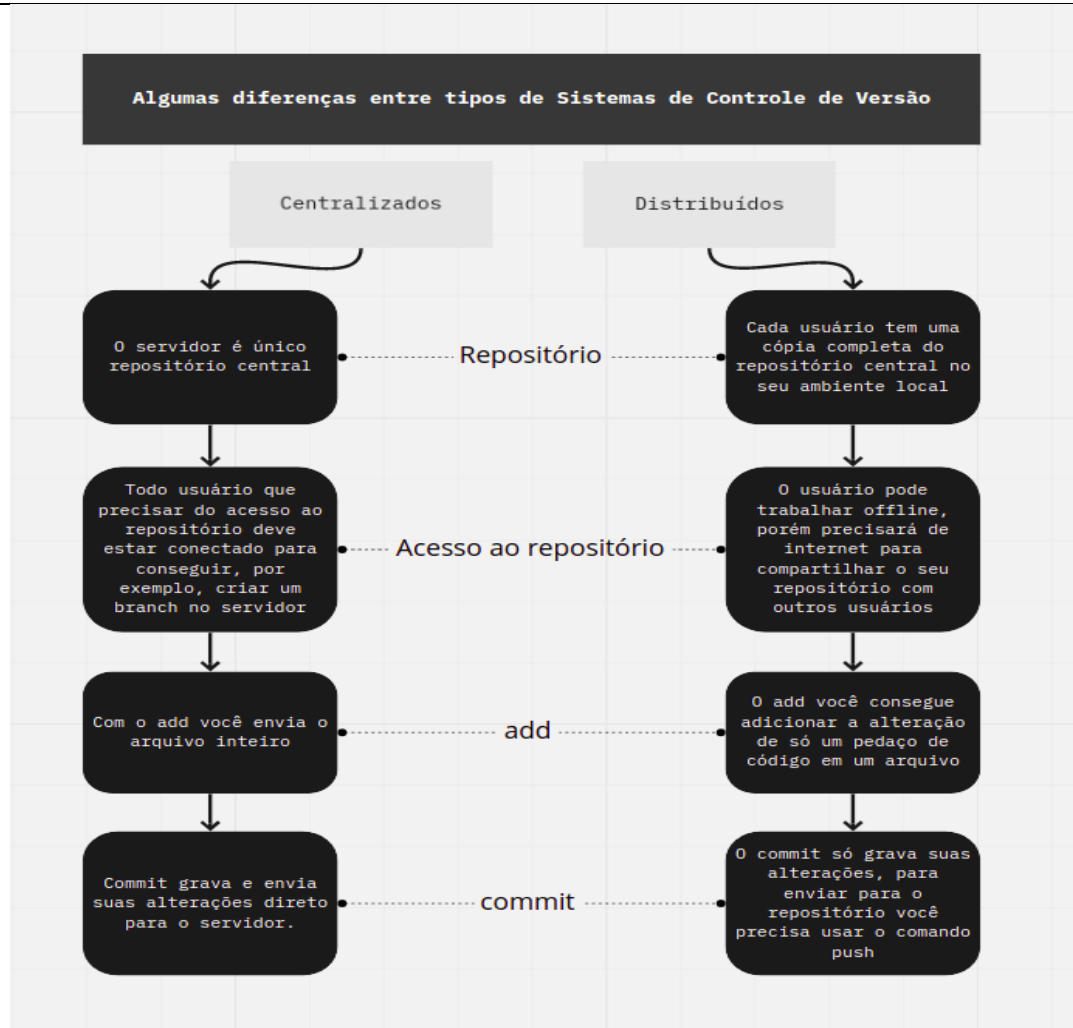
Também é possível detectar e mesclar alterações nos mesmos arquivos, além de identificar conflitos, tudo de maneira automática. Essas ferramentas são chamadas de sistemas de controle de versão. Nesse tipo de ferramenta, há um repositório que nos permite obter qualquer versão já existente do código. Sempre que quisermos controlar as versões de algum arquivo, temos que informar que queremos rastreá-lo no repositório. A cada mudança que desejamos efetivar, devemos armazenar as alterações nesse repositório.

Alterações nos mesmos arquivos são mescladas de maneira automática sempre que possível. Já possíveis conflitos são identificados a cada vez que obtemos as mudanças dos nossos colegas de time.

Desde a década de 1990, existe esse tipo de ferramenta. Alguns exemplos de sistemas de controle de versão mais antigos são CVS, *ClearCase*, *SourceSafe* e SVN (que ainda é bastante usado nas empresas). Em meados da década de 2000, surgiram sistemas de controle de versão mais modernos, mais rápidos e confiáveis, como Mercurial, Bazaar e, é claro, Git.

Precisamos entender que ele é o responsável por gravar todas as alterações feitas, e além disso, precisa permitir que todas as alterações feitas ocorram paralelamente e não comprometam o andamento do projeto.

Os sistemas de versionamento podem ser classificados como distribuídos ou centralizados, vamos ver abaixo:



A partir dessa imagem, conseguimos ver algumas diferenças e o que chama mais atenção — é importante focar sem entrar tanto em partes técnicas — é o que mais diferencia: a forma como trabalhamos com eles e também o ambiente local que o Git nos proporciona, dando uma cópia completa do repositório. Apesar de alguns fluxos terem o mesmo nome, eles funcionam de formas distintas.

5.3. O que é o Git?

Como sempre falamos, precisamos entender a história e conceito do que vamos estudar para termos plena ciência do conceito que iremos aplicar.

O Git é um sistema de controle de versão distribuído, o que significa que um clone local do projeto é um repositório de controle de versão completo. Esses repositórios locais totalmente funcionais facilitam o trabalho offline ou remoto. Os desenvolvedores confirmam o trabalho localmente e depois sincronizam a cópia do repositório com a cópia no servidor. Esse paradigma é diferente do controle de versão centralizado, no qual os clientes devem sincronizar o código com um servidor antes de criar novas versões do código.

A flexibilidade e a popularidade do Git o tornam uma ótima escolha para qualquer equipe. Muitos desenvolvedores e universitários já sabem como usar o Git. A comunidade de usuários do Git criou recursos para treinar desenvolvedores, e a popularidade do Git facilita a obtenção de ajuda quando necessário. Quase todos os ambientes de desenvolvimento têm suporte ao Git e as ferramentas de linha de comando do Git implementadas em todos os principais sistemas operacionais.

Toda a história da criação do *Git* se dá em torno do *Kernel* do Linux. Ele e seu versionamento foram mantidos durante muito tempo no *Bitkeeper*, mas a empresa por trás dele, a *BitMoover*, resolveu ir para uma linha de comercialização do licenciamento de acesso ao *software*.

Por um tempo, ela permitiu que os desenvolvedores do kernel do Linux usassem a versão gratuita (*Freeware*), mas um dos mantenedores, Andrew Tridgell, desenvolveu uma versão *open source* do *client*, uma ferramenta que poderia trabalhar com o código fonte armazenado no *BitKeeper*. Porém, a *BitMoover* ficou extremamente insatisfeita e considerou como violação da licença e engenharia reversa a criação desse *client*.

Após meses de negociação, esse acesso gratuito foi revogado e surgiu a necessidade da criação de algo que não só substituísse. Mas fosse melhor que o *Bitkeeper*. Linus e todas as pessoas que mantinham o Kernel começaram a busca, mas nenhuma das opções de controle de versão descentralizadas disponíveis atendiam às necessidades.

Nasce, então, o *Git* com a proposta de resolver a necessidade de um sistema de distribuição descentralizada, e que segue a filosofia de software livre, assim como o Linux, e sem se preocupar em ficar preso a uma ferramenta proprietária que de um dia pro outro os criadores resolvessem comercializá-lo.

Visto de um ângulo técnico, o *Git* é uma ferramenta semelhante ao *Bitkeeper* por trabalhar de forma distribuída e uma coisa que acho interessante é que grande parte do *flow* que conhecemos no *Git*, veio do *flow* aprendido com o uso do *Bitkeeper*. Porém, mesmo com essa semelhança a ideia nunca foi ter um clone.

Quando foi criado, a pretensão era resolver exclusivamente o problema relacionado ao Kernel do Linux. Mas hoje o *Git* se tornou uma das ferramentas mais utilizadas no mundo inteiro para controle de versão.

5.4. Benefícios do Git

5.4.1. Desenvolvimento simultâneo:

Todos têm sua própria cópia local do código e podem trabalhar simultaneamente em seus próprios *branches*. O *Git* funciona offline, já que quase todas as operações são locais.

5.4.2. Lançamentos mais rápidos

Os *branches* permitem o desenvolvimento flexível e simultâneo. O *branch* principal contém um código estável e de alta qualidade, com base no qual você lançará versões. Os *branches* de recurso contêm trabalhos em andamento, que são mesclados no *branch* principal após a conclusão. Ao separar o *branch* de lançamento do desenvolvimento em andamento, é mais fácil gerenciar um código estável e enviar atualizações mais rapidamente.

5.4.3. Integração interna

Devido à sua popularidade, o *Git* pode ser integrado à maioria dos produtos e ferramentas. Todas as principais IDEs têm suporte interno ao *Git*, e muitas ferramentas oferecem suporte à integração contínua, implantação contínua, testes automatizados, rastreamento de itens de trabalho, métricas e integração de recursos de relatório com o *Git*. Essa integração simplifica o fluxo de trabalho diário.

5.4.4. Forte suporte da comunidade

O *Git* usa um código aberto e se tornou o padrão de fato para controle de versão. Há inúmeras opções de ferramentas e recursos disponíveis para as equipes aproveitarem. O volume de suporte da comunidade para o *Git* em comparação com outros sistemas de controle de versão facilita a obtenção de ajuda quando necessário.

5.4.5. O *Git* funciona com qualquer equipe

Usar o *Git* com uma ferramenta de gerenciamento de código-fonte aumenta a produtividade de uma equipe, incentivando a colaboração, aplicando políticas,

automatizando processos e melhorando a visibilidade e a rastreabilidade do trabalho. A equipe pode se contentar com ferramentas individuais para controle de versão, rastreamento de itens de trabalho e integração e implantação contínuas. Ou os membros da equipe podem escolher uma solução como o GitHub ou o Azure DevOps que ofereça suporte a todas essas tarefas em um só lugar.

5.4.6. Solicitações *pull*

Use solicitações *pull* para discutir alterações de código com a equipe antes de mesclá-las no branch principal. As discussões em solicitações *pull* são inestimáveis para garantir a qualidade do código e melhorar o conhecimento de toda a sua equipe. Plataformas como o GitHub e o Azure DevOps oferecem uma experiência avançada de solicitação de *pull*, em que os desenvolvedores podem procurar alterações de arquivo, deixar comentários, inspecionar commits, exibir compilações e votar para aprovar o código.

5.4.7. Políticas de *branch*

As equipes podem configurar o GitHub e o Azure DevOps para aplicar fluxos de trabalho e processos consistentes em toda a equipe. Eles podem configurar políticas de *branch* para garantir que as solicitações *pull* atendam aos requisitos antes da conclusão. As políticas de *branch* protegem *branches* importantes, evitando transmissões diretas, exigindo revisores e garantindo compilações limpas.

5.5. O que é o GitHub?

GitHub é uma plataforma de hospedagem de códigos-fonte e de outros tipos de arquivos que utiliza o *software* Git. Como a GitHub é uma plataforma baseada na nuvem e está disponível na Web, o Git assume o papel de Sistema de Controle de Versões distribuído.

Dentro do GitHub, os projetos podem ter tarefas atribuídas a pessoas específicas ou a equipes, com definições de permissões e funções por login - como a função de moderador do projeto, por exemplo.

Para quem é do mundo do desenvolvimento de códigos, o GitHub é uma boa oportunidade para aprimorar conhecimentos, compartilhar projetos e construir portfólio, muitas empresas a utilizam para realização de testes em processos seletivos, e consequentemente consulta para ver os projetos de seu candidato.

O GitHub é uma plataforma que contém uma versão gratuita e planos privados com funcionalidades adicionais. Para saber se a versão gratuita atende sua necessidade é preciso fazer um cadastro e testar. A pessoa não precisa ter conhecimentos em programação – afinal, o GitHub admite diversos tipos de arquivo. Até por isso, sua interface é muito amigável.

5.5.1. As principais funcionalidades da ferramenta são:

5.5.1.1. Repositório

O repositório é o local onde são armazenados os arquivos do projeto. Esse diretório pode ficar na GitHub ou em um dispositivo local. Ele admite arquivos de vários formatos, como imagem, áudio, o código-fonte, textos e qualquer outro tipo de documento necessário ao projeto.

5.5.1.2 *Branches*

Esse recurso é também chamado de “ramificações” e fica dentro do repositório. Aqui, a pessoa consegue promover alterações isoladamente, sem afetar a versão original, e pode ter uma visão de como as mudanças propostas se comportam, ao mesmo tempo em que pode comparar as versões.

As *branches* permitem desenvolver funcionalidades, consertar bugs e experimentar ideias de maneira segura.

5.5.1.3. Commits

Cada alteração promovida em um arquivo precisa ser executada e salva para se criar outra branch. O nome dessa ação é *Commit*.

5.5.1.4. Pull Requests

Quando profissionais de desenvolvimento chegam a uma versão considerada boa o suficiente, é possível criar um *pull request* para sugerir a implementação ao original. Nisso, as outras pessoas envolvidas no projeto vão poder analisar essa colaboração, podendo incorporá-las ou não.

As pessoas do projeto comparam o original e o que está na *branch* do *pull request*. Se for aprovada, é possível abrir uma solicitação de mesclagem (ou merge) das mudanças ao projeto original.

Para quem deseja ter uma visão mais abrangente de tudo o que o GitHub disponibiliza, recomendamos dar uma olhada no GitHub Docs.

Os impressionantes números do GitHub

- O GitHub é utilizado por mais de 3 milhões de profissionais de desenvolvimento brasileiros, sendo o terceiro maior público em número de logins.
- Em 2018, o GitHub foi comprado pela Microsoft pela cifra de US\$ 7,5 bilhões.
- O GitHub conta com mais de 100 milhões profissionais de desenvolvimento, mais de 4 milhões de organizações cadastradas e mais de 420 milhões de repositórios.
- Mais de 37 milhões de negócios já adotaram o Copilot GitHub. Desenvolvida pela OpenAI, o Copilot GitHub tem como objetivo de criar linhas de código automáticos ou por meio da solicitação de quem usa.
- O recurso dá suporte a milhares de linguagens, entre elas *Python*, *JavaScript*, *TypeScript*, *Ruby*, *Java* e *Go*, sendo bastante útil para resolver dúvidas ou criar soluções novas.

5.6. Git no Windows

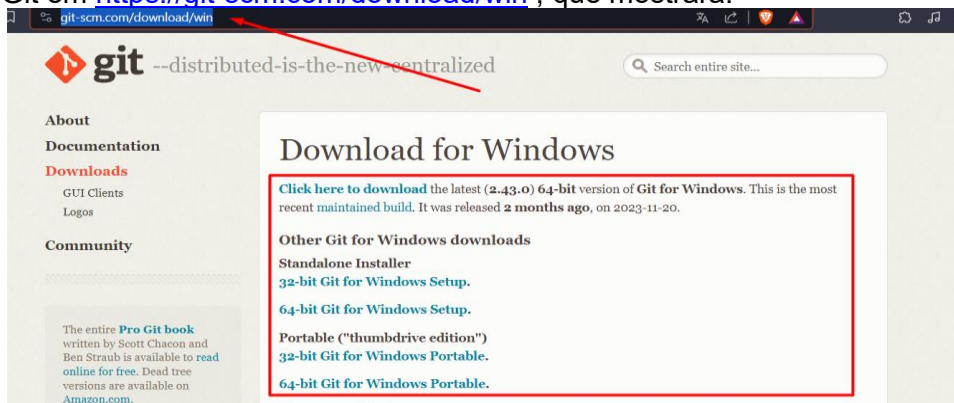
Podemos instalar o Git em outros sistemas operacionais, em nosso caso iremos trabalhar com o Windows, por isso, mostrarei um passo a passo para o mesmo, e também como podemos manipular o mesmo.

5.6.1. Ambiente para manuseio do Git

Temos duas formas de manipular o Git no *Windows*, através de IDE ou através de linha de comando, como estamos com foco em desenvolvimento de software a ideia aqui é utilizarmos prompt de comando, pois o mercado é muito acostumado com essa prática. Para *prompt* de comando, podemos utilizar o cmd do Windows, terminal que está embutido no *Visual Studio Code*, *PowerShell*, o *Cmder* também é muito utilizado.

5.6.2. Instalando o Git no Windows

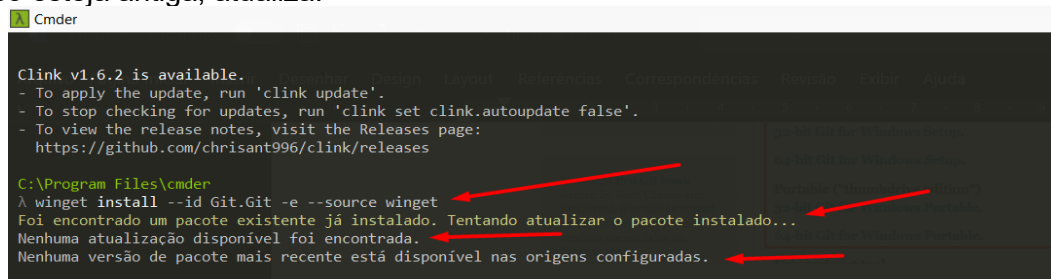
Temos duas formas de fazer a instalação, através do download diretamente no site do Git em <https://git-scm.com/download/win>, que mostrará:



Ou através de comando (winget - Windows Package Manager) que colocamos no prompt de comando escolhido:

```
winget install --id Git.Git -e --source winget
```

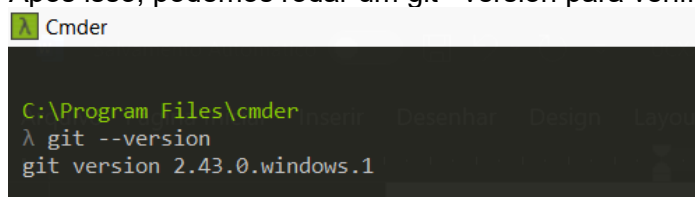
Caso tenha uma versão do Git instalada em seu computador, ele verifica a versão, e caso esteja antiga, atualiza:



```
C:\Program Files\cmdr
Clink v1.6.2 is available.
- To apply the update, run 'clink update'.
- To stop checking for updates, run 'clink set clink.autoupdate false'.
- To view the release notes, visit the Releases page:
  https://github.com/chrisant996/clink/releases

C:\Program Files\cmdr
λ winget install --id Git.Git -e --source winget
Foi encontrado um pacote existente já instalado. Tentando atualizar o pacote instalado...
Nenhuma atualização disponível foi encontrada.
Nenhuma versão de pacote mais recente está disponível nas origens configuradas.
```

Após isso, podemos rodar um `git --version` para verificar:



```
C:\Program Files\cmdr
λ git --version
git version 2.43.0.windows.1
```

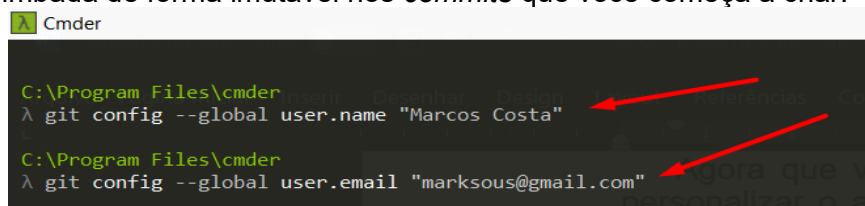
5.6.3. Configuração inicial do Git

Agora que você tem o Git em seu sistema, podemos fazer algumas coisas para personalizar o ambiente Git. Faremos apenas uma vez por computador e o efeito se manterá após atualizações. Mas também podemos mudá-las em qualquer momento rodando esses comandos novamente.

O Git vem com uma ferramenta chamada **git config** que permite ver e atribuir variáveis de configuração que controlam todos os aspectos de como o Git aparece e opera. No Windows, Git procura pelo arquivo `.gitconfig` no diretório `$HOME` (`C:\Users\$USER` para a maioria). Ele também procura por `/etc/gitconfig`, mesmo sendo relativo à raiz do sistema, que é onde quer que você tenha instalado Git no seu sistema.

5.6.4. Sua Identidade

A primeira coisa que devemos fazer ao instalar Git é configurar nosso nome de usuário e endereço de e-mail. Isto é importante porque cada *commit* usa esta informação, e ela é carimbada de forma imutável nos *commits* que você começa a criar:



```
C:\Program Files\cmdr
λ git config --global user.name "Marcos Costa"

C:\Program Files\cmdr
λ git config --global user.email "marksous@gmail.com"
```

Reiterando, você precisará fazer isso somente uma vez se tiver usado a opção `--global`, porque então o Git usará esta informação para qualquer coisa que você fizer naquele sistema. Se você quiser substituir essa informação com nome diferente para um projeto específico, você pode rodar o comando sem a opção `--global` dentro daquele projeto.

5.6.5. Seu Editor

Agora que a sua identidade está configurada, você pode escolher o editor de texto padrão que será chamado quando Git precisar que você entre uma mensagem. Se não

for configurado, o Git usará o editor padrão, que normalmente é o Vim. Em nosso caso, iremos configurar o *Visual Studio Code*, você pode fazer o seguinte:

Cmdr

```
C:\Program Files\cmdr
λ git config --global core.editor "code --wait"
```

5.6.6. Iniciando um repositório no Git

Para começarmos os trabalhos, devemos criar um repositório local de seu projeto, para isso, pense em seu computador, em um local onde ele ficará armazenado.

Para exemplificar, irei criar um repositório dentro da pasta htdocs do XAMPP, nesse caso para programas desenvolvidos com PHP, para isso utilizo o comando CD para “navegar” nas pastas em meu computador.

Cmdr

```
C:\Program Files\cmdr
λ cd\
C:\
λ cd xampp
C:\xampp
λ cd htdocs
```

Após isso, eu crio a pasta que representará meu repositório, o comando **mkdir** serve para criarmos a pasta, o **ls** serve para vermos o conteúdo do disco.

Cmdr

```
C:\xampp\htdocs
λ mkdir aula-git
C:\xampp\htdocs
λ ls
applications.html  aula-git/  compras/  estagio/  estagio_marcos/  estagio_old2809/
aula.php          bitnami.css  esp32/    estagio.zip  estagio_old/     estagio-20231009T142010Z-001.zip
```

Podemos observar que a pasta aula-git foi criada.

```
C:\xampp\htdocs
λ ls
applications.html  aula-git/  compras/  estagio/  estagio_marcos/
aula.php          bitnami.css  esp32/    estagio.zip  estagio_old/
```

Nesse momento nosso repositório foi criado, mas ainda não foi iniciado para isso precisamos entrar no repositório criado:

Cmdr

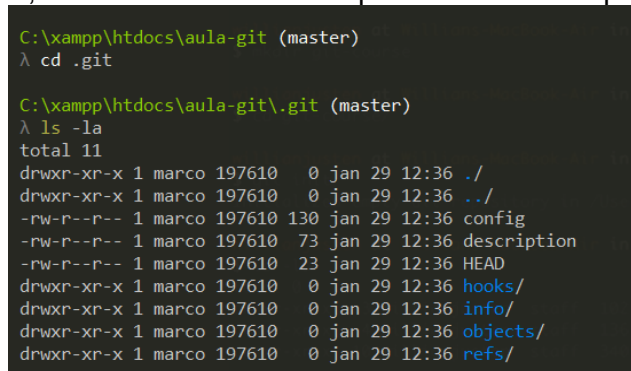
```
C:\xampp\htdocs
λ cd aula-git
C:\xampp\htdocs\aula-git
λ |
```

Agora iniciamos o repositório com o **git init**.



```
C:\xampp\htdocs> cd aula-git
C:\xampp\htdocs\aula-git> git init
Initialized empty Git repository in C:/xampp/htdocs/aula-git/.git/
C:\xampp\htdocs\aula-git (master)>
```

Observem que o Git iniciou o repositório na *branch master*, iremos falar disso mais tarde, mas vamos nos atentar que ele coloca uma pasta *.git*, se irmos até mesma teremos:



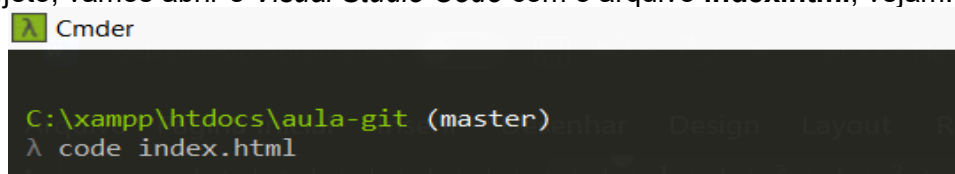
```
C:\xampp\htdocs\aula-git (master)> cd .git
C:\xampp\htdocs\aula-git\.git (master)> ls -la
total 11
drwxr-xr-x 1 marco 197610  0 jan 29 12:36 ./
drwxr-xr-x 1 marco 197610  0 jan 29 12:36 ../
-rw-r--r-- 1 marco 197610 130 jan 29 12:36 config
-rw-r--r-- 1 marco 197610  73 jan 29 12:36 description
-rw-r--r-- 1 marco 197610  23 jan 29 12:36 HEAD
drwxr-xr-x 1 marco 197610  0 jan 29 12:36 hooks/
drwxr-xr-x 1 marco 197610  0 jan 29 12:36 info/
drwxr-xr-x 1 marco 197610  0 jan 29 12:36 objects/
drwxr-xr-x 1 marco 197610  0 jan 29 12:36 refs/
```

Nesta pasta temos arquivos e pastas sobre a configuração necessária para que o Git trabalhe em nosso projeto.

Agora temos o repositório de nosso projeto criado e com o Git iniciado.

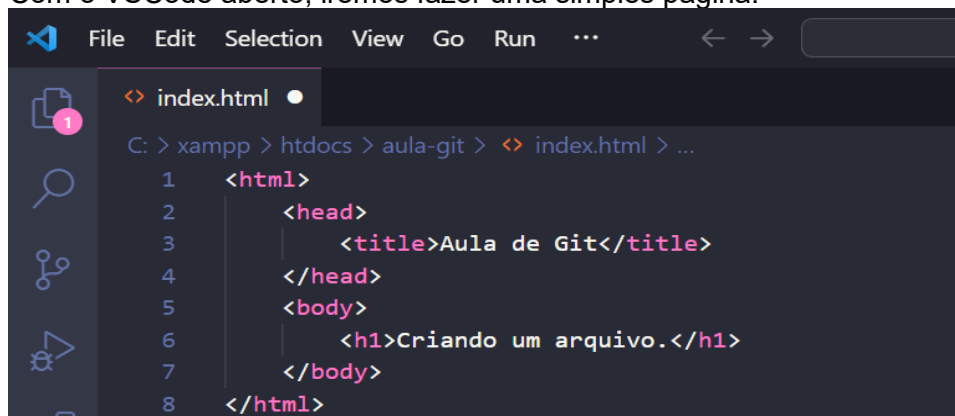
5.6.7. Criando arquivo e verificando a pasta

Agora vamos fazer um simples exemplo de uma página html para simularmos um projeto, vamos abrir o *Visual Studio Code* com o arquivo **index.html**, vejamos:



```
C:\xampp\htdocs\aula-git (master)> code index.html
```

Com o VSCode aberto, iremos fazer uma simples página.



```
File Edit Selection View Go Run ...
<> index.html
C: > xampp > htdocs > aula-git > <> index.html > ...
1  <html>
2    <head>
3      <title>Aula de Git</title>
4    </head>
5    <body>
6      <h1>Criando um arquivo.</h1>
7    </body>
8  </html>
```

Ao voltarmos ao prompt, com o comando **ls**, verificamos que o arquivo foi salvo corretamente.

```
C:\xampp\htdocs\aula-git (master)
λ ls
index.html
```

Agora temos que entender que o nosso arquivo está vinculado ao nosso computador, para mandarmos para um repositório na nuvem, devemos ter uma conta e o repositório criado por exemplo, no GitHub, com isso temos que vincular nosso repositório local com o remoto:

```
C:\xampp\htdocs\aula-git (master)
λ git remote add origin https://github.com/marccossousa33dev/aula_git.git
```

Apesar de já termos criado o arquivo index.html, vamos criar a branch para trabalharmos nosso arquivo, para isso vamos criá-la e já mudarmos para a mesma.

```
C:\xampp\htdocs\aula-git (master)
λ git checkout -b exemploAula
Switched to a new branch 'exemploAula'

C:\xampp\htdocs\aula-git (exemploAula)
λ
```

Agora vamos verificar o status desta Branch.

```
C:\xampp\htdocs\aula-git (exemploAula)
λ git status
On branch exemploAula

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html

nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\aula-git (exemploAula)
```

Somos informado que temos um arquivo que não está commitado e para que isso aconteça, ele informa que temos que usar o git add . ou git add index.html.

```
C:\xampp\htdocs\aula-git (exemploAula)
λ git add .

C:\xampp\htdocs\aula-git (exemploAula)
λ
```

Se repetirmos o comando git status, temos:

```
C:\xampp\htdocs\aula-git (exemploAula)
λ git status
On branch exemploAula

No commits yet

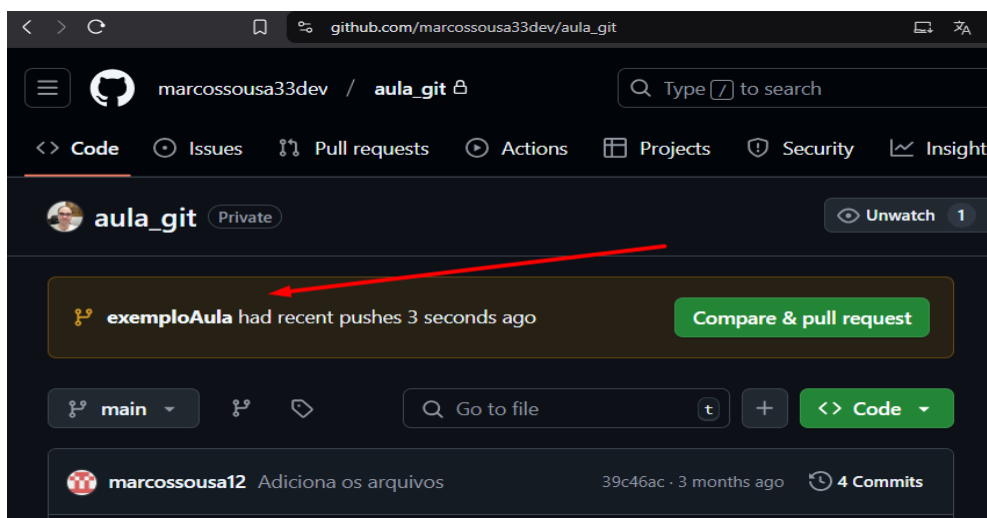
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   index.html
```

Temos um arquivo para ser commitado.

```
C:\xampp\htdocs\aula-git (exemploAula)
λ git commit -m "Aplicando exemplo de commit"
[exemploAula (root-commit) 6b2cca7] Aplicando exemplo de commit
1 file changed, 8 insertions(+)
create mode 100644 index.html
```

Neste momento nosso arquivo está pronto para ser enviado ao repositório remoto

```
C:\xampp\htdocs\aula-git (exemploAula)
λ git push git@github.com:marcossousa33dev/aula_git.git
Enter passphrase for key '/c/Users/marco/.ssh/id_rsa':
Enter passphrase for key '/c/Users/marco/.ssh/id_rsa':
Enter passphrase for key '/c/Users/marco/.ssh/id_rsa':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 315 bytes | 315.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'exemploAula' on GitHub by visiting:
remote:   https://github.com/marcossousa33dev/aula_git/pull/new/exemploAula
remote:
To github.com:marcossousa33dev/aula_git
 * [new branch]      exemploAula -> exemploAula
```



Nosso arquivo agora está no repositório remoto, agora só precisamos fazer o Pull Request e colocar na main.

Resumindo, abaixo os principais comandos, lembrando que devemos praticar para fixarmos melhor o conteúdo:

Configuração Inicial

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@example.com"
```

Criar um Repositório

```
git init
```

Clonar um Repositório

```
git clone <url-do-repositorio>
```

Verificar o Status do Repositório

```
git status
```

Adicionar Arquivos ao Índice (Staging Area)

```
git add <arquivo>
git add .
```

Fazer um Commit

```
git commit -m "Mensagem do commit"
```

Verificar o Histórico de Commits

```
git log
```

Enviar Alterações para o Repositório Remoto

```
git push origin <nome-da-branch>
```

Atualizar o Repositório Local com Alterações do Remoto

```
git pull
```

Criar uma Nova Branch

```
git checkout -b <nome-da-branch>
```

Trocar para uma Branch Existente

```
git checkout <nome-da-branch>
```

Verificar Branches Existentes

```
git Branch
```

Excluir uma Branch

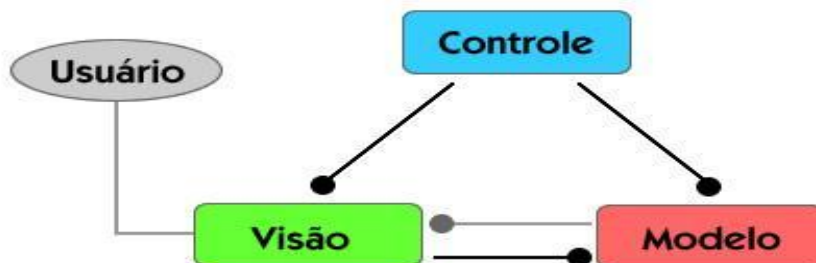
```
git branch -d <nome-da-branch>
```

5. Padrão MVC

MVC é o acrônimo de *Model-View-Controller* (em português, Modelo-Visão-Controle) e que, como o próprio nome supõe, separa as camadas de uma aplicação em diferentes níveis. Ao contrário do que muitos desenvolvedores dizem, o MVC não é um Padrão de Projeto, mas sim um Padrão de Arquitetura de Software.

O MVC define a divisão de uma aplicação em três componentes: Modelo, Visão e Controle. Cada um destes componentes tem uma função específica e estão conectados entre si. O objetivo é separar a arquitetura do software para facilitar a compreensão e a manutenção.

A camada de Visão é relacionada ao visual da aplicação, ou seja, as telas que serão exibidas para o usuário. Nessa camada apenas os recursos visuais devem ser implementados, como janelas, botões e mensagens. Já a camada de Controle atua como intermediária entre as regras de negócio (camada Modelo) e a Visão, realizando o processamento de dados informados pelo usuário e repassando-os para as outras camadas. E por fim, temos a camada de Modelo, que consiste na essência das regras de negócio, envolvendo as classes do sistema e o acesso aos dados. Observe que cada camada tem uma tarefa distinta dentro do software.

**5.1 Explicação do conceito do MVC****5.1.1 Certo, mas qual é a vantagem?**

A princípio, pode-se dizer que o sistema fica mais organizado quando está estruturado com MVC. Um novo desenvolvedor que começar a trabalhar no projeto não terá grandes dificuldades em compreender a estrutura do código. Além disso, as exceções são mais fáceis de serem identificadas e tratadas. Ao invés de revirar o código atrás do erro, o MVC pode indicar em qual camada o erro ocorre. Outro ponto importante do MVC é a segurança da transição de dados. Através da camada de Controle, é possível evitar que qualquer dado inconsistente chegue à camada de Modelo para persistir no banco de dados. Imagine a camada de Controle como uma espécie de Firewall: o usuário entra com os dados e a camada de Controle se responsabiliza por bloquear os dados que venham a causar inconsistências no banco de dados. Portanto, é correto afirmar que essa camada é muito importante.

5.1.2 Posso ter apenas três camadas no MVC?

Eis que essa é outra dúvida bastante questionada pelos desenvolvedores. Embora o MVC sugira a organização do sistema em três camadas, nada impede que você “estenda” essas camadas para expandir a dimensão do projeto. Por exemplo, vamos supor que um determinado

projeto tenha um módulo web para consulta de dados. Aonde esse módulo se encaixaria dentro das três camadas?

Bem, este módulo poderia ser agrupado com os outros objetos da camada Visão, mas seria bem mais conveniente criar uma camada exclusiva (como “Web”) estendida da camada de Visão e agrupar todos os itens relacionados a esse módulo dentro dela. O mesmo funcionaria para um módulo Mobile ou Webservice. Neste caso, a nível de abstração, essas novas camadas estariam dentro da camada de Visão, mas são unidades estendidas.

5.1.3 Então podemos concluir resumidamente

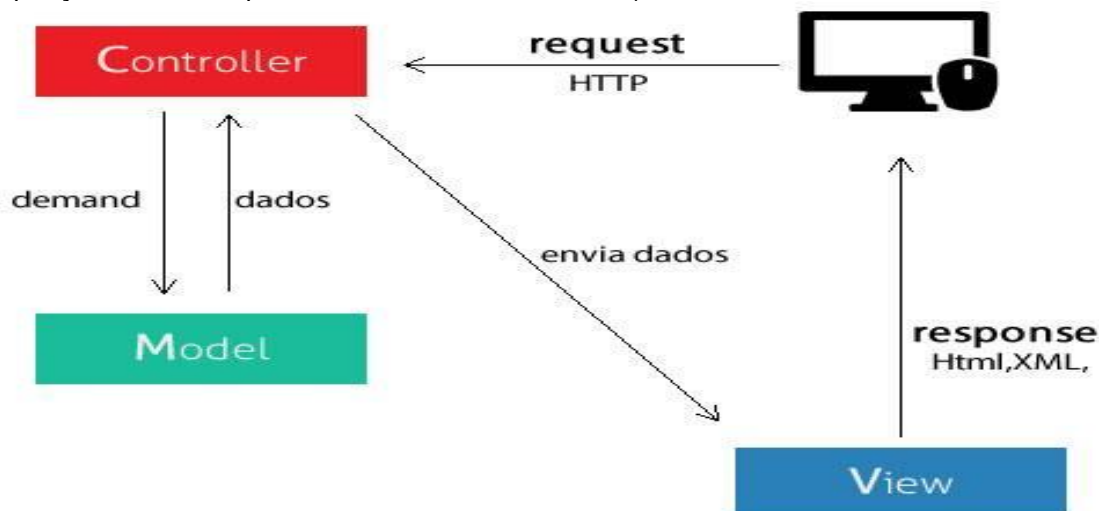
MVC é nada mais que um padrão de arquitetura de software, separando sua aplicação em 3 camadas. A camada de interação do usuário (*view*), a camada de manipulação dos dados(*model*) e a camada de controle(*controller*).

Model: quando você pensar em manipulação de dados, pense em model. Ele é responsável pela leitura e escrita de dados, e também de suas validações.

View: é a camada de interação com o usuário. Ela apenas faz a exibição dos dados.

Controller: é o responsável por receber todas as requisições do usuário. Seus métodos chamados *actions* são responsáveis por uma página, controlando qual *model* usar e qual *view* será mostrado ao usuário.

(A imagem a seguir representa o fluxo do MVC em um contexto de Internet, com uma requisição HTTP e resposta em formato HTML ou XML)



5.1.4 O diálogo das camadas na Web (Brincadeirainha)

View - Fala **Controller**! O usuário acabou de pedir para acessar o Facebook! Pega os dados de *login* dele aí.

Controller – Blz. Já te mando a resposta. Aí *model*, meu parceiro, toma esses dados de *login* e verifica se ele loga.

Model – Os dados são válidos. Mandando a resposta de login.

Controller – Blz. **View**, o usuário informou os dados corretos. Vou mandar pra vc os dados dele e você carrega a página de perfil.

View – Vlww. Mostrando ao usuário...

5.2 API: conceito, exemplos de uso e importância da integração para desenvolvedores

A integração a APIs pode facilitar demais o trabalho de quem trabalha com desenvolvimento. Entenda melhor o conceito de API e veja exemplos de como elas funcionam.

A sigla API é uma abreviação para *Application Programming Interface*, ou, em português, interface de programação de aplicação.

5.2.1 API: Conceito

Em uma definição formal, o conceito de API está relacionado a um conjunto de rotinas e padrões estabelecidos por um software para a utilização das suas funcionalidades por outros aplicativos.

O conceito de API nada mais é do que uma forma de comunicação entre sistemas. Elas permitem a integração entre dois sistemas, em que um deles fornece informações e serviços que podem ser utilizados pelo outro, sem a necessidade de o sistema que consome a API conhecer detalhes de implementação do software.

Metaforicamente, podemos pensar no que é API como um garçom. Quando estamos em um restaurante, buscamos o que desejamos no menu e solicitamos ao garçom. O garçom encaminha esse pedido à cozinha, que prepara o pedido. No fim, o garçom traz o prato pronto até a gente. Não temos detalhes de como esse prato foi preparado, apenas recebemos o que solicitamos.

Com os exemplos de API disponíveis, a ideia é a mesma do garçom. Ela vai receber seu pedido, levar até o sistema responsável pelo tratamento e te devolver o que solicitou (o que pode ser uma informação, ou o resultado do processamento de alguma tarefa, por exemplo).

5.2.2 A API de integração são grandes aliadas dos *devs* e das empresas.

Importância das APIs:

Atualmente, quando temos um mundo tão conectado, as APIs são essenciais para a entrega de produtos cada vez mais ricos para os usuários.

Independente do produto que se deseja oferecer, seja ele um site, um aplicativo, um *bot*, é muito importante a utilização do conceito de APIs integradas a sistemas para trazer uma gama de funcionalidades para sua aplicação.

Podemos pensar na utilidade do que é API por dois pontos de vista: como produtor ou consumidor.

Quando você produz, você está criando APIs para que outras aplicações ou sistemas possam se integrar ao seu sistema. Isso não significa que você irá criar uma API apenas para expor seu sistema a terceiros. Vai depender do seu propósito, mas a API também pode ser apenas para seu uso, como de uma interface sua para os clientes.

Seguindo nessa linha de raciocínio, imagine que você possua um sistema de comércio. Ao criar uma API de acesso ao seu sistema, você possibilita a oferta de seus produtos nas interfaces de seu interesse. O que você vai expor de seu sistema na API depende do que deseja oferecer como funcionalidade, mas poderia, por exemplo:

- Listar produtos;
- Ofertar promoções;
- Efetivar vendas;
- Realizar cobrança do pedido.

Algumas aplicações do conceito de API

Até aqui, exploramos o que é API e as possibilidades de você criar a API para facilitar a integração com aplicações.

No entanto, hoje em dia já possuímos uma quantidade enormes de APIs que podemos utilizar para enriquecer nossas aplicações, e são de todos os tipos. Vamos citar um exemplo de verificação de CEP.

Em nosso exemplo vamos utilizar o ViaCep que é um webservice que serve para consulta de endereços no Brasil, ele é gratuito, para consultar o cep basta fazermos uma requisição *http* para a API do ViaCep, e então obter o retorno com informações como CEP, nome da cidade, código do município, UF e mais.

Para verificar como fazemos essa parte, o fornecedora da API pode disponibilizar a documentação da mesma, nesse caso acessando a url: <https://viacep.com.br/> nos é relatado como fazemos a consulta dos dados.

Para esse exemplo e vermos o retorno dos dados, vamos utilizar o *INSOMNIA* (<https://insomnia.rest/>) para testarmos a requisição, que é uma ferramenta popular de *API Client* usada para testar, enviar e depurar requisições HTTP e APIs REST, e que também utilizaremos em nossas aulas, que serve para testarmos através de métodos *POST* nossos *ENDPOINTS* para nossas APIs.

Passos:

1º - Criamos um nome para nossa requisição;

2º - Passamos a URL da API de acordo com a documentação vista no ViaCep.

Validação do CEP

Quando consultado um CEP de formato inválido, por exemplo: "950100100" (9 dígitos), "95010A10" (alfanumérico), "95010 10" (espaço), o código de retorno da consulta será um 400 (Bad Request). Antes de acessar o webservice, valide o formato do CEP e certifique-se que o mesmo possua {8} dígitos. Exemplo de como validar o formato do CEP em javascript está disponível nos exemplos abaixo.

Quando consultado um CEP de formato válido, porém inexistente, por exemplo: "99999999", o retorno conterá um valor de "erro" igual a "true". Isso significa que o CEP consultado não foi encontrado na base de dados. Veja como manipular este "erro" em javascript nos exemplos abaixo.

Formatos de Retorno

Veja exemplos de acesso ao webservice e os diferentes tipos de retorno:

JSON

URL: viacep.com.br/ws/01001000/json/
 UNICODE: viacep.com.br/ws/01001000/json/unicode/

```
{
  "cep": "01001-000",
  "logradouro": "Praça da Sé",
  "complemento": "lado ímpar",
  "bairro": "Sé",
  "localidade": "São Paulo",
  "uf": "SP",
  "ibge": "3550308",
  "gia": "1004",
  "ddd": "11",
  "siafi": "7107"
}
```

3º - Enviamos a requisição;

4º - Retorno da API no formato JSON

Aula_PW_II GET <https://viacep.com.br/ws/06756030/json/> Send 200 OK 424 ms 252 B

desenv Cookies Body Auth Query Header Docs Preview Header 14 Cookie Timeline

Filter

Testes para aula

GET API CEP

Aula_Etec

POST API Login

```
1 {
2   "cep": "06756-030",
3   "logradouro": "Rua Marino Martins de Oliveira",
4   "complemento": "",
5   "bairro": "Parque Monte Alegre",
6   "localidade": "Taboão da Serra",
7   "uf": "SP",
8   "ibge": "3552809",
9   "gia": "6750",
10  "ddd": "11",
11  "siafi": "7157"
12 }
```

5.2.3 API REST

Representational State Transfer (REST), ou Transferência de Estado Representacional. Lá vem a Ciência da Computação, com suas siglas e nomes complexos.

Vamos simplificar um pouco. *API REST* é oriundo de uma dissertação de doutorado de Roy Fielding, nos anos 2000.

De maneira bem resumida, ele orienta a usar o protocolo HTTP para representar o estado/operação sobre a informação.

Em suma, o que precisamos saber é *API REST* é um serviço sem estado (*cookies* ou *session*) que utiliza corretamente os verbos HTTP:

- **POST**: para criar um objeto no servidor
- **DELETE**: para apagar um objeto do servidor.
- **PUT**: para atualizar um objeto no servidor.
- **GET**: para obter um ou mais objetos do servidor.

Ou seja, uma série de instruções que permitem a criação de um serviço com uma interface bem definida. Para isso, há uma série de recomendações com relação a rotas das *URLs*, dentre outras coisas.

Quando os princípios listados nessa recomendação são adotados em sua totalidade, temos uma *API RESTful*.

5.3 JSON

Vamos trabalhar em nosso projeto a passagem e retorno dos dados pelas *controllers* em formato *JSON*, mas afinal... o que é isso?

JSON é um formato de representação de dados baseado na linguagem de programação *JavaScript*, daí o nome *JavaScript Object Notation*. Ou seja, Notação de Objeto em Javascript.

Vamos pensar no exemplo de um objeto pessoa com nome Pedro e altura 1,90. A representação deste objeto em *JSON* ficaria assim:

```
{  
  "nome": "Pedro",  
  "altura": 1.90  
}
```

Este é um exemplo bem simples, mas podemos observar que o *JSON* não vai muito além disso.

O mais importante que você precisa saber sobre o *JSON* é que ele é composto por chave/valor (ou no inglês *key/value*), as chaves representam os nomes dos atributos da classe e os valores, bem, são os valores do objeto.

No exemplo acima, temos as chaves nome e altura e os valores Pedro e 1.90, respectivamente.

5.3.1 A sintaxe básica do JSON

A sintaxe de um *JSON* é bem simples como já falamos, mas agora vamos aprofundar um pouco mais.

Vejamos os elementos básicos do *JSON*.

{ e } - delimita um objeto.

[e] - delimita um *array*.

: - separa chaves (atributos) de valores.

, - separa os atributos chave/valor.

Entendendo estes quatro elementos básicos você já será capaz de ler um *JSON* com mais facilidade.

A leitura simples de um *JSON* é que ele representa um objeto que tem atributos e cada atributo tem valores.

Os *JSONs* são estruturados em objetos e/ou *arrays* (ou listas). Os objetos são representados por chaves { } e os *arrays* são representados por colchetes [].

Essa é a primeira coisa que você deve olhar no *JSON*: Onde estão as chaves e os colchetes?

Identificando estes dois tipos de estruturas você já sabe se os dados serão acessados por chaves (quando for um objeto), ou por índices/números (quando for um *array*).

Além disso, tem algumas outras regras, nos objetos os atributos devem seguir de um caractere : (dois-pontos) e o valor do atributo. E os atributos devem ser separados por vírgulas.

Já os *arrays* só podem ser de um determinado tipo de dados (veja a próxima sessão), não pode misturar texto com número, por exemplo.

>> O que são Vetores e Matrizes (*arrays*)

Veja abaixo um exemplo de objeto e um exemplo de *array*.

```
{
  "atributo1": "valor1",
  "atributo2": 2,
  "atributo3": true
}
```

[2,4,5,6]

Outro ponto interessante é que um objeto *JSON* pode ter atributos do tipo *array* e um *array* pode ser do tipo objeto ou *array*. Confuso? Veja o exemplo abaixo para entender melhor.

```
{
  "atributoDoTipoArray" : [1,2,3,54]
}
```

```
{{
  "a":1
},{
  "b":1
}}
```

Uma observação é que tanto *array* quanto objeto podem ser vazios em *JSON*. Assim:

```
{
  }
```

5.3.2 Os tipos de dados do *JSON*

Além de objeto e *array* serem considerados os tipos de dados principais. O *JSON* também tem os tipos de dados primitivos que nós já falamos aqui no { Dicas de Programação }.

Os tipos de dados básicos do *JSON* são:

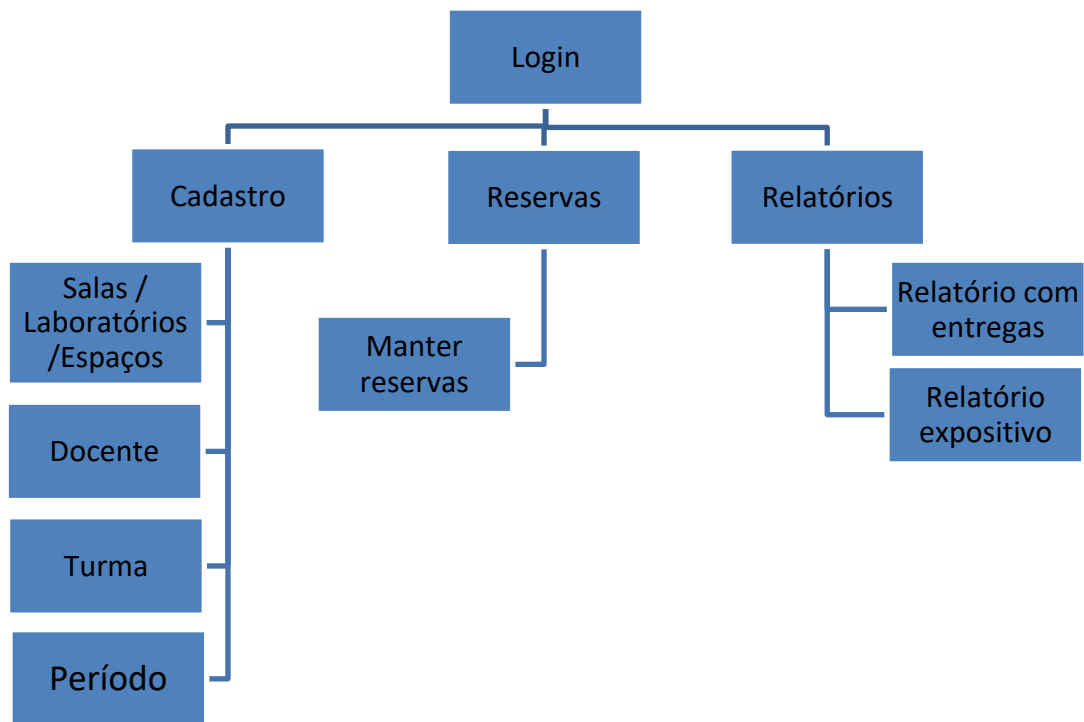
- **String** - separados por aspas (duplas ou simples). Ex. "Brasil" ou 'Brasil'
- **Número** - sem aspas e pode ser inteiro ou real, quando for do tipo real deve-se usar o caractere . (ponto) para separar a parte inteira das casas decimais. Ex. 1 (inteiro) ou 23.454 (real)
- **Booleano** - tipo lógico normal, pode assumir valores *true* ou *false*.
- **Nulo** - este é o valor nulo mesmo. Ex. { "nome" : *null* }

Veja abaixo um exemplo de objeto *JSON* com todos estes tipos de dados.

```
{
  "texto" : "Brasil",
  "numero" : 23,
  "numeroReal" : 54.87,
  "booleano": true,
  "nulo": null
}
```

6. Mãos à obra

Em nossas aulas utilizaremos a práxis (prática), então a ideia agora, é montarmos um sistema de reservas de salas de aula ou laboratórios, onde a ideia é que qualquer instituição que possua salas de aula, laboratórios, ou espaços que possam ser reservados de acordo com datas e horários desejados, identificamos esta práxis como necessidade em muitas instituições de ensino que terá a seguinte decomposição funcional de acordo com a ideia:



A cada aula nosso objetivo é montarmos a estrutura de cada módulo da ideia acima, portanto a frequência nas aulas é de suma importância para o bom andamento.

6.1 Banco de dados

Com a regra de negócio estabelecida, o banco de dados foi confeccionado, lembrando que podemos no decorrer de nossa aplicação, identificar novas necessidades, ou muitas vezes, considerações do cliente que deverão ser consideradas na estrutura das tabelas ou até mesmo na estrutura do banco de dados, assim ficou a estrutura de tabelas e relações:

```

1      #CRIAR A BASE DE DADOS
2      • create database mapa;
3
4      #SELECIONAR O BANCO DE DADOS
5      • use mapa;
6
7      #CRIANDO A TABELA DE USUARIOS
8      • create table tbl_usuario(
9          id_usuario integer not null auto_increment primary key,
10         nome        varchar(50),
11         usuario     varchar(15),
12         senha       varchar(32),
13         email       varchar(80),
14         dtcria      datetime default now(),
15         estatus     char(01) default ''
16     );
17

```



```

18      #CRIAR A TABELA DE CADASTRO DA SALA
19 • ○ create table tbl_sala(
20         codigo      integer primary key,
21         descricao   varchar(30) default '',
22         andar       integer,
23         capacidade   integer,
24         dtcria       datetime default now(),
25         estatus      char(01) default ''
26     );
27
28      #CRIAR A TABELA DE CADASTRO DE PROFESSOR
29 • ○ create table tbl_professor(
30         codigo      integer auto_increment primary key,
31         nome        varchar(30) default '',
32         cpf         varchar(11) default '',
33         tipo        char(1) default 'F',
34         dtcria       datetime default now(),
35         estatus      char(01) default ''
36     );
37
38      #CRIAR A TABELA DE CADASTRO DE TURMA
39 • ○ create table tbl_turma(
40         codigo      integer auto_increment primary key,
41         descricao   varchar(50) default '',
42         capacidade   integer default 0,
43         dataInicio   date,
44         dtcria       datetime default now(),
45         estatus      char(01) default ''
46     );
47
48      #CRIAR A TABELA DE CADASTRO DE HORÁRIOS
49 • ○ create table tbl_horario(
50         codigo      integer auto_increment primary key,
51         descricao   varchar(50) default '',
52         hora_ini     time,
53         hora_fim     time,
54         dtcria       datetime default now(),
55         estatus      char(01) default ''
56     );
57
58      #CRIAR TABELA DE MAPEAMENTO DE SALA
59 • ○ create table tbl_mapa(
60         codigo      integer auto_increment primary key,

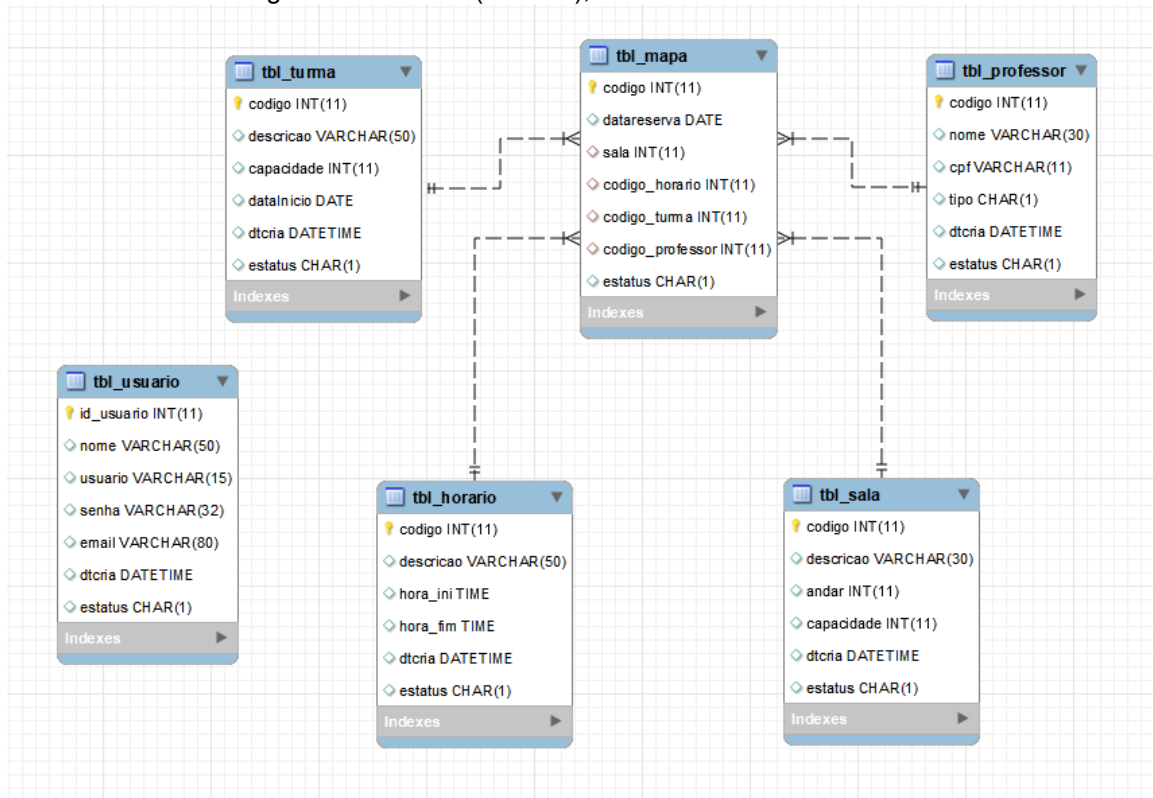
```

```

61      datareserva      date,
62      sala              integer default 0,
63      codigo_horario    integer default 0,
64      codigo_turma      integer default 0,
65      codigo_professor  integer default 0,
66      estatus char(01) default'',
67
68      foreign key (sala) references tbl_sala(codigo),
69      foreign key (codigo_horario) references tbl_horario(codigo),
70      foreign key (codigo_turma) references tbl_turma(codigo),
71      foreign key (codigo_professor) references tbl_professor(codigo)
72  );

```

Fazendo a engenharia reversa (Ctrl + R), a estrutura ficou assim:



Sendo assim, confeccionem este banco de dados, lembre-se de guardar o script do mesmo para que possa utilizar quando necessário, recomendo que utilizem o GitHub para isso.

6.2 Introdução ao CodeIgniter

O *CodeIgniter* (ou CI, como muitos chamam, e nós também) é um *framework MVC open source*, escrito em PHP e mantido atualmente pelo *British Columbia Institute of Technology* e por uma grande comunidade de desenvolvedores ao redor do mundo. Sua simplicidade faz dele um dos mais utilizados e com uma curva de aprendizado bem pequena.

Sua documentação é bem completa e detalhada, facilitando o processo de pesquisa sobre determinada biblioteca ou funcionalidade. Com isso, o tempo gasto com a leitura da documentação diminui, e você pode passar mais tempo trabalhando no código do seu projeto. Com o CI, é possível desenvolver sites, APIs e sistemas das mais diversas complexidades, tudo de forma otimizada, organizada e rápida. Suas bibliotecas nativas facilitam ainda mais o processo de

desenvolvimento, e ainda permitem ser estendidas para que o funcionamento se adapte à necessidade de cada projeto. Diversas bibliotecas de terceiros (*third-party*) estão disponíveis no *GitHub*, *Composer* e em outros repositórios de arquivos, e podem ser muito úteis.

Mais detalhes sobre o CI podem ser vistos no site do *framework* (<https://codeigniter.com>) e em sua documentação (https://codeigniter.com/user_guide).

6.2.1. Requisitos mínimos

Para um melhor aproveitamento, é recomendado o uso de PHP 5.4 ou mais recente. Ele até funciona com PHP 5.2, mas é recomendado que você não utilize versões antigas do PHP, por questões de segurança e desempenho potenciais, bem como recursos ausentes.

Algumas aplicações fazem uso de banco de dados, e o *CodeIgniter* suporta atualmente:

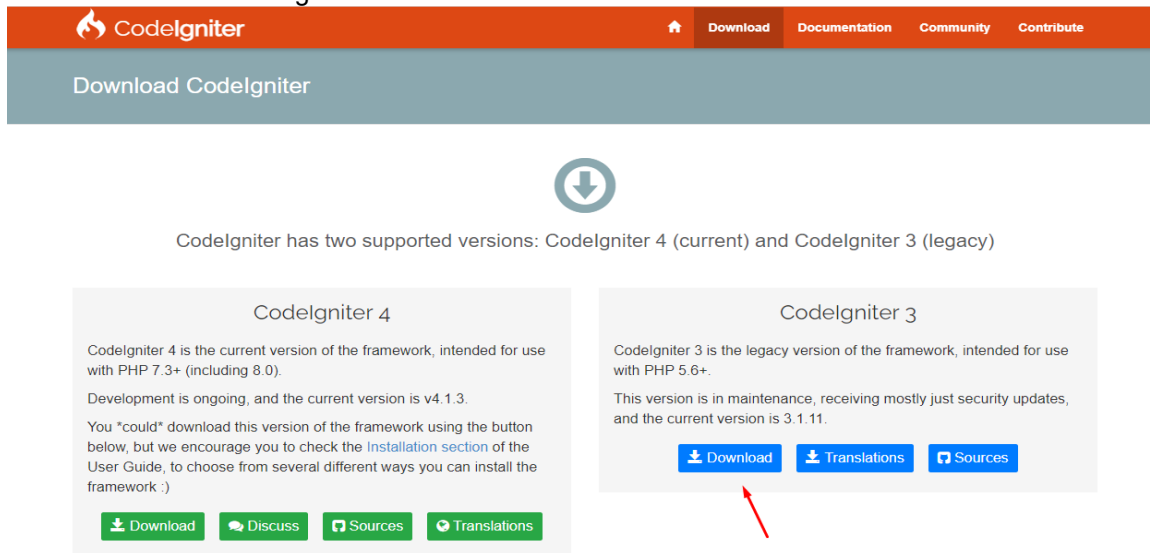
- MySQL(5.1+), mysqli e PDO drivers
- Oracle (drivers oci8 e PDO)
- PostgreSQL (postgre a PDO)
- MSSQL (mssql, sqlsrv e PDO)
- Interbase/Firebird (ibase e PDO)
- ODBC (odbc e PDO)

O CI pode ser instalado em sistemas operacionais UNIX (Linux e Mac OS X) e Windows, desde que o ambiente de desenvolvimento com Apache ou Nginx (UNIX) e IIS (Windows) estejam devidamente montados e funcionando.

Para nossas aulas trabalharemos com a versão do *CodeIgniter* 3, você pode fazer o *download* em <https://codeigniter.com/download> , mas irei disponibilizar a estrutura dele já pronta para nossas aulas no Github.

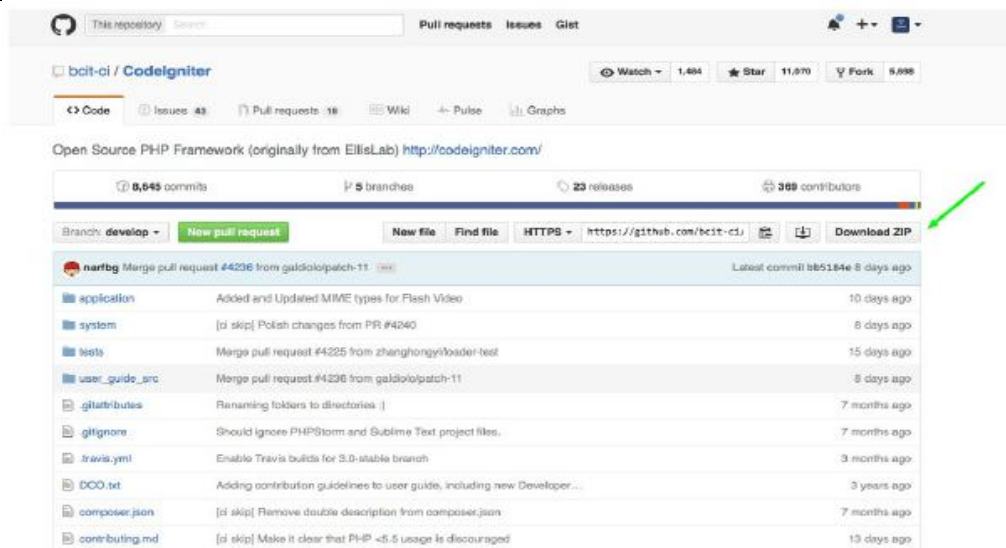
6.2.2. Instalando o *CodeIgniter*

O processo de instalação do CI é muito simples e fácil de ser executado. O primeiro passo é fazer o *download* do *framework* em nosso caso irei subir o arquivo no *Microsoft Teams* para download. Para isso, você tem duas possibilidades: Clique no link que passei acima, e faça conforme mostrado na figura:



Nossas aulas nesse momento estão baseadas no *CodeIgniter* 3.

Acesse o repositório do projeto no GitHub (<https://github.com/bcit-ci/CodeIgniter>) e faça o download clicando em Download ZIP, conforme a figura a seguir:



6.2.3 Estrutura de arquivos e diretórios do *CodeIgniter*

Agora que o *CodeIgniter* já está instalado e funcionando, mostrarei com mais detalhes a estrutura de arquivos e diretórios. É importante conhecê-la para que possa organizar o seu código da melhor maneira possível, e fazer o uso correto dos recursos que o CI disponibiliza.

Diretório *application*

Esse é o diretório da aplicação, onde ficarão todos os arquivos relacionados ao projeto.

Ele é composto de 12 diretórios, que farão com que os arquivos do projeto fiquem bem divididos e separados dos arquivos do "core" do CI. Assim, você poderá atualizar a versão do CI sem ter de mudar os arquivos do projeto de diretório ou estrutura.

- **cache** — Diretório que armazena os arquivos que são colocados em cache.
- **config** — Diretório que armazena os arquivos de configuração do CI, como por exemplo, `database.php`, `constants.php`, `routes.php`, entre outros que veremos no decorrer das aulas.
- **controllers** — Diretório que armazena os arquivos com os *controllers* do projeto. Ao instalar o CI, ele já vem com um *controller* criado, que é o *Welcome.php*.
- **core** — Diretório usado para estender classes e funcionalidades do core do CI, adaptando-se às necessidades do projeto.
- **helpers** — Diretório que armazena os arquivos com funções que funcionarão como assistentes durante o desenvolvimento. Por exemplo, você pode criar um helper (assistente) para determinado grupo de tarefas realizadas com frequência dentro do projeto, ou então estender as funcionalidades dos *helpers* nativos do CI.
- **hooks** — Diretório que armazena os arquivos que também estendem funcionalidades padrão do CI. Você pode criar um hook para alterar uma funcionalidade padrão, como por exemplo, minificar o código HTML de saída quando o método `load->view()` for executado.
- **language** — Diretório que armazena os arquivos com os dicionários de idiomas, assim você pode desenvolver projetos multi-idíomas de forma fácil, e também utilizar os outros idiomas dos pacotes de tradução oficiais do CI, que podem ser encontrados em <https://github.com/bcit-ci/codeigniter3-translations>.
- **libraries** — Diretório que armazena as *libraries* (bibliotecas) criadas para o projeto, ou *libraries* estendidas do core do CI.
- **logs** — Diretório que armazena os arquivos de log, que são configurados em `application/config/config.php`.
- **models** — Diretório que armazena os arquivos onde ficarão as regras de negócio do projeto.
- **third_party** — Diretório que armazena código de terceiros, como classes que podem auxiliar em alguma rotina do projeto e que foram desenvolvidas por outros programadores e/ou não possuem os padrões do CI.

- **views** — Diretório que armazena arquivos que serão carregados no *browser*. Você pode inserir outros diretórios dentro dele para organizar os arquivos da melhor maneira possível.

Nem sempre você fará uso de todos os diretórios dentro de *application*, os mais utilizados são: **config**, **controllers**, **libraries**, **logs**, **models** e **views**. Mas o uso fica a critério do desenvolvedor e da necessidade do projeto.

Diretório system

Esse diretório armazena o *core* do CI, e o conteúdo dos arquivos contidos nesse diretório não devem ser alterados. Isso porque, para manter o *CodeIgniter* atualizado, na maioria das versões basta substituir o conteúdo desse diretório pelo conteúdo do mesmo diretório na versão mais recente.

Ele possui apenas seis outros diretórios, muito bem divididos, e com os arquivos nomeados de forma bem intuitiva. Assim, caso queira estudar mais a fundo o funcionamento do CI, você pode fazê-lo analisando o código-fonte dos arquivos desse diretório.

Arquivo index.php

O arquivo *index.php* é o arquivo base de um projeto feito com CI. Ele carrega o arquivo de *core* necessário para a carga das *libraries*, *helpers*, classes, entre outros arquivos do framework e execução do código escrito por você.

Nesse arquivo, você pode alterar a localização dos diretórios *system* e *application*, configurar as exibições de erro para os ambientes do projeto (desenvolvimento, teste e produção, por exemplo), customizar valores de configurações, entre outras possibilidades. Quase não se faz alteração nesse arquivo, mas durante as aulas serão feitas algumas, para que possam atender aos requisitos dos exemplos.

6.3 Vamos codificar

Nesse momento iremos começar a codificar em PHP com o *Framework CodeIgniter*, para isso vamos colocar nosso projeto na pasta base do htdocs do XAMP, irei fornecer a base do projeto que estará com todos os arquivos necessários para a base de nosso projeto, inclusive o *.htaccess*.

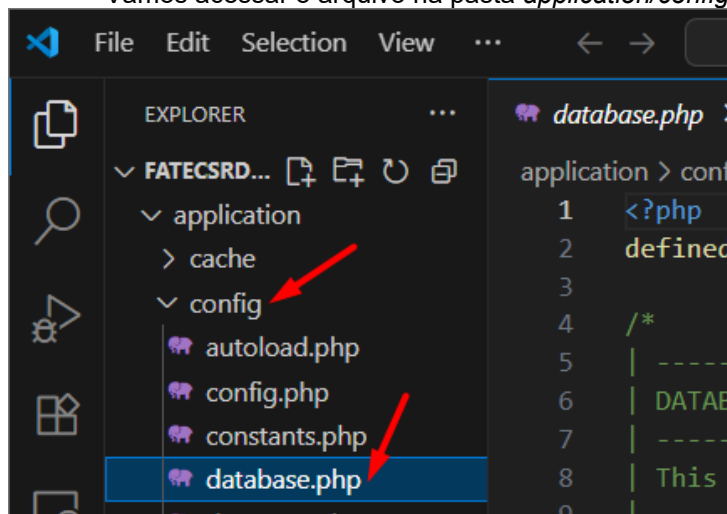
6.3.1 Configurando o CodeIgniter para nosso projeto

Nesse momento teremos que configurar nosso *framework* para a estrutura de nosso projeto, iremos configurar nesse alterando os seguintes arquivos na nossa estrutura:

- *database.php* (configuraremos o banco de dados de estágio);
- *autoload.php* (configuraremos a chamada automática do banco de dados);
- *routes.php* (configuraremos a rota de nossa *controller* inicial do projeto);
- *config.php* (configuraremos nossa *base_url()*);
- *htaccess* (URLs amigáveis).

6.3.2 Configurando o arquivo *databases.php*

Vamos acessar o arquivo na pasta *application/config*, assim:

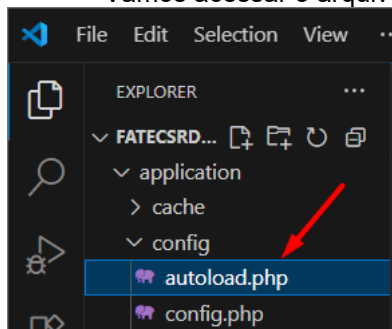


Informaremos os dados de configuração, da seguinte forma:

```
database.php X
application > config > database.php > ...
65 | a lot of SQL queries ... disable this to avoid that problem.
66 |
67 | The $active_group variable lets you choose which connection group to
68 | make active. By default there is only one group (the 'default' group).
69 |
70 | The $query_builder variable lets you determine whether or not to load
71 | the query builder class.
72 | */
73 | $active_group = 'default';
74 | $query_builder = TRUE;
75 |
76 | $db['default'] = array(
77 |     'dsn' => 'BD_Mapa', //Nome da conexão
78 |     'hostname' => 'localhost:3306', //Servidor onde está o banco de dados
79 |     'username' => 'root', //Usuário do banco de dados
80 |     'password' => '', //Caso possua, a senha do banco de dados
81 |     'database' => 'mapa', //Nome do banco de dados criado
82 |     'dbdriver' => 'mysqli', //Driver do banco de dados, iremos utilizar
83 |     //esse por estarmos trabalhando com o Banco MySQL
84 | )
85 |
```

6.3.3 Configurando o arquivo *autoload.php*

Vamos acessar o arquivo na pasta *application/config*, assim:

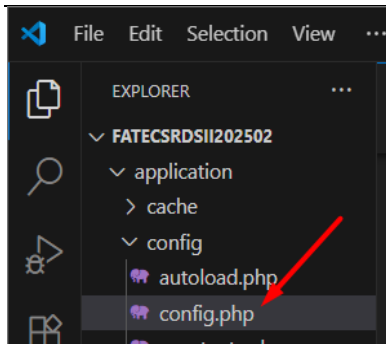


E acrescentamos a chamada da biblioteca *database*, conforme abaixo:

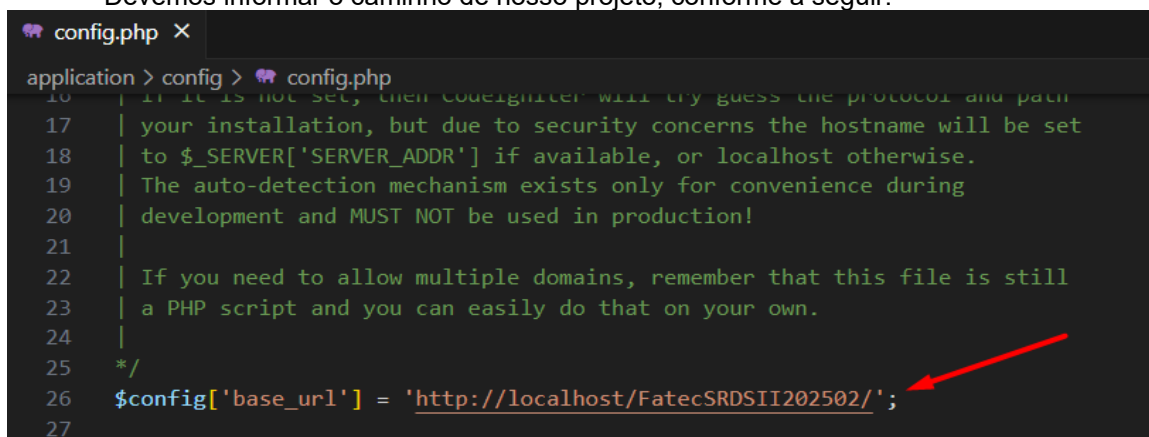
```
autoload.php X database.php config.php
application > config > autoload.php
56 | You can also supply an alternative library
57 | in the controller:
58 |
59 | $autoload['libraries'] = array('user_agent');
60 | */
61 | $autoload['libraries'] = array('database');
62 |
```

6.3.4 Configurando o arquivo *config.php*

Vamos acessar o arquivo na pasta *application/config*, para vamos configurar a URL que será utilizada, assim:



Devemos informar o caminho de nosso projeto, conforme a seguir:

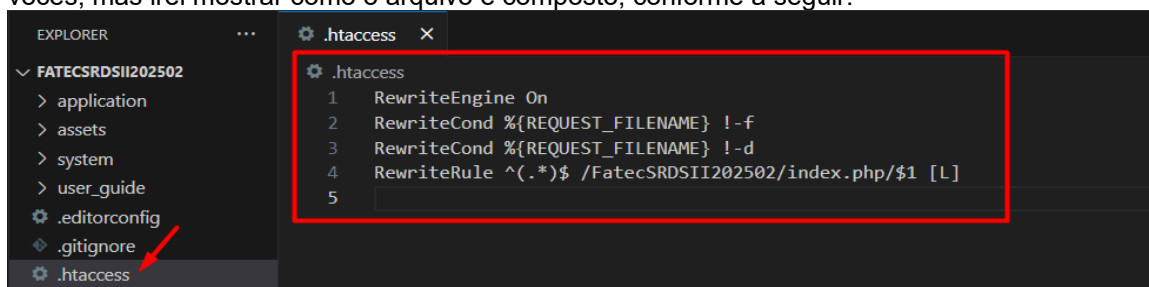


6.3.5 Configurando o arquivo *htaccess*

O CodeIgniter por padrão já dá suporte a URL amigáveis, como os grandes frameworks em PHP, o grande problema é que ele ainda insiste em mostrar o arquivo *index.php* na URL o que deixa toda requisição horrível. Veja o padrão de URL na instalação padrão do CodeIgniter: **<http://seusite.com.br/index.php/controller/method/parameter>**

Como deveria ser: **<http://seusite.com.br/controller/method/parameter>**

Como estamos usando o Apache como nosso servidor *web*, criamos um arquivo *.htaccess* no caminho "C:\XAMP\HTDOCS\Estagio com o seguinte conteúdo (irei fornecer a pasta para vocês, mas irei mostrar como o arquivo é composto, conforme a seguir:

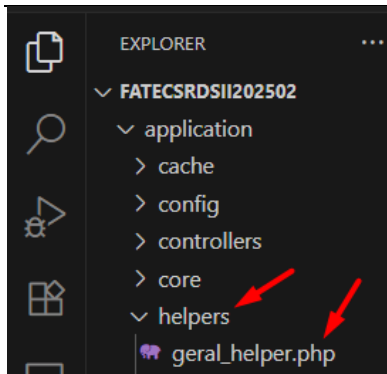


Com isso terminamos a configuração do projeto Mapa, vamos dizer que é a base para que o projeto "rode".

6.5 Utilização da *Helpers* no *CodeIgniter*

Faremos agora a utilização do **HELPERS** no *CodeIgniter*, como falamos no início da abordagem desse framework, não é nada mais que um ajudante, um grupo de funções onde você poderá chamar no decorrer de seu projeto, assim que precisar, em nosso caso faremos essa função inicialmente, e caso precisemos, criaremos outras na sequência de nossa prática.

Para criar um *helper* você terá que criar um arquivo no formato **nomedoarquivo_helper.php**. Então toda vez que você criar um *helper* utilize o sufixo **_helper**. Esse arquivo deve ser colocado na pasta "*helpers*" localizado na sua pasta "*application*". Nosso implemento se chamará **geral_helper.php**.



A seguir vamos programar nossa função no **Helper**, que será bem simples, somente será uma rotina para verificar se os campos passados pelo *FrontEnd* serão os mesmos utilizados dentro das *Controllers*, isso foi idealizado em nosso projeto para evitar erros no lado do *BackEnd*, vejamos:

```
geral_helper.php x
application > helpers > geral_helper.php
1  <?php
2  defined('BASEPATH') or exit('No direct script access allowed');
3
4  /*
5   Função para verificar os parâmetros vindos do FrontEnd
6   */
7  function verificarParam($atributos, $lista){
8      //1º -Verificar se os elementos do Front estão nos atributos necessários
9      foreach($lista as $key => $value){
10         if(array_key_exists($key, get_object_vars($atributos))){
11             $estatus = 1;
12         }else{
13             $estatus = 0;
14             break;
15         }
16     }
17
18     // 2º - Verificando a quantidade de elementos
19     if (count(get_object_vars($atributos)) != count($lista)){
20         $estatus = 0;
21     }
22
23     return $estatus;
24 }
25
26 ?>
```

Acima vemos dois comandos interessantes, que nos auxiliam a verificar os *arrays* necessários, em um deles temos a necessidade de converter objetos em *array*, para que a comparação seja realizada.

- `array_key_exists()` — Checa se uma chave ou índice existe em um array;
- `get_object_vars` — Retorna um *array* associativo das definidas acessíveis propriedades do objeto especificado por *object*.

Com isso, este é o projeto configurado de forma básica para formarmos a base de nosso projeto, este estará no Github.

Agora, vamos implementar mais uma função em nossa **Helper**, para verificar os atributos passados quanto a sua tipagem e estrutura desejada, chamamos a mesma de `validarDados()`, que será mais uma função que nos auxiliará na **Controller** de nosso projeto. Segue:

```

26  /*
27  Função para verificar os tipos de dados
28  */
29  function validarDados($valor, $tipo, $tamanhoZero = true) {
30
31      // Verifica vazio ou nulo
32      if (is_null($valor) || $valor === '') {
33          return array('codigoHelper' => 2, 'msg' => 'Conteúdo nulo ou vazio.');
```

```

34      }
35      // Se considerar '0' como vazio
36      if ($tamanhoZero && ($valor === 0 || $valor === '0')) {
37          return array('codigoHelper' => 3, 'msg' => 'Conteúdo zerado.');
```

```

38      }
39
40      switch ($tipo) {
41          case 'int':
42              // Filtra como inteiro, aceita '123' ou 123
43              if (filter_var($valor, FILTER_VALIDATE_INT) === false) {
44                  return array('codigoHelper' => 4, 'msg' => 'Conteúdo não inteiro.');
```

```

45              }
46              break;
47
48          case 'string':
49              // Garante que é string não vazia após trim
50              if (!is_string($valor) || trim($valor) === '') {
51                  return array('codigoHelper' => 5, 'msg' => 'Conteúdo não é um texto.');
```

```

52              }
53              break;
54          case 'date':
55              //Verifico se tem padrão de data
56              if(!preg_match('/^\d{4}-\d{2}-\d{2}$/', $valor, $match)){
57                  return array('codigoHelper' => 6, 'msg' => 'Data em formato inválido.');
```

```

58              }else{
59                  // Tenta criar DateTime no formato Y-m-d
60                  $d = DateTime::createFromFormat('Y-m-d', $valor);
61                  if(($d->format('Y-m-d') === $valor) == false ){
62                      return array('codigoHelper' => 6, 'msg' => 'Data inválida.');
```

```

63                  }
64              }
65              break;
66          case 'hora':
67              //Verifico se tem padrão de hora
68              if(!preg_match('/^(?:[01]\d|2[0-3]):[0-5]\d$/', $valor)){
69                  return array('codigoHelper' => 7, 'msg' => 'Hora em formato inválido.');
```

```

70              }
71              break;
72          default:
73              return array('codigoHelper' => 0, 'msg' => 'Tipo de dado não definido.');
```

```

74      }
75      // Valor default da variável $retorno caso não ocorra erro
76      return array('codigoHelper' => 0, 'msg' => 'Validação correta.');
```

```

77  }

```

Na linha 43 temos a função `filter_var()`, que é usada para **validar ou sanitizar** (limpar) dados. Ela faz parte da extensão **Filter** do PHP, que ajuda a garantir que os dados recebidos estejam no formato correto ou seguros para uso, e validam se é inteiro.

Na linha 49 de nosso código a função `is_string()` serve para verificar se estamos o conteúdo da variável é ou não uma *String*, seu retorno é verdadeiro ou falso.

Nas linhas 55 e 67 temos a função **`preg_match()`** que faz uma busca usando expressões regulares, que serve para fazermos validações, ou seja, descrever formatos de texto que queira trabalhar, em nossos casos, seriam encontrar padrões para data e hora, explanando detalhadamente:

`preg_match('/^(?:[01]\d|2[0-3]):[0-5]\d$/', $valor)`

^ e \$

- ^ indica **início da string**
- \$ indica **fim da string**

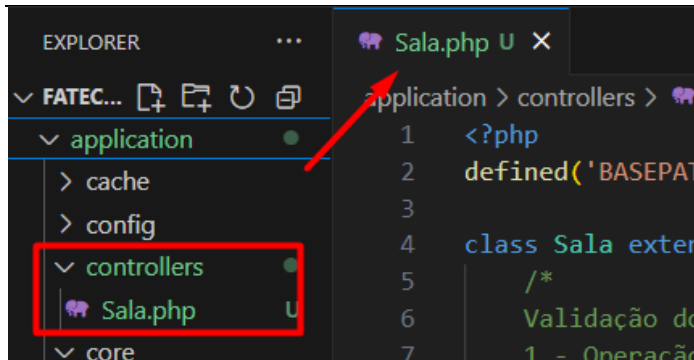
- Isso garante que a string inteira seja um horário, e não apenas parte dela.
- (?:[01]\d|2[0-3])
- (?:....) é um **grupo não capturante**, usado apenas para agrupar sem salvar o valor.
 - [01]\d casa com horas de 00 a 19:
 - [01] → pode ser 0 ou 1
 - \d → qualquer dígito (0 a 9)
 - 2[0-3] casa com horas de 20 a 23
 - Juntos, esse grupo valida **horas de 00 a 23**
- :
- Literalmente o caractere dois-pontos, separando horas e minutos.
- [0-5]\d
- [0-5] → primeiro dígito dos minutos (de 00 a 59)
 - \d → segundo dígito dos minutos
- preg_match('/^(\d{4})-(\d{2})-(\d{2})\$/', \$valor, \$match)
- ^ e \$
- ^ → início da string
 - \$ → fim da string
- Isso garante que a string inteira seja uma data nesse formato, sem caracteres extras.
- (\d{4})
- \d → representa um dígito (0 a 9)
 - {4} → exatamente 4 dígitos
 - Esse grupo captura o **ano** (ex: 2025)
-
- Um hífen literal, separando ano, mês e dia.
- (\d{2})
- Dois dígitos para o **mês** (ex: 01 a 12)
- Outro -**
- Outro hífen literal
- (\d{2})
- Dois dígitos para o **dia** (ex: 01 a 31)

6.5 Confeção das *controllers* e *models*

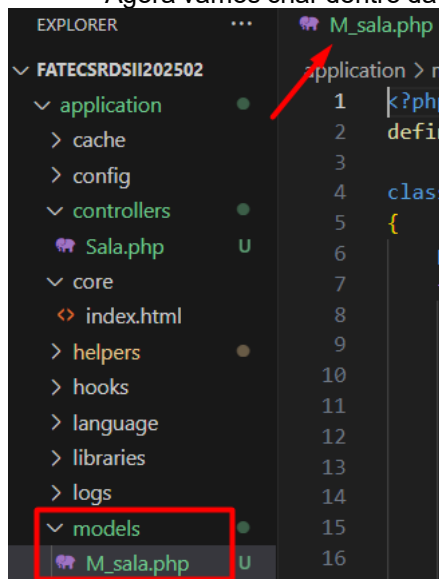
Vamos fazer objeto a objeto de nosso projeto, a ideia é construir a controller e posteriormente a model, e depois implementar no *Insomnia* os *endpoints* que testam o que codificamos.

6.5.1 Objeto Sala

Neste objeto vamos tratar o CRUD (*Create Read Update and Delete*) dos dados pertencentes, inicialmente vamos criar dentro da pasta **controllers** de nosso projeto base o arquivo Sala.php, vejamos:



Agora vamos criar dentro da pasta **models** de nosso projeto o arquivo M_sala.php, vejam:



6.5.1.1 Controller Sala (*getters* e *setters* e o método *inserir()*)

Vamos inicialmente documentar através de comentários, os retornos possíveis dentro de nossa classe, para isso, vamos utilizar um código para cada erro que nosso sistema possa retornar com a respectiva explicação deste erro, isso vai facilitar a manutenção do código em caso de algum erro e ajudar o *frontend* na hora de sua codificação, e caso nossa aplicação não tenha uma *view*, nosso backend está fazendo validações de entrada e tipo de dados, conforme explicado em sala de aula.

Em seguida, vamos codificar a parte de encapsulamento de nossa classe Sala, onde colocaremos os *getters* e *setters*, após isso iremos já fazer o código do método inserção de nossa *controller* da sala de aula em nosso projeto, conforme a seguir:

```

Sala.php U X
application > controllers > Sala.php
1  <?php
2  defined('BASEPATH') OR exit('No direct script access allowed');
3
4  class Sala extends CI_Controller {
5      /*
6       * Validação dos tipos de retornos nas validações (Código de erro)
7       * 1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
8       * 2 - Conteúdo passado nulo ou vazio
9       * 3 - Conteúdo zerado
10      * 4 - Conteúdo não inteiro
11      * 5 - Conteúdo não é um texto
12      * 6 - Data em formato inválido
13      * 7 - Hora em formato inválido
14      * 99 - Parâmetros passados do front não correspondem ao método
15      */
16
17      //Atributos privados da classe
18      private $codigo;
19      private $descricao;
20      private $andar;
21      private $capacidade;
22      private $status;
23
24      //Getters dos atributos
25      public function getCodigo()
26      {
27          return $this->codigo;
28      }
29
30      public function getDescricao()
31      {
32          return $this->descricao;
33      }
34
35      public function getAndar()
36      {
37          return $this->andar;
38      }
39
40      public function getCapacidade()
41      {
42          return $this->capacidade;
43      }
44
45      public function getStatus()
46      {
47          return $this->status;
48      }
49
50      //Setters dos atributos
51      public function setCodigo($codigoFront)
52      {
53          $this->codigo = $codigoFront;
54      }

```



```

55
56     public function setDescricao($descricaoFront)
57     {
58         $this->descricao = $descricaoFront;
59     }
60
61     public function setAndar($andarFront)
62     {
63         $this->andar = $andarFront;
64     }
65
66     public function setCapacidade($capacidadeFront)
67     {
68         $this->capacidade = $capacidadeFront;
69     }
70
71     public function setStatus($statusFront)
72     {
73         $this->tipoUsuario = $statusFront;
74     }
75
76     public function inserir() {
77         //Atributos para controlar o status de nosso método
78         $erros = [];
79         $sucesso = false;
80
81         try {
82
83             $json = file_get_contents('php://input');
84             $resultado = json_decode($json);
85             $lista = [
86                 "codigo" => '0',
87                 "descricao" => '0',
88                 "andar" => '0',
89                 "capacidade" => '0'
90             ];
91
92             if (verificarParam($resultado, $lista) != 1) {
93                 // Validar vindos de forma correta do frontend (Helper)
94                 $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
95             }else{
96                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
97                 $retornoCodigo = validarDados($resultado->codigo, 'int', true);
98                 $retornoDescricao = validarDados($resultado->descricao, 'string', true);
99                 $retornoAndar = validarDados($resultado->andar, 'int', true);
100                 $retornoCapacidade = validarDados($resultado->capacidade, 'int', true);
101
102                 if($retornoCodigo['codigoHelper'] != 0){
103                     $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
104                                 'campo' => 'Codigo',
105                                 'msg' => $retornoCodigo['msg']];
106                 }
107
108                 if($retornoDescricao['codigoHelper'] != 0){
109                     $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
110                                 'campo' => 'Descrição',
111                                 'msg' => $retornoDescricao['msg']];
112                 }
113
114                 if($retornoAndar['codigoHelper'] != 0){
115                     $erros[] = ['codigo' => $retornoAndar['codigoHelper'],
116                                 'campo' => 'Andar',
117                                 'msg' => $retornoAndar['msg']];
118                 }
119
120                 if($retornoCapacidade['codigoHelper'] != 0){
121                     $erros[] = ['codigo' => $retornoCapacidade['codigoHelper'],
122                                 'campo' => 'Capacidade',
123                                 'msg' => $retornoCapacidade['msg']];
124                 }
125                 //Se não encontrar erros
126                 if (empty($erros)) {
127                     $this->setCodigo($resultado->codigo);

```

```

128         $this->setDescricao($resultado->descricao);
129         $this->setAndar($resultado->andar);
130         $this->setCapacidade($resultado->capacidade);
131
132         $this->load->model('M_sala');
133         $resBanco = $this->M_sala->inserir(
134             $this->getCodigo(),
135             $this->getDescricao(),
136             $this->getAndar(),
137             $this->getCapacidade()
138         );
139
140         if ($resBanco['codigo'] == 1) {
141             $sucesso = true;
142         } else {
143             // Captura erro do banco
144             $erros[] = [
145                 'codigo' => $resBanco['codigo'],
146                 'msg' => $resBanco['msg']
147             ];
148         }
149     }
150 }
151 } catch (Exception $e) {
152     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
153 }
154
155 // Monta retorno único
156 if ($sucesso == true) {
157     $retorno = ['sucesso' => $sucesso, 'msg' => 'Sala cadastrada corretamente.'];
158 } else {
159     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
160 }
161
162 // Transforma o array em JSON
163 echo json_encode($retorno);
164 }
165 }
166 ?>

```

6.5.1.2 Model M_sala (métodos inserir() e consultarSala())

Vamos inicialmente documentar através de comentários, os retornos possíveis dentro de nossa classe, para isso, vamos utilizar um código para cada erro que nosso sistema possa retornar com a respectiva explicação deste erro, como fizemos na *controller*. Faremos o método público *inserir()* e o método auxiliar (privado) *consultarSala()* que servirá para antes de inserirmos, verificar se a mesma não já encontra em nossa base de dados, inclusive verificando se a mesma não está desativada. Agora, vamos codificar da forma a seguir:

```

M_sala.php
application > models > M_sala.php
1 <?php
2 defined('BASEPATH') or exit('No direct script access allowed');
3
4 class M_sala extends CI_Model
5 {
6     /*
7     Validação dos tipos de retornos nas validações (Código de erro)
8     0 - Erro de exceção
9     1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
10    8 - Houve algum problema de inserção, atualização, consulta ou exclusão
11    9 - Sala desativada no sistema
12    10 - Sala já cadastrada
13    98 - Método auxiliar de consulta que não trouxe dados
14    */
15
16    public function inserir($codigo, $descricao, $andar, $capacidade){
17        try {
18            //Verifico se a sala já está cadastrada
19            $retornoConsulta = $this->consultarSala($codigo);
20
21            if ($retornoConsulta['codigo'] != 9 &&
22                $retornoConsulta['codigo'] != 10) {

```

```

M_sala.php U x
application > models > M_sala.php
5  {
16  public function inserir($codigo, $descricao, $andar, $capacidade){
17      try {
21          if ($retornoConsulta['codigo'] != 9 &&
23
24          //Query de inserção dos dados
25          $this->db->query("insert into tbl_sala (codigo, descricao, andar, capacidade)
26              values ($codigo, '$descricao', $andar, $capacidade)");
27
28          //Verificar se a inserção ocorreu com sucesso
29          if ($this->db->affected_rows() > 0) {
30              $dados = array(
31                  'codigo' => 1,
32                  'msg' => 'Sala cadastrada corretamente'
33              );
34
35              } else {
36                  $dados = array(
37                      'codigo' => 8,
38                      'msg' => 'Houve algum problema na inserção na tabela de salas.'
39                  );
40              }
41          } else {
42              $dados = array('codigo' => $retornoConsulta['codigo'],
43                  'msg' => $retornoConsulta['msg']);
44          }
45      } catch (Exception $e) {
46          $dados = array(
47              'codigo' => 0,
48              'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
49          );
50      }
51
52      //Envia o array $dados com as informações tratadas
53      //acima pela estrutura de decisão if
54      return $dados;
55  }
56
57  //Método privado, pois será auxiliar nesta classe
58  private function consultaSala($codigo){
59      try {
60          //Query para consultar dados de acordo com parâmetros passados
61          $sql = "select * from tbl_sala where codigo = $codigo ";
62
63          $retornoSala = $this->db->query($sql);
64
65          //Verificar se a consulta ocorreu com sucesso
66          if ($retornoSala->num_rows() > 0) {
67              $linha = $retornoSala->row();
68              if (trim($linha->estatus) == "D") {
69                  $dados = array(
70                      'codigo' => 9,
71                      'msg' => 'Sala desativada no sistema, caso precise reativar a mesma,
72                          fale com o administrador.'
73                  );
74              } else {
75
76                  $dados = array(
77                      'codigo' => 10,
78                      'msg' => 'Sala já cadastrada no sistema.'
79                  );
80              }
81
82          } else {
83              $dados = array(
84                  'codigo' => 98,
85                  'msg' => 'Sala não encontrada.'
86              );
87          }
88      }
89  }

```

```

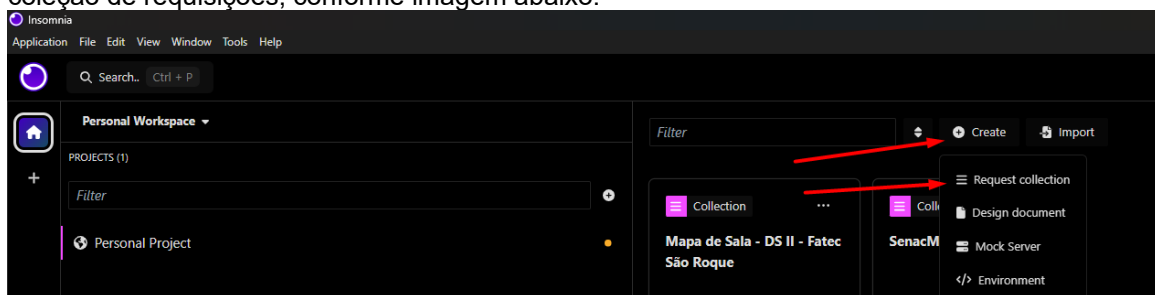
88     } catch (Exception $e) {
89         $dados = array(
90             'codigo' => 0,
91             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
92         );
93     }
94     //Envia o array $dados com as informações tratadas
95     //acima pela estrutura de decisão if
96     return $dados;
97 }
98 }
99 }>

```

Assim finalizamos método inserir, tanto na *Controller* quanto na *Model*.

6.5.1.3 Objeto Sala no *Insomnia* método **inserir()**

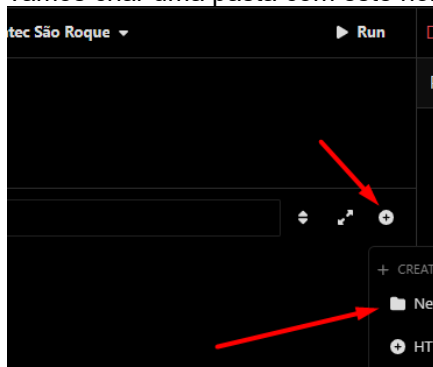
Vamos configurar nosso ambiente para o projeto de nossas aulas, você pode baixar o Insomnia em <https://insomnia.rest/download>, ele possui várias formas de manipulação, recomendo vocês darem uma olhada materiais a respeito, aqui na aula farei de forma direta, vamos criar uma coleção de requisições, conforme imagem abaixo:

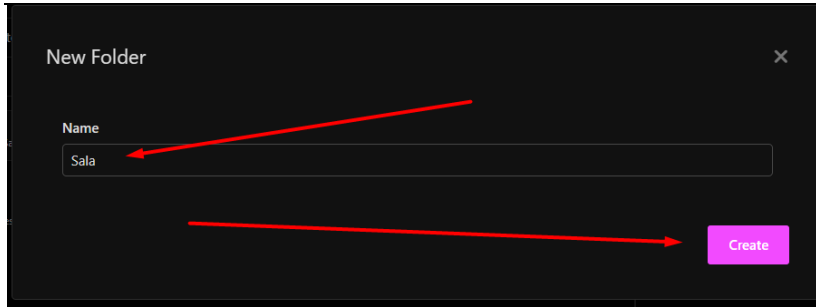


Após isso, será mostrada esta tela para nós, o nome da coleção pode ser qualquer um, eu defini assim, devido as nossas aulas:

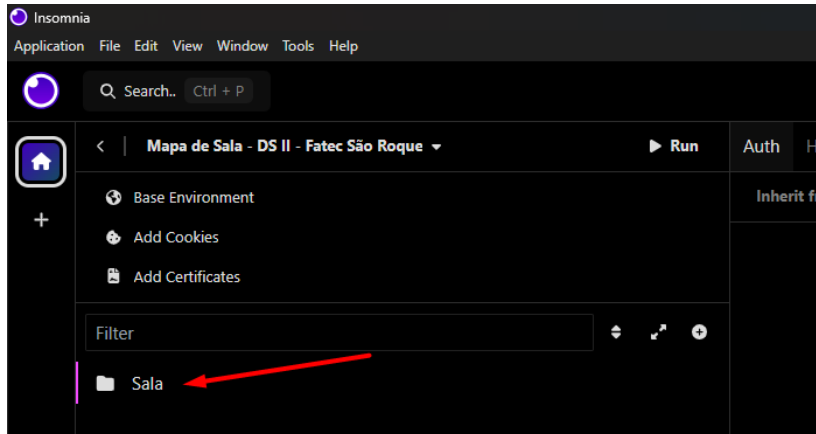


A ideia agora é a organização de nosso projeto, como vamos trabalhar com alguns objetos, a ideia é criar uma pasta para cada um, então como estamos trabalhando com o objeto Sala, vamos criar uma pasta com este nome:





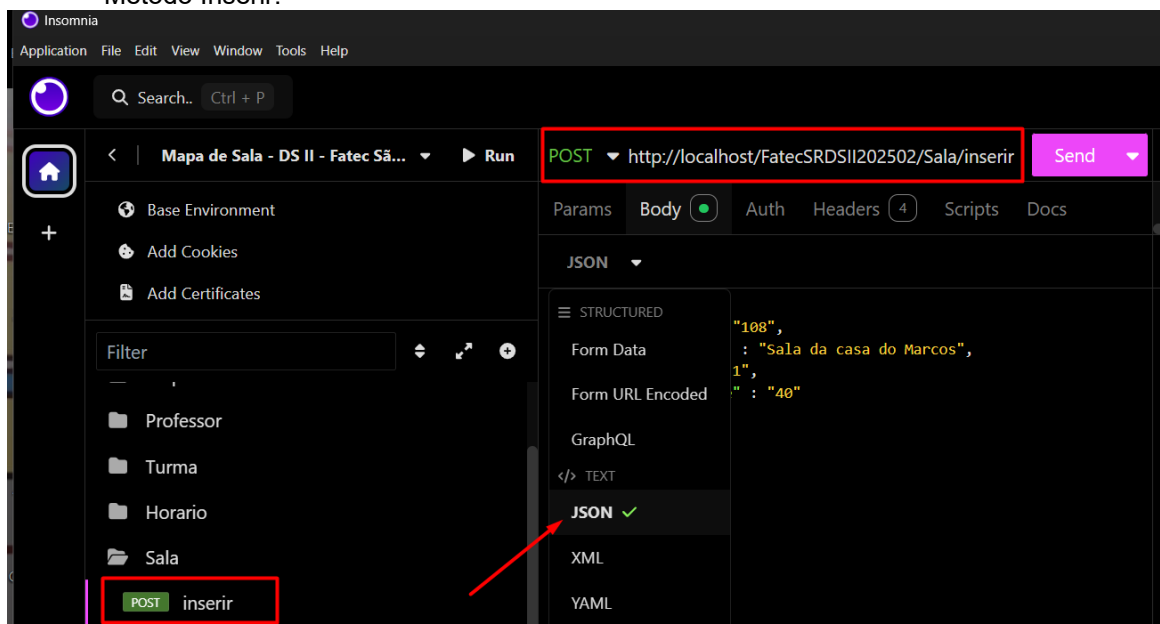
Ficando assim:



Feito isso, agora vamos efetivamente criar os EndPoints de nosso objeto, ou seja, incluir, consultar, alterar e desativar a referida sala.



Método Inserir:



POST http://localhost/FatecSRDSII202502/Sala/inserir Send 200 OK 44 ms 56 B Just Now

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0 → Mock

JSON

```
1 {
2   "codigo": "11",
3   "descricao": "Sala de aula do Marcos",
4   "andar": "1",
5   "capacidade": "40"
6 }
```

Preview

```
1 {
2   "sucesso": true,
3   "msg": "Sala cadastrada corretamente."
4 }
```

Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:

POST http://localhost/FatecSRDSII202502/Sala/inserir Send 200 OK 48 ms 313 B

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "codigo": "",
3   "descricao": "",
4   "andar": "",
5   "capacidade": ""
6 }
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 2,
6       "campo": "Codigo",
7       "msg": "Conteúdo nulo ou vazio."
8     },
9     {
10      "codigo": 2,
11      "campo": "Descrição",
12      "msg": "Conteúdo nulo ou vazio."
13     },
14     {
15      "codigo": 2,
16      "campo": "Andar",
17      "msg": "Conteúdo nulo ou vazio."
18     },
19     {
20      "codigo": 2,
21      "campo": "Capacidade",
22      "msg": "Conteúdo nulo ou vazio."
23     }
24   ]
25 }
```

Campos vazios.

POST http://localhost/FatecSRDSII202502/Sala/inserir Send 200 OK 40 ms 99 B

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "codigo": "teste",
3   "descricao": "Sala de aula do Marcos",
4   "andar": "1",
5   "capacidade": "40"
6 }
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Codigo",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```

Código não numérico.

POST http://localhost/FatecSRDSII202502/Sala/inserir Send 200 OK 62 ms 98 B

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "codigo": "1",
3   "descricao": "Sala de aula do Marcos",
4   "andar": "qqqq",
5   "capacidade": "40"
6 }
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Andar",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```

Andar não numérico.

```

1 {
2   "codigo": "1",
3   "descricao": "Sala de aula do Marcos",
4   "andar": "1",
5   "capacidade": "yt"
6 }
    
```

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Capacidade",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
    
```

Capacidade não numérica.

```

1 {
2   "codigo": "1",
3   "descricao": "Sala de aula do Marcos",
4   "andar": "1",
5   "capacidade": "3"
6 }
    
```

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 10,
6       "msg": "Sala já cadastrada no sistema."
7     }
8   ]
9 }
    
```

Sala já cadastrada no sistema.

```

1 {
2   "codigo": "108",
3   "descricao": "Sala de aula do Marcos",
4   "andar": "1",
5   "capacidade": "3"
6 }
    
```

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 9,
6       "msg": "Sala desativada no sistema, caso precise reativar a mesma, fale com o administrador."
7     }
8   ]
9 }
    
```

Sala que já estava desativada no sistema

Assim finalizamos os *endpoints* com verbalização *Rest*, implementem toda esta parte para que possamos dar seguimento as aulas.

6.5.1.4 Controller Sala (acrescentar função na *helper* e o método consultar())

Para validarmos a consulta em nosso projeto, ou seja, a ideia é que caso o nosso usuário não passe conteúdo nos atributos, o sistema traga todos os registros, mas caso passe algum, necessitamos validar, então para isso vamos criar mais uma função em nossa *helper*, a função **validarDadosConsulta()**, onde passaremos o valor e o tipo de dado, caso esteja vazio, não validamos, e caso sim, fazemos a referida validação, e na *controller* fazemos as referidas referências.

6.5.1.4.1 Código da *helper*, função **validarDadosConsulta()**

Para isso vamos abrir nossa *helper* que já esta codificada.

```

1  defined('BASEPATH') OR exit('Não é permitido acessar este arquivo diretamente. ...
2
3  ...
4
5  ...
6
7  ...
8
9  ...
10 ...
11 ...
12 ...
13 ...
14 ...
15 ...
16 ...
17 ...
18 ...
19 ...
20 ...
21 ...
22 ...
23 ...
24 ...
25 ...
26 ...
27 ...
28 ...
29 ...
30 ...
31 ...
32 ...
33 ...
34 ...
35 ...
36 ...
37 ...
38 ...
39 ...
40 ...
41 ...
42 ...
43 ...
44 ...
45 ...
46 ...
47 ...
48 ...
49 ...
50 ...
51 ...
52 ...
53 ...
54 ...
55 ...
56 ...
57 ...
58 ...
59 ...
60 ...
61 ...
62 ...
63 ...
64 ...
65 ...
66 ...
67 ...
68 ...
69 ...
70 ...
71 ...
72 ...
73 ...
74 ...
75 ...
76 ...
77 ...
78 ...
79 ...
80 ...
81 ...
82 ...
    
```


E acrescentar a função:

```

78  /*
79  Função para verificar os tipos de dados para Consulta
80  */
81  function validarDadosConsulta($valor, $tipo) {
82
83      if ($valor != '') {
84          switch ($tipo) {
85              case 'int':
86                  // Filtra como inteiro, aceita '123' ou 123
87                  if (filter_var($valor, FILTER_VALIDATE_INT) === false) {
88                      return array('codigoHelper' => 4, 'msg' => 'Conteúdo não inteiro.');
```

6.5.1.4.2 Código da *controller*, função *consultar()*

```

168  public function consultar() {
169      //Atributos para controlar o status de nosso método
170      $erros = [];
171      $sucesso = false;
172
173      try {
174
175          $json = file_get_contents('php://input');
176          $resultado = json_decode($json);
177          $lista = [
178              "codigo" => '0',
179              "descricao" => '0',
180              "andar" => '0',
181              "capacidade" => '0'
182          ];
183
184          if (verificarParam($resultado, $lista) != 1) {
185              // Validar vindos de forma correta do frontend (Helper)
186              $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
187          } else {
188              // Validar campos quanto ao tipo de dado e tamanho (Helper)
189              $retornoCodigo = validarDadosConsulta($resultado->codigo, 'int');
```

```

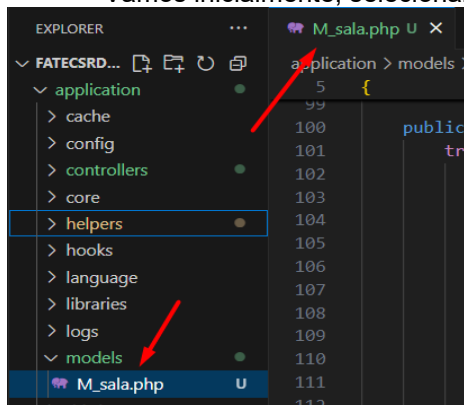
194         if($retornoCodigo['codigoHelper'] != 0){
195             $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
196                         'campo' => 'Codigo',
197                         'msg' => $retornoCodigo['msg']];
198         }
199
200         if($retornoDescricao['codigoHelper'] != 0){
201             $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
202                         'campo' => 'Descrição',
203                         'msg' => $retornoDescricao['msg']];
204         }
205
206         if($retornoAndar['codigoHelper'] != 0){
207             $erros[] = ['codigo' => $retornoAndar['codigoHelper'],
208                         'campo' => 'Andar',
209                         'msg' => $retornoAndar['msg']];
210         }
211
212         if($retornoCapacidade['codigoHelper'] != 0){
213             $erros[] = ['codigo' => $retornoCapacidade['codigoHelper'],
214                         'campo' => 'Capacidade',
215                         'msg' => $retornoCapacidade['msg']];
216         }
217         //Se não encontrar erros
218         if (empty($erros)) {
219             $this->setCodigo($resultado->codigo);
220             $this->setDescricao($resultado->descricao);
221             $this->setAndar($resultado->andar);
222             $this->setCapacidade($resultado->capacidade);
223
224             $this->load->model('M_sala');
225             $resBanco = $this->M_sala->consultar($this->getCodigo(),
226                                                 $this->getDescricao(),
227                                                 $this->getAndar(),
228                                                 $this->getCapacidade());
229
230             if ($resBanco['codigo']== 1) {
231                 $sucesso = true;
232             } else {
233                 // Captura erro do banco
234                 $erros[] = [
235                     'codigo' => $resBanco['codigo'],
236                     'msg' => $resBanco['msg']
237                 ];
238             }
239         }
240     }
241 } catch (Exception $e) {
242     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
243 }
244
245 // Monta retorno único
246 if ($sucesso == true) {
247     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
248                 'msg' => $resBanco['msg'],
249                 'dados' => $resBanco['dados']];
250 } else {
251     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
252 }
253
254 // Transforma o array em JSON
255 echo json_encode($retorno);
256 }

```

Assim finalizamos este método em nossa **controller**.

6.5.1.5 Model M_sala (método **consultar()**)

Vamos inicialmente, selecionar a model M_sala que já está criada.

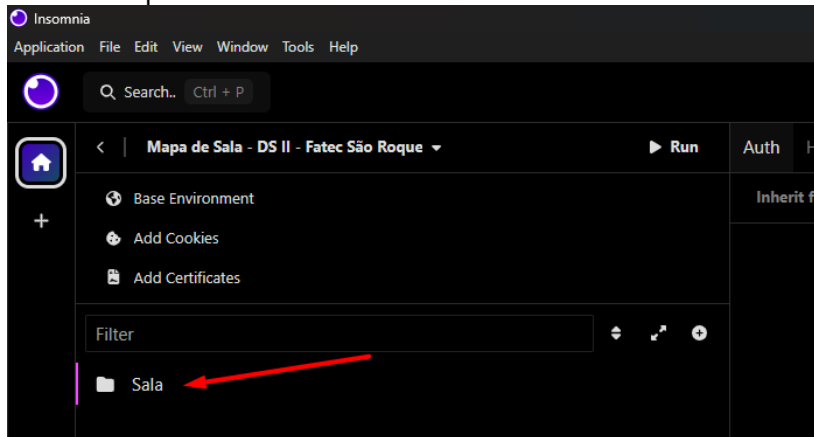


Agora, iremos implementar um método que montará a *query* de forma dinâmica, ou seja, de acordo com os parâmetros passados pela *controller*, a mesma se molda para montar o **SELECT** correto para atender o que é pedido, para isso vamos usar uma série de *If's* para moldar a consulta final, observe:

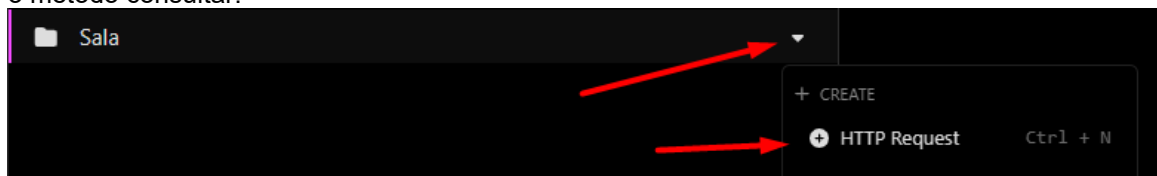
```
100 public function consultar($codigo, $descricao, $andar, $capacidade){
101     try {
102         //Query para consultar dados de acordo com parâmetros passados
103         $sql = "select * from tbl_sala where status = ' ' ";
104
105         if (trim($codigo) != '') {
106             $sql = $sql . " and codigo = $codigo ";
107         }
108
109         if (trim($andar) != '') {
110             $sql = $sql . " and andar = '$andar' ";
111         }
112
113         if (trim($descricao) != '') {
114             $sql = $sql . " and descricao like '%$descricao%' ";
115         }
116         if (trim($capacidade) != '') {
117             $sql = $sql . " and capacidade = '$capacidade' ";
118         }
119
120         $sql = $sql . " order by codigo ";
121
122         $retorno = $this->db->query($sql);
123
124         //Verificar se a consulta ocorreu com sucesso
125         if ($retorno->num_rows() > 0) {
126             $dados = array(
127                 'codigo' => 1,
128                 'msg' => 'Consulta efetuada com sucesso.',
129                 'dados' => $retorno->result()
130             );
131
132         } else {
133             $dados = array(
134                 'codigo' => 11,
135                 'msg' => 'Sala não encontrada.'
136             );
137         }
138     } catch (Exception $e) {
139         $dados = array(
140             'codigo' => 00,
141             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
142         );
143     }
144     //Envia o array $dados com as informações tratadas
145     //acima pela estrutura de decisão if
146     return $dados;
147 }
```

6.5.1.6 Objeto Sala no *Insomnia* método **consultar()**

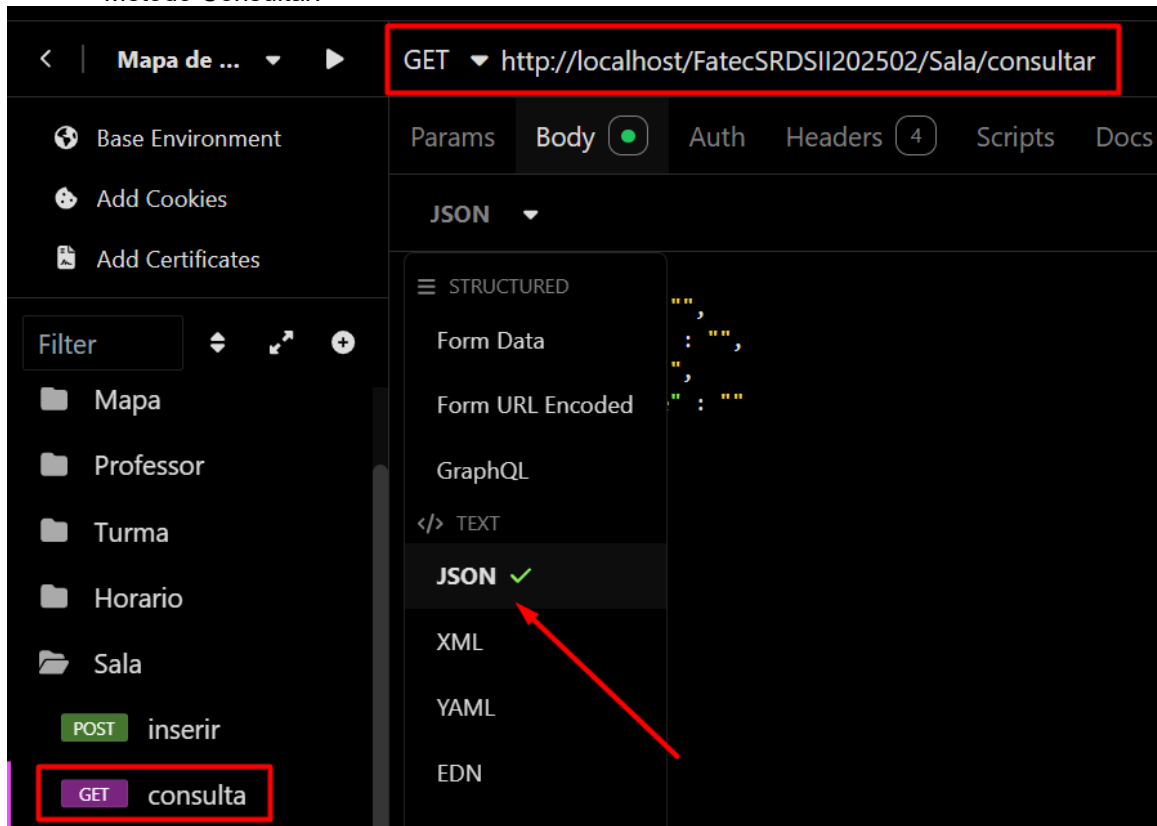
Na pasta Sala:



Feito isso, agora vamos efetivamente criar o EndPoint de nosso objeto, agora neste caso o método consultar.



Método Consultar:



GET http://localhost/FatecSRDSII202502/Sala/consultar Send 200 OK 58 ms 967 B

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0 → Mock

JSON

```
1 {
2   "codigo" : "",
3   "descricao" : "",
4   "andar" : "",
5   "capacidade" : ""
6 }
```

Preview

```
1 {
2   "sucesso": true,
3   "codigo": 1,
4   "msg": "Consulta efetuada com sucesso.",
5   "dados": [
6     {
7       "codigo": "11",
8       "descricao": "Sala de aula do Marcos",
9       "andar": "1",
10      "capacidade": "40",
11      "dtcria": "2025-08-23 22:35:09",
12      "estatus": ""
13     },
14     {
15       "codigo": "86",
16       "descricao": "Sala da casa do Marcos",
17       "andar": "1",
18       "capacidade": "40",
19       "dtcria": "2025-08-23 12:29:58",
20       "estatus": ""
21     }
22   ]
23 }
```

Fizemos o “caminho feliz”, agora vamos fazer algumas validações

JSON

```
1 {
2   "codigo" : "um",
3   "descricao" : "",
4   "andar" : "",
5   "capacidade" : ""
6 }
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Codigo",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```

JSON

```
1 {
2   "codigo" : "",
3   "descricao" : "",
4   "andar" : "oitavo",
5   "capacidade" : ""
6 }
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Andar",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```

JSON

```
1 {
2   "codigo" : "",
3   "descricao" : "",
4   "andar" : "",
5   "capacidade" : "vinte e cinco"
6 }
7
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Capacidade",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```

JSON

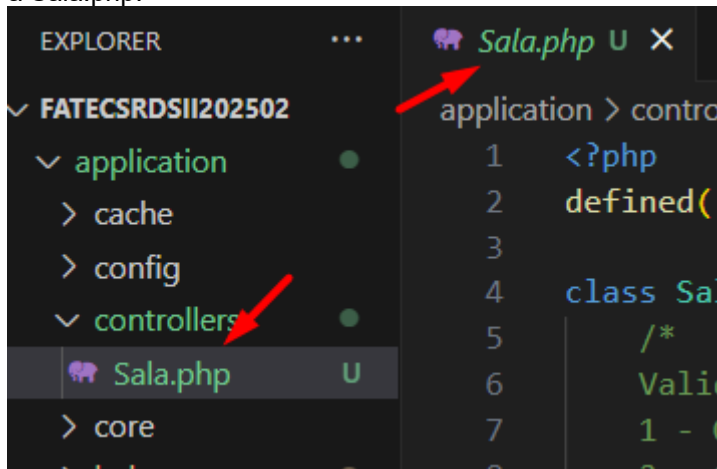
```
1 {
2   "codigo" : "",
3   "descricao" : "sala do marcos",
4   "andar" : "",
5   "capacidade" : ""
6 }
7
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 11,
6       "msg": "Sala não encontrada."
7     }
8   ]
9 }
```

6.5.1.7 Controller Sala (métodos alterar() e desativar())

Vamos dar continuidade na controller que estávamos trabalhando anteriormente, no caso a Sala.php.



Só que agora vamos implementar os dois métodos restantes de nosso CRUD, ou seja, os métodos de alteração e desativação, mas antes de continuarmos, nesta classe, vamos criar o código de erro **12 – Na atualização, pelos menos um atributo deve ser passado**. Isso significa que ao pedirmos uma atualização, deverá ser informado, pelo menos um campo para atualizarmos.

```
class Sala extends CI_Controller {
    /*
    Validação dos tipos de retornos nas validações (Código de erro)
    1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta)
    2 - Conteúdo passado nulo ou vazio
    3 - Conteúdo zerado
    4 - Conteúdo não inteiro
    5 - Conteúdo não é um texto
    6 - Data em formato inválido
    7 - Hora em formato inválido
    12 - Na atualização, pelo menos um atributo deve ser passado
    99 - Parâmetros passados do front não correspondem ao método
    */
}
```

Realizada esta parte, vamos ao código no método alterar em nossa controller.

```
258 public function alterar() {
259     //Atributos para controlar o status de nosso método
260     $erros = [];
261     $sucesso = false;
262
263     try {
264
265         $json = file_get_contents('php://input');
266         $resultado = json_decode($json);
267         $lista = [
268             "codigo" => '0',
269             "descricao" => '0',
270             "andar" => '0',
271             "capacidade" => '0'
272         ];
273
274         if (verificarParam($resultado, $lista) != 1) {
275             // Validar vindos de forma correta do frontend (Helper)
276             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
277         } else {

```

```

278 //Pelo menos um dos três parâmetros precisam ter dados para acontecer a atualização
279 if(trim($resultado->descricao) == '' && trim($resultado->andar) == '' &&
280 trim($resultado->capacidade) == ''){
281     $erros[] = ['codigo' => 12,
282               'msg' => 'Pelo menos um parâmetro precisa ser passado para atualização'];
283 }else{
284     // Validar campos quanto ao tipo de dado e tamanho (Helper)
285     $retornoCodigo = validarDados($resultado->codigo, 'int', true);
286     $retornoDescricao = validarDadosConsulta($resultado->descricao, 'string');
287     $retornoAndar = validarDadosConsulta($resultado->andar, 'int');
288     $retornoCapacidade = validarDadosConsulta($resultado->capacidade, 'int');
289
290     if($retornoCodigo['codigoHelper'] != 0){
291         $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
292                   'campo' => 'Codigo',
293                   'msg' => $retornoCodigo['msg']];
294     }
295
296     if($retornoDescricao['codigoHelper'] != 0){
297         $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
298                   'campo' => 'Descrição',
299                   'msg' => $retornoDescricao['msg']];
300     }
301
302     if($retornoAndar['codigoHelper'] != 0){
303         $erros[] = ['codigo' => $retornoAndar['codigoHelper'],
304                   'campo' => 'Andar',
305                   'msg' => $retornoAndar['msg']];
306     }
307
308     if($retornoCapacidade['codigoHelper'] != 0){
309         $erros[] = ['codigo' => $retornoCapacidade['codigoHelper'],
310                   'campo' => 'Capacidade',
311                   'msg' => $retornoCapacidade['msg']];
312     }
313     //Se não encontrar erros
314     if (empty($erros)) {
315         $this->setCodigo($resultado->codigo);
316         $this->setDescricao($resultado->descricao);
317         $this->setAndar($resultado->andar);
318         $this->setCapacidade($resultado->capacidade);
319
320         $this->load->model('M_sala');
321         $resBanco = $this->M_sala->alterar($this->getCodigo(),
322                                         $this->getDescricao(),
323                                         $this->getAndar(),
324                                         $this->getCapacidade());
325
326         if ($resBanco['codigo'] == 1) {
327             $sucesso = true;
328         } else {
329             // Captura erro do banco
330             $erros[] = [
331                 'codigo' => $resBanco['codigo'],
332                 'msg' => $resBanco['msg']
333             ];
334         }
335     }
336 }
337
338 } catch (Exception $e) {
339     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
340 }
341
342 // Monta retorno único
343 if ($sucesso == true) {
344     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
345               'msg' => $resBanco['msg']];
346 } else {
347     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
348 }
349
350 // Transforma o array em JSON
351 echo json_encode($retorno);
352 }

```


Com isso fizemos o método alterar, percebam que no início nas validações utilizamos todas as funções que utilizamos em nossa *helper*, pois, o código é obrigatório, porém, os demais itens não têm necessidade de serem passados, ao menos um.

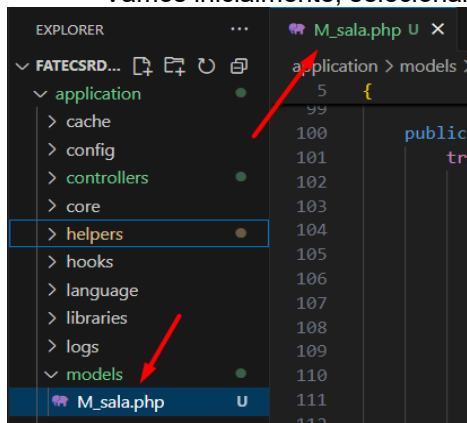
Dando sequência a nossa *controller*, vamos implementar agora o método desativar(), veja:

```
354 public function desativar() {
355     //Atributos para controlar o status de nosso método
356     $erros = [];
357     $sucesso = false;
358
359     try {
360
361         $json = file_get_contents('php://input');
362         $resultado = json_decode($json);
363         $lista = [
364             "codigo" => '0'
365         ];
366
367         if (verificarParam($resultado, $lista) != 1) {
368             // Validar vindos de forma correta do frontend (Helper)
369             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
370         } else {
371             // Validar código quanto ao tipo de dado e tamanho (Helper)
372             $retornoCodigo = validarDados($resultado->codigo, 'int', true);
373
374             if ($retornoCodigo['codigoHelper'] != 0) {
375                 $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
376                     'campo' => 'Codigo',
377                     'msg' => $retornoCodigo['msg']];
378             }
379
380             //Se não encontrar erros
381             if (empty($erros)) {
382                 $this->setCodigo($resultado->codigo);
383
384                 $this->load->model('M_sala');
385                 $resBanco = $this->M_sala->desativar($this->getCodigo());
386
387                 if ($resBanco['codigo'] == 1) {
388                     $sucesso = true;
389                 } else {
390                     // Captura erro do banco
391                     $erros[] = [
392                         'codigo' => $resBanco['codigo'],
393                         'msg' => $resBanco['msg']
394                     ];
395                 }
396             }
397         }
398     } catch (Exception $e) {
399         $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
400     }
401
402     // Monta retorno único
403     if ($sucesso == true) {
404         $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
405             'msg' => $resBanco['msg']];
406     } else {
407         $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
408     }
409
410     // Transforma o array em JSON
411     echo json_encode($retorno);
412 }
413
414
415 }
```

A única validação que fazemos neste caso, é o código da sala, que será o elemento mais importante, pois, por ele desativaremos a mesma na *model*.

6.5.1.8 Model M_sala (métodos **alterar()** e **desativar()**)

Vamos inicialmente, selecionar a model M_sala que já está criada.



Agora, iremos implementar um método que montará a *query* de forma dinâmica, ou seja, de acordo com os parâmetros passados pela *controller*, a mesma se molda para montar o **UPDATE** correto para atender o que é pedido, para isso vamos usar uma série de *If's* para a atualização final, observe:

```
149 public function alterar($codigo, $descricao, $andar, $capacidade){
150     try {
151         //Verifico se a sala já está cadastrada
152         $retornoConsulta = $this->consultaSala($codigo);
153
154         if ($retornoConsulta['codigo'] == 10) {
155             //Inicio a query para atualização
156             $query = "update tbl_sala set ";
157
158             //Vamos comparar os itens
159             if ($descricao != '') {
160                 $query .= "descricao = '$descricao', ";
161             }
162
163             if ($andar != '') {
164                 $query .= "andar = $andar, ";
165             }
166
167             if ($capacidade != '') {
168                 $query .= "capacidade = $capacidade, ";
169             }
170
171             //Termino a concatenação da query
172             $queryFinal = rtrim($query, ", ") . " where codigo = $codigo";
173
174             //Executo a Query de atualização dos dados
175             $this->db->query($queryFinal);
176
177             //Verificar se a atualização ocorreu com sucesso
178             if ($this->db->affected_rows() > 0) {
179                 $dados = array(
180                     'codigo' => 1,
181                     'msg' => 'Sala atualizada corretamente.'
182                 );
183             } else {
184                 $dados = array(
185                     'codigo' => 8,
186                     'msg' => 'Houve algum problema na atualização na tabela de sala.'
187                 );
188             }
189         } else {
190             $dados = array('codigo' => $retornoConsulta['codigo'],
191                           'msg' => $retornoConsulta['msg']);
192         }
193     }
194 }
```

```

195     } catch (Exception $e) {
196         $dados = array(
197             'codigo' => 00,
198             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
199         );
200     }
201     //Envia o array $dados com as informações tratadas
202     //acima pela estrutura de decisão if
203     return $dados;
204 }

```

E finalizando, vamos implementar o método `desativar()`, veja:

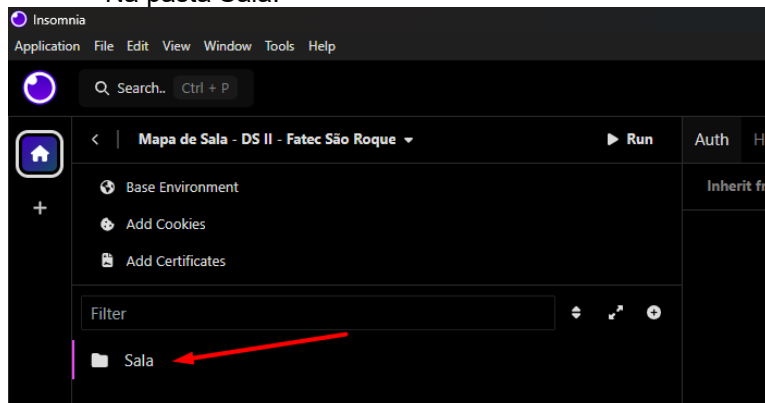
```

206 public function desativar($codigo){
207     try {
208         //Verifico se a sala já está cadastrada
209         $retornoConsulta = $this->consultaSala($codigo);
210
211         if ($retornoConsulta['codigo'] == 10) {
212
213             //Query de atualização dos dados
214             $this->db->query("update tbl_sala set status = 'D'
215                             where codigo = $codigo");
216
217             //Verificar se a atualização ocorreu com sucesso
218             if ($this->db->affected_rows() > 0) {
219                 $dados = array(
220                     'codigo' => 1,
221                     'msg' => 'Sala DESATIVADA corretamente.'
222                 );
223             } else {
224                 $dados = array(
225                     'codigo' => 8,
226                     'msg' => 'Houve algum problema na DESATIVAÇÃO da Sala.'
227                 );
228             }
229         } else {
230             $dados = array('codigo' => $retornoConsulta['codigo'],
231                             'msg' => $retornoConsulta['msg']);
232         }
233     } catch (Exception $e) {
234         $dados = array(
235             'codigo' => 00,
236             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
237         );
238     }
239     //Envia o array $dados com as informações tratadas
240     //acima pela estrutura de decisão if
241     return $dados;
242 }
243 }
244 }
245 }
246 }

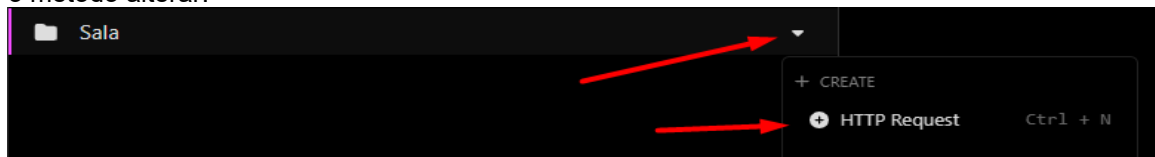
```

6.5.1.9 Objeto Sala no *Insomnia* métodos `consultar()` e `desativar()`

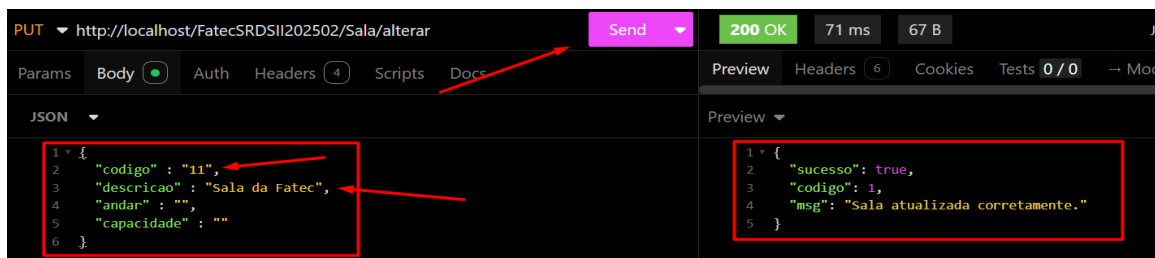
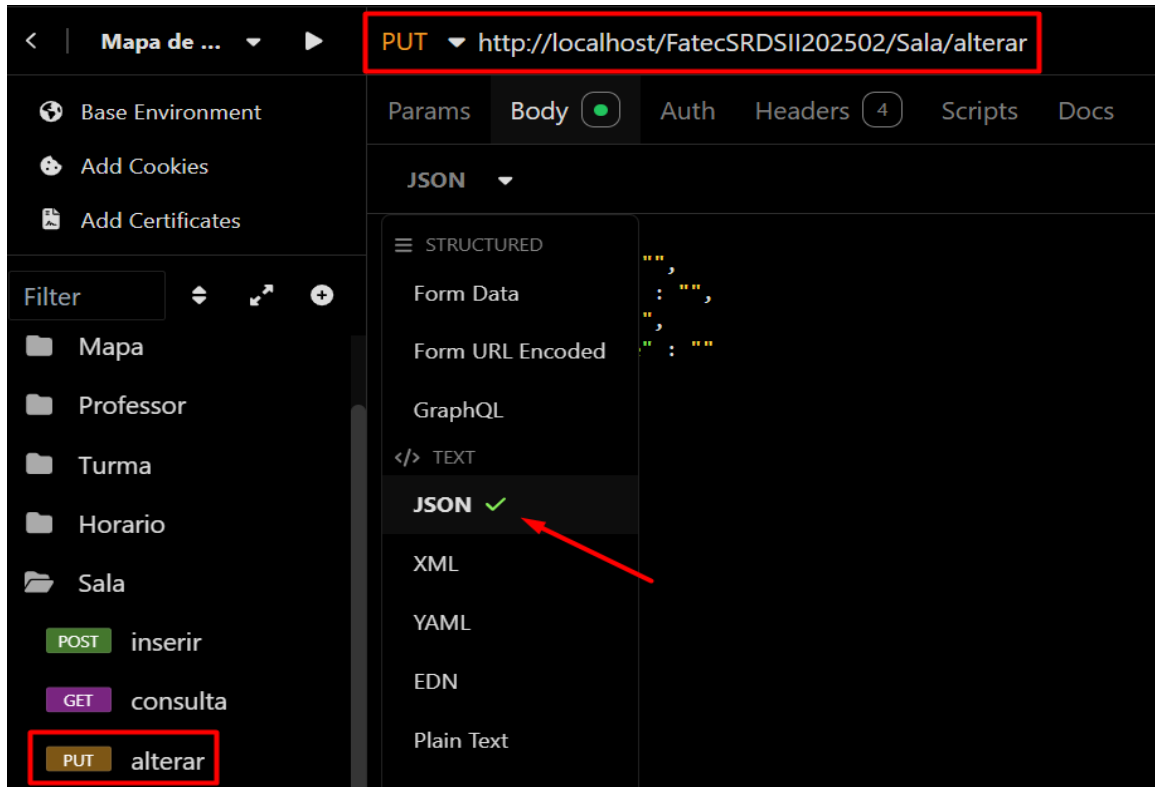
Na pasta Sala:



Feito isso, agora vamos efetivamente criar o EndPoint de nosso objeto, agora neste caso o método alterar.



Método Alterar:



Fizemos o “caminho feliz”, agora vamos fazer algumas validações:



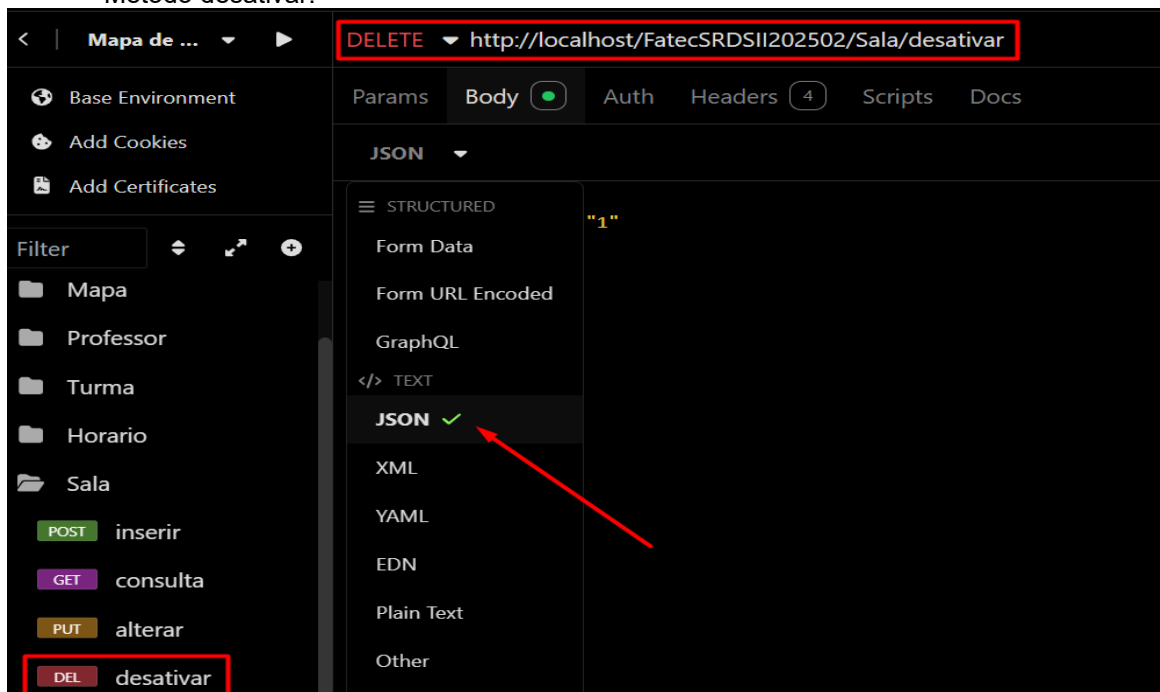
JSON	Preview
<pre>1 { 2 "codigo": "um", 3 "descricao": "teste", 4 "andar": "", 5 "capacidade": "" 6 }</pre>	<pre>1 { 2 "sucesso": false, 3 "erros": [4 { 5 "codigo": 4, 6 "campo": "Codigo", 7 "msg": "Conteúdo não inteiro." 8 } 9] 10 }</pre>

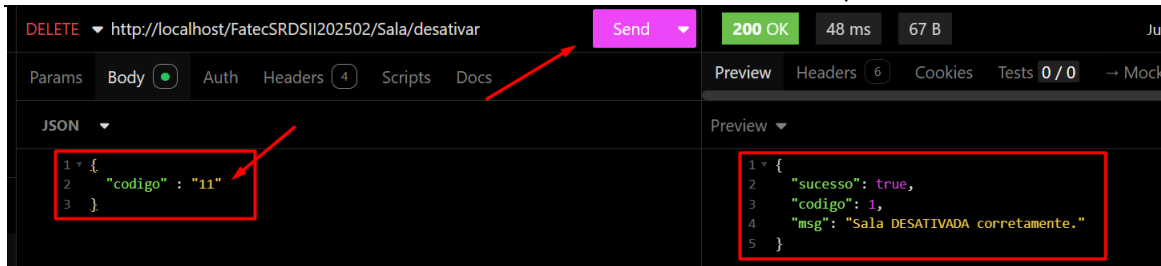
JSON	Preview
<pre>1 { 2 "codigo": "1", 3 "descricao": "Teste", 4 "andar": "", 5 "capacidade": "" 6 } 7</pre>	<pre>1 { 2 "sucesso": false, 3 "erros": [4 { 5 "codigo": 11, 6 "msg": "Sala não encontrada." 7 } 8] 9 }</pre>

JSON	Preview
<pre>1 { 2 "codigo": "11", 3 "descricao": "Teste", 4 "andar": "dois", 5 "capacidade": "" 6 } 7</pre>	<pre>1 { 2 "sucesso": false, 3 "erros": [4 { 5 "codigo": 4, 6 "campo": "Andar", 7 "msg": "Conteúdo não inteiro." 8 } 9] 10 }</pre>

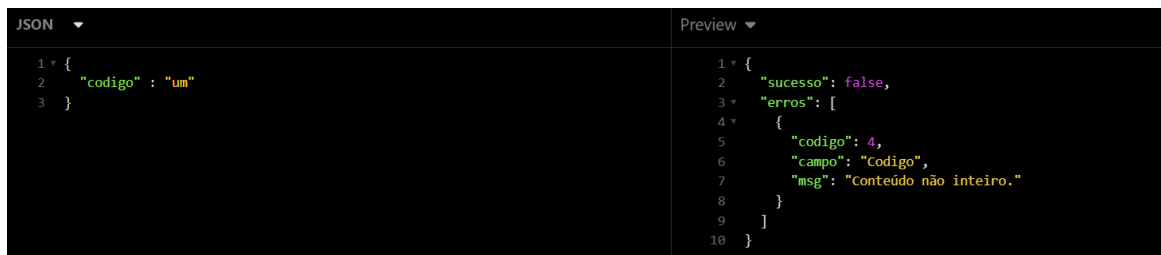
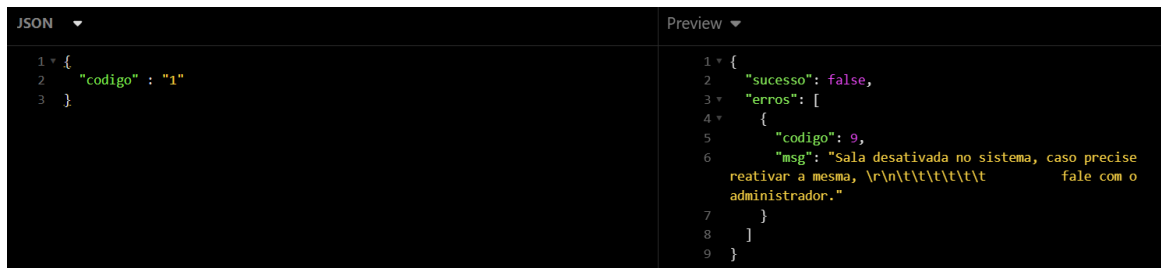
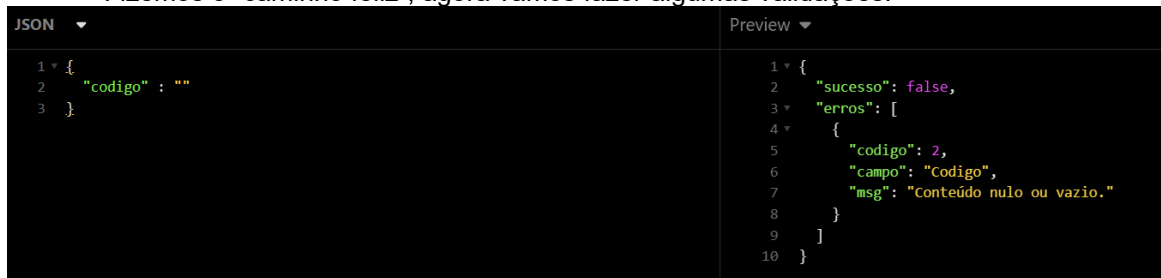
JSON	Preview
<pre>1 { 2 "codigo": "11", 3 "descricao": "Teste", 4 "andar": "2", 5 "capacidade": "quarenta" 6 } 7</pre>	<pre>1 { 2 "sucesso": false, 3 "erros": [4 { 5 "codigo": 4, 6 "campo": "Capacidade", 7 "msg": "Conteúdo não inteiro." 8 } 9] 10 }</pre>

Método desativar:

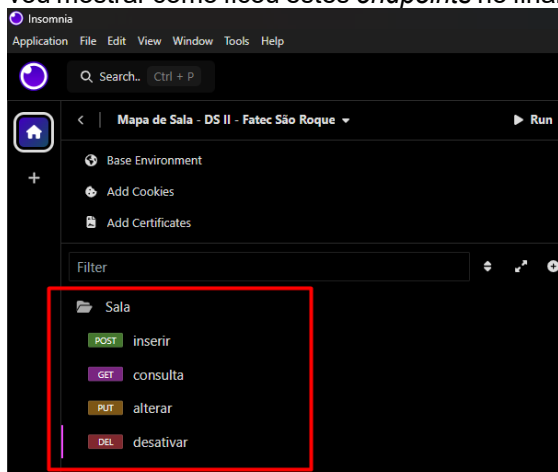




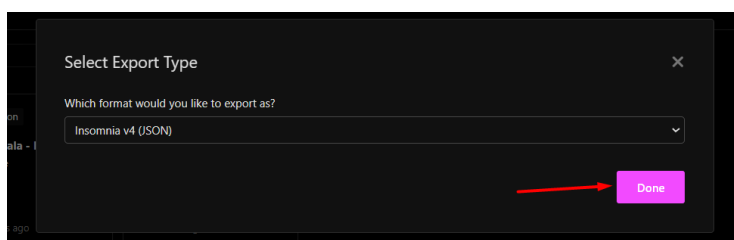
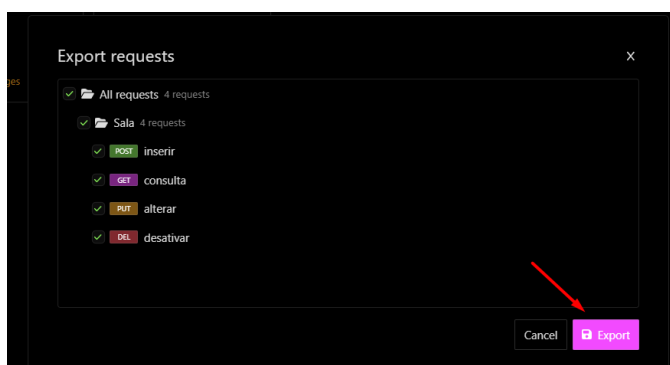
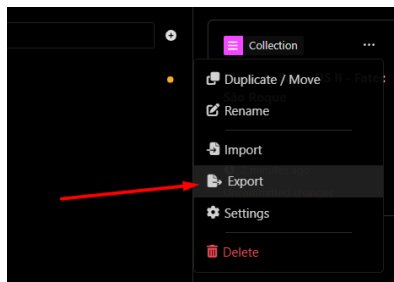
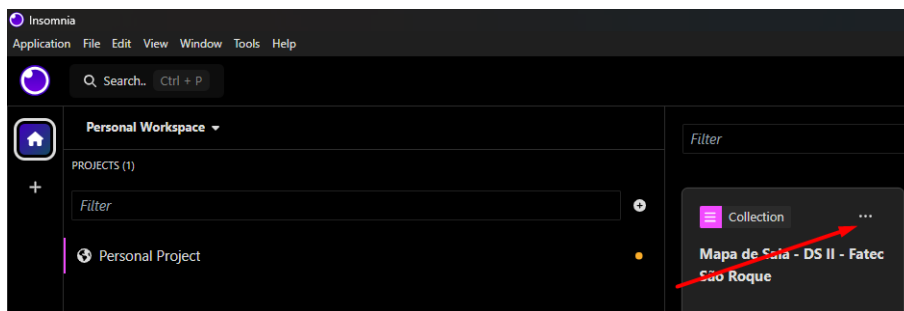
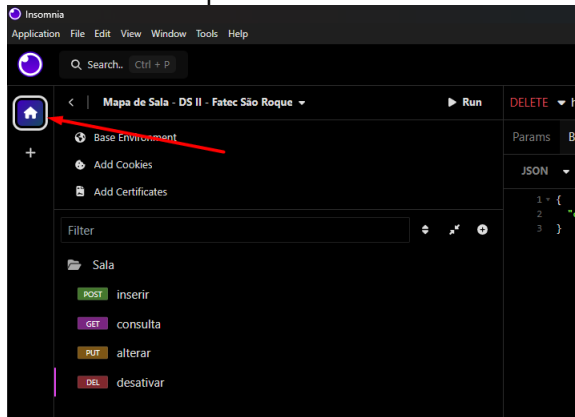
Fizemos o “caminho feliz”, agora vamos fazer algumas validações:



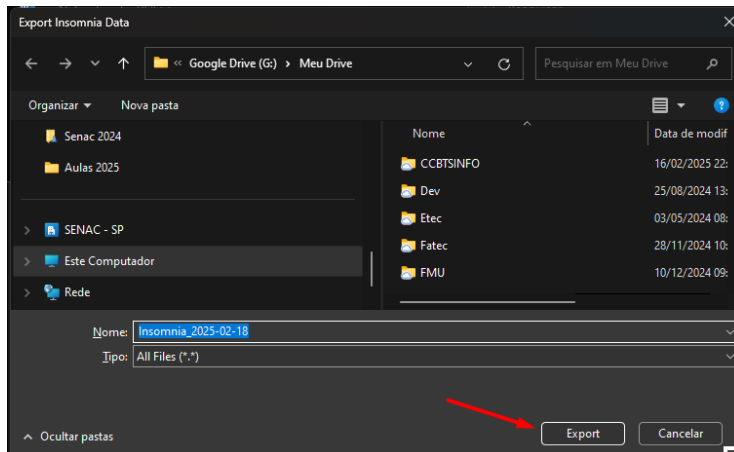
Com isso, finalizamos a implementação de nosso primeiro objeto do sistema, os *endpoints* no *Insomnia* são importantes para o salvamento, recomendo que ao final de cada aula, façam a exportação do mesmo e guardem para que possam atualizar e utilizar durante as aulas, abaixo vou mostrar como ficou estes *endpoints* no final e como exportar os mesmos em um arquivo JSON.



Como exportar:
Retorne para o início:

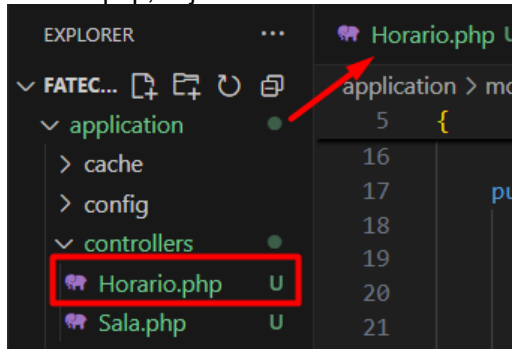


Escolha o local:

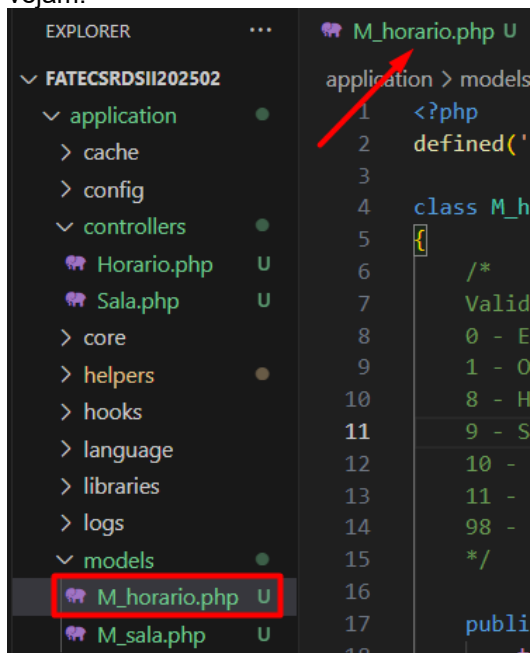


6.5.2 Objeto Horário

Neste objeto vamos tratar também o CRUD (*Create Read Update and Delete*) dos dados pertencentes, inicialmente vamos criar dentro da pasta *controllers* de nosso projeto base o arquivo *Horario.php*, vejamos:



Agora vamos criar dentro da pasta *models* de nosso projeto o arquivo *M_horario.php*, vejamos:



6.5.2.1 Implementação de mais um método na *Helper*

Novamente, vamos implementar mais uma função em nossa **Helper**, para verificar os atributos de hora inicial e final, e também, data inicial e final, para verificarmos se, tanto data e hora iniciais, não são maiores que as finais, chamamos a mesma de `compararDataHora()`, que será mais uma função que nos auxiliará na **Controller** de nosso projeto. Segue:

```
123  /*
124  Função para verificar se datas ou horários iniciais são maiores
125  entre eles
126  */
127  function compararDataHora($valorInicial, $valorFinal, $tipo) {
128      // Passamos a string para hora
129      $valorInicial = strtotime($valorInicial);
130      $valorFinal = strtotime($valorFinal);
131
132      if ($valorInicial != '' && $valorFinal != '') {
133          if ($valorInicial > $valorFinal) {
134              switch ($tipo) {
135                  case 'hora':
136                      return array('codigoHelper' => 13, 'msg' => 'Hora Final menor que a Hora Inicial.');
```

6.5.2.2 *Controller* Horario (*getters* e *setters* e os método `inserir()`, `consultar()`, `alterar()` e `desativar()`)

Vamos inicialmente documentar através de comentários, os retornos possíveis dentro de nossa classe, como fizemos em nossa classe anterior, no caso, a Sala.

Em seguida, vamos codificar a parte de encapsulamento de nossa classe Horario, onde colocaremos os *getters* e *setters*, após isso iremos já fazer todos os métodos de nossa *controller*, a inserção, consulta, alteração e desativação, conforme a seguir:

```
Horario.php X
application > controllers > Horario.php
1  <?php
2  defined('BASEPATH') OR exit('No direct script access allowed');
3
4  class Horario extends CI_Controller {
5      /*
6      Validação dos tipos de retornos nas validações (Código de erro)
7      1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
8      2 - Conteúdo passado nulo ou vazio
9      3 - Conteúdo zerado
10     4 - Conteúdo não inteiro
11     5 - Conteúdo não é um texto
12     6 - Data em formato inválido
13     7 - Hora em formato inválido
14     12 - Na atualização, pelo menos um atributo deve ser passado
15     13 - Hora Final menor que a Hora Inicial
16     14 - Data Final menor que a Data Inicial.
17     99 - Parâmetros passados do front não correspondem ao método
18     */
19
20     //Atributos privados da classe
21     private $codigo;
22     private $descricao;
23     private $horaInicial;
24     private $horaFinal;
25     private $status;
```

```

26
27 //Getters dos atributos
28 public function getCodigo()
29 {
30     return $this->codigo;
31 }
32
33 public function getDescricao()
34 {
35     return $this->descricao;
36 }
37
38 public function getHoraInicial()
39 {
40     return $this->horaInicial;
41 }
42
43 public function getHoraFinal()
44 {
45     return $this->horaFinal;
46 }
47
48 public function getStatus()
49 {
50     return $this->status;
51 }
52
53 //Setters dos atributos
54 public function setCodigo($codigoFront)
55 {
56     $this->codigo = $codigoFront;
57 }
58
59 public function setDescricao($descricaoFront)
60 {
61     $this->descricao = $descricaoFront;
62 }
63
64 public function setHoraInicial($horaInicialFront)
65 {
66     $this->horaInicial = $horaInicialFront;
67 }
68
69 public function setHoraFinal($horaFinalFront)
70 {
71     $this->horaFinal = $horaFinalFront;
72 }
73
74 public function setStatus($statusFront)
75 {
76     $this->status = $statusFront;
77 }
78
79 public function inserir() {
80     //Atributos para controlar o status de nosso método
81     $erros = [];
82     $sucesso = false;
83
84     try {
85
86         $json = file_get_contents('php://input');
87         $resultado = json_decode($json);
88         $lista = [
89             "descricao" => '0',
90             "horaInicial" => '0',
91             "horaFinal" => '0'
92         ];
93

```

```

94     if (verificarParam($resultado, $lista) != 1) {
95         // Validar vindos de forma correta do frontend (Helper)
96         $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
97     }else{
98         // Validar campos quanto ao tipo de dado e tamanho (Helper)
99         $retornoDescricao = validarDados($resultado->descricao,'string',true);
100        $retornoHoraInicial = validarDados($resultado->horaInicial,'hora',true);
101        $retornoHoraFinal = validarDados($resultado->horaFinal,'hora',true);
102        $retornoComparacaoHoras = compararDataHora($resultado->horaInicial,
103                                                    $resultado->horaFinal, 'hora');
104
105        if($retornoDescricao['codigoHelper'] != 0){
106            $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
107                      'campo' => 'Descrição',
108                      'msg' => $retornoDescricao['msg']];
109        }
110
111        if($retornoHoraInicial['codigoHelper'] != 0){
112            $erros[] = ['codigo' => $retornoHoraInicial['codigoHelper'],
113                      'campo' => 'Hora Inicial',
114                      'msg' => $retornoHoraInicial['msg']];
115        }
116
117        if($retornoHoraFinal['codigoHelper'] != 0){
118            $erros[] = ['codigo' => $retornoHoraFinal['codigoHelper'],
119                      'campo' => 'Hora Final',
120                      'msg' => $retornoHoraFinal['msg']];
121        }
122
123        // Validar se a hora inicial é maior que a hora final
124        if($retornoComparacaoHoras['codigoHelper'] != 0){
125            $erros[] = ['codigo' => $retornoComparacaoHoras['codigoHelper'],
126                      'campo' => 'Hora Inicial e Hora Final',
127                      'msg' => $retornoComparacaoHoras['msg']];
128        }
129
130        //Se não encontrar erros
131        if (empty($erros)) {
132            $this->setDescricao($resultado->descricao);
133            $this->setHoraInicial($resultado->horaInicial);
134            $this->setHoraFinal($resultado->horaFinal);
135
136            $this->load->model('M_horario');
137            $resBanco = $this->M_horario->inserir(
138                $this->getDescricao(),
139                $this->getHoraInicial(),
140                $this->getHoraFinal()
141            );
142
143            if ($resBanco['codigo']== 1) {
144                $sucesso = true;
145            } else {
146                // Captura erro do banco
147                $erros[] = [
148                    'codigo' => $resBanco['codigo'],
149                    'msg' => $resBanco['msg']
150                ];
151            }
152        }
153    } catch (Exception $e) {
154        $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
155    }
156
157    // Monta retorno único
158    if ($sucesso == true) {
159        $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
160                  'msg' => $resBanco['msg']];
161    } else {
162        $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
163    }
164

```

```

165
166     // Transforma o array em JSON
167     echo json_encode($retorno);
168 }
169
170 public function consultar() {
171     //Atributos para controlar o status de nosso método
172     $erros = [];
173     $sucesso = false;
174
175     try {
176
177         $json = file_get_contents('php://input');
178         $resultado = json_decode($json);
179         $lista = [
180             "codigo" => '0',
181             "descricao" => '0',
182             "horaInicial" => '0',
183             "horaFinal" => '0'
184         ];
185
186         if (verificarParam($resultado, $lista) != 1) {
187             // Validar vindos de forma correta do frontend (Helper)
188             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
189         } else {
190             // Validar campos quanto ao tipo de dado e tamanho (Helper)
191             $retornoCodigo = validarDadosConsulta($resultado->codigo, 'int');
192             $retornoDescricao = validarDadosConsulta($resultado->descricao, 'string');
193             $retornoHoraInicial = validarDadosConsulta($resultado->horaInicial, 'hora');
194             $retornoHoraFinal = validarDadosConsulta($resultado->horaFinal, 'hora');
195             $retornoComparacaoHoras = compararDataHora($resultado->horaInicial,
196                                                         $resultado->horaFinal, 'hora');
197
198             if($retornoCodigo['codigoHelper'] != 0){
199                 $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
200                             'campo' => 'Codigo',
201                             'msg' => $retornoCodigo['msg']];
202             }
203
204             if($retornoDescricao['codigoHelper'] != 0){
205                 $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
206                             'campo' => 'Descrição',
207                             'msg' => $retornoDescricao['msg']];
208             }
209
210             if($retornoHoraInicial['codigoHelper'] != 0){
211                 $erros[] = ['codigo' => $retornoHoraInicial['codigoHelper'],
212                             'campo' => 'Hora Inicial',
213                             'msg' => $retornoHoraInicial['msg']];
214             }
215
216             if($retornoHoraFinal['codigoHelper'] != 0){
217                 $erros[] = ['codigo' => $retornoHoraFinal['codigoHelper'],
218                             'campo' => 'Hora Final',
219                             'msg' => $retornoHoraFinal['msg']];
220             }
221
222             // Validar se a hora inicial é maior que a hora final
223             if($retornoComparacaoHoras['codigoHelper'] != 0){
224                 $erros[] = ['codigo' => $retornoComparacaoHoras['codigoHelper'],
225                             'campo' => 'Hora Inicial e Hora Final',
226                             'msg' => $retornoComparacaoHoras['msg']];
227             }
228
229             //Se não encontrar erros
230             if (empty($erros)) {
231                 $this->setCodigo($resultado->codigo);

```

```

232     $this->setDescricao($resultado->descricao);
233     $this->setHoraInicial($resultado->horaInicial);
234     $this->setHoraFinal($resultado->horaFinal);
235
236     $this->load->model('M_horario');
237     $resBanco = $this->M_horario->consultar($this->getCodigo(),
238                                           $this->getDescricao(),
239                                           $this->getHoraInicial(),
240                                           $this->getHoraFinal());
241
242     if ($resBanco['codigo'] == 1) {
243         $sucesso = true;
244     } else {
245         // Captura erro do banco
246         $erros[] = [
247             'codigo' => $resBanco['codigo'],
248             'msg' => $resBanco['msg']
249         ];
250     }
251 }
252 }
253 } catch (Exception $e) {
254     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
255 }
256
257 // Monta retorno único
258 if ($sucesso == true) {
259     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
260               'msg' => $resBanco['msg'],
261               'dados' => $resBanco['dados']];
262 } else {
263     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
264 }
265
266 // Transforma o array em JSON
267 echo json_encode($retorno);
268 }
269
270 public function alterar() {
271     //Atributos para controlar o status de nosso método
272     $erros = [];
273     $sucesso = false;
274
275     try {
276
277         $json = file_get_contents('php://input');
278         $resultado = json_decode($json);
279         $lista = [
280             "codigo" => '0',
281             "descricao" => '0',
282             "horaInicial" => '0',
283             "horaFinal" => '0'
284         ];
285
286         if (verificarParam($resultado, $lista) != 1) {
287             // Validar vindos de forma correta do frontend (Helper)
288             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
289         } else {
290             //Pelo menos um dos três parâmetros precisam ter dados para acontecer a atualização
291             if(trim($resultado->descricao) == '' && trim($resultado->horaInicial) == '' &&
292               trim($resultado->horaFinal) == ''){
293                 $erros[] = ['codigo' => 12,
294                           'msg' => 'Pelo menos um parâmetro precisa ser passado para atualização'];
295             } else {
296                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
297                 $retornoCodigo = validarDados($resultado->codigo, 'int', true);
298                 $retornoDescricao = validarDadosConsulta($resultado->descricao, 'string');
299                 $retornoHoraInicial = validarDadosConsulta($resultado->horaInicial, 'hora');
300                 $retornoHoraFinal = validarDadosConsulta($resultado->horaFinal, 'hora');
301                 $retornoComparacaoHoras = compararDataHora($resultado->horaInicial,
302                                                           $resultado->horaFinal, 'hora');

```

```

303
304         if($retornoCodigo['codigoHelper'] != 0){
305             $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
306                         'campo' => 'Codigo',
307                         'msg' => $retornoCodigo['msg']];
308         }
309
310         if($retornoDescricao['codigoHelper'] != 0){
311             $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
312                         'campo' => 'Descrição',
313                         'msg' => $retornoDescricao['msg']];
314         }
315
316         if($retornoHoraInicial['codigoHelper'] != 0){
317             $erros[] = ['codigo' => $retornoHoraInicial['codigoHelper'],
318                         'campo' => 'Hora Inicial',
319                         'msg' => $retornoHoraInicial['msg']];
320         }
321
322         if($retornoHoraFinal['codigoHelper'] != 0){
323             $erros[] = ['codigo' => $retornoHoraFinal['codigoHelper'],
324                         'campo' => 'Hora Final',
325                         'msg' => $retornoHoraFinal['msg']];
326         }
327
328         // Validar se a hora inicial é maior que a hora final
329         if($retornoComparacaoHoras['codigoHelper'] != 0){
330             $erros[] = ['codigo' => $retornoComparacaoHoras['codigoHelper'],
331                         'campo' => 'Hora Inicial e Hora Final',
332                         'msg' => $retornoComparacaoHoras['msg']];
333         }
334
335         //Se não encontrar erros
336         if (empty($erros)) {
337             $this->setCodigo($resultado->codigo);
338             $this->setDescricao($resultado->descricao);
339             $this->setHoraInicial($resultado->horaInicial);
340             $this->setHoraFinal($resultado->horaFinal);
341
342             $this->load->model('M_horario');
343             $resBanco = $this->M_horario->alterar($this->getCodigo(),
344                                                 $this->getDescricao(),
345                                                 $this->getHoraInicial(),
346                                                 $this->getHoraFinal());
347
348             if ($resBanco['codigo']== 1) {
349                 $sucesso = true;
350             } else {
351                 // Captura erro do banco
352                 $erros[] = [
353                     'codigo' => $resBanco['codigo'],
354                     'msg' => $resBanco['msg']
355                 ];
356             }
357         }
358     }
359 }
360 } catch (Exception $e) {
361     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
362 }
363
364 // Monta retorno único
365 if ($sucesso == true) {
366     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
367                 'msg' => $resBanco['msg']];
368 } else {
369     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
370 }

```

```

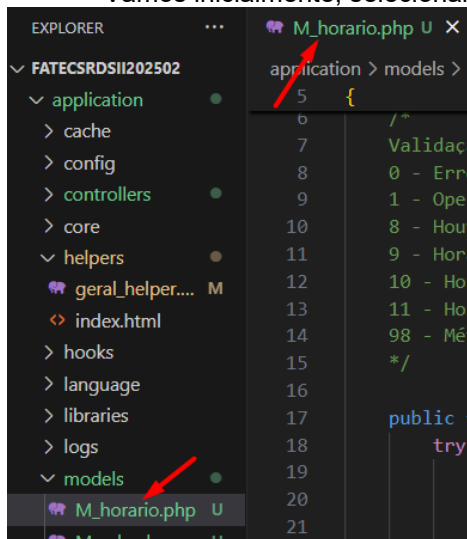
371
372     // Transforma o array em JSON
373     echo json_encode($retorno);
374 }
375
376 public function desativar() {
377     //Atributos para controlar o status de nosso método
378     $erros = [];
379     $sucesso = false;
380
381     try {
382
383         $json = file_get_contents('php://input');
384         $resultado = json_decode($json);
385         $lista = [
386             "codigo" => '0'
387         ];
388
389         if (verificarParam($resultado, $lista) != 1) {
390             // Validar vindos de forma correta do frontend (Helper)
391             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
392         } else {
393             // Validar código quanto ao tipo de dado e tamanho (Helper)
394             $retornoCodigo = validarDados($resultado->codigo, 'int', true);
395
396             if ($retornoCodigo['codigoHelper'] != 0) {
397                 $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
398                     'campo' => 'Codigo',
399                     'msg' => $retornoCodigo['msg']];
400             }
401
402             //Se não encontrar erros
403             if (empty($erros)) {
404                 $this->setCodigo($resultado->codigo);
405
406                 $this->load->model('M_horario');
407                 $resBanco = $this->M_horario->desativar($this->getCodigo());
408
409                 if ($resBanco['codigo'] == 1) {
410                     $sucesso = true;
411                 } else {
412                     // Captura erro do banco
413                     $erros[] = [
414                         'codigo' => $resBanco['codigo'],
415                         'msg' => $resBanco['msg']
416                     ];
417                 }
418             }
419         }
420     } catch (Exception $e) {
421         $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
422     }
423
424     // Monta retorno único
425     if ($sucesso == true) {
426         $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
427             'msg' => $resBanco['msg']];
428     } else {
429         $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
430     }
431
432     // Transforma o array em JSON
433     echo json_encode($retorno);
434 }
435
436 }
437
438 ?>

```

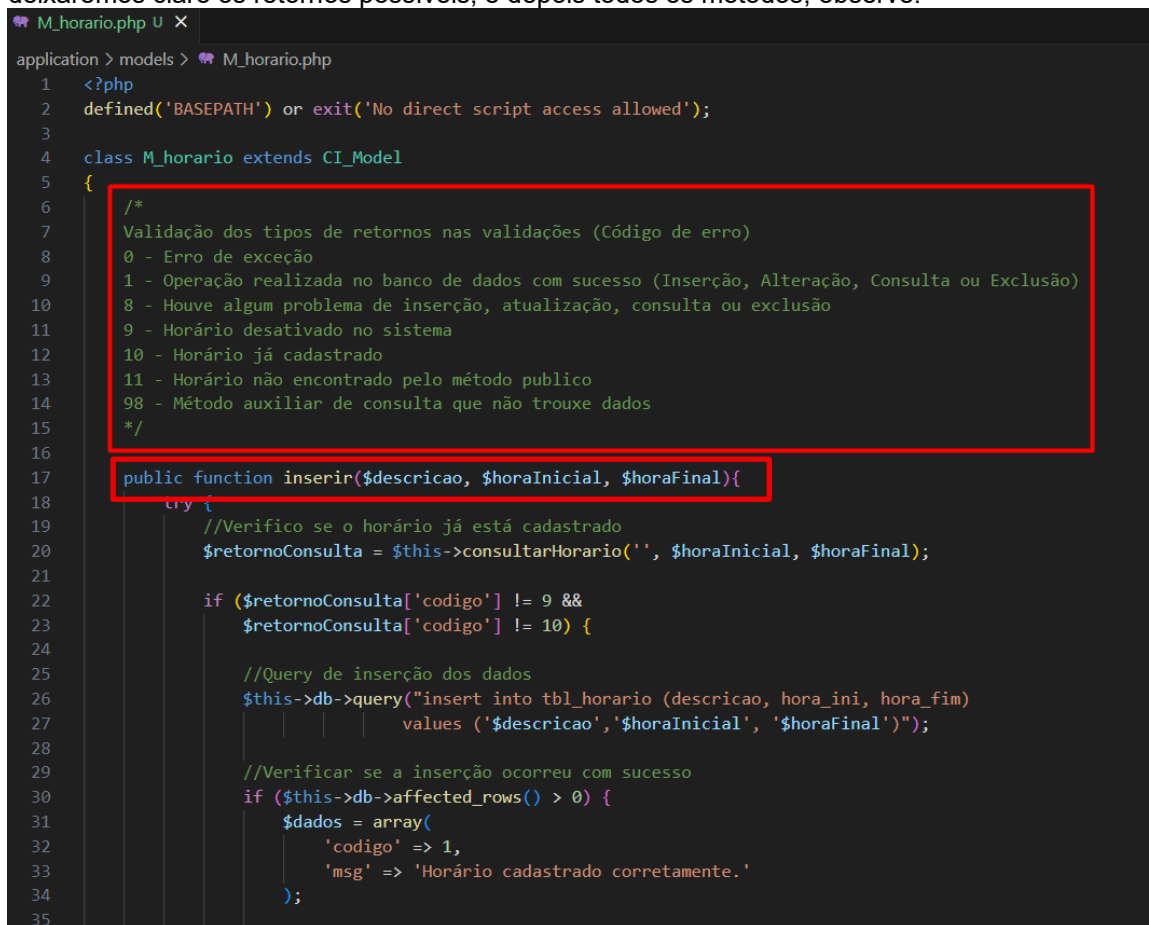
Assim terminamos a *Controller* de Horário.

6.5.2.2 Model M_horario (métodos **inserir()**, **consultar()**, **alterar()** e **desativar()**)

Vamos inicialmente, selecionar a model M_horario que já está criada.



Agora, iremos implementar todos os métodos públicos e auxiliar de nossa classe, lembrando que tanto em consultar, quanto em alterar faremos os métodos que montarão a *query* de forma dinâmica, ou seja, de acordo com os parâmetros passados pela *controller*, a mesma se molda para montar o **UPDATE** ou **SELECT** correto para atender o que é pedido, para isso vamos usar uma série de *If's* para a atualização final, mas vamos começar pelos comentários, onde deixaremos claro os retornos possíveis, e depois todos os métodos, observe:



```

36         } else {
37             $dados = array(
38                 'codigo' => 8,
39                 'msg' => 'Houve algum problema na inserção na tabela de horários.'
40             );
41         }
42     } else {
43         $dados = array('codigo' => $retornoConsulta['codigo'],
44                         'msg' => $retornoConsulta['msg']);
45     }
46 } catch (Exception $e) {
47     $dados = array(
48         'codigo' => 0,
49         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
50     );
51 }
52
53 //Envia o array $dados com as informações tratadas
54 //acima pela estrutura de decisão if
55 return $dados;
56 }
57
58 //Método privado, pois será auxiliar nesta classe
59 private function consultarHorario($codigo, $horaInicial, $horaFinal){
60     try {
61         //Query para consultar dados de acordo com parâmetros passados
62         if($codigo != ''){
63             $sql = "select * from tbl_horario
64                     where codigo = $codigo ";
65         }else{
66             $sql = "select * from tbl_horario
67                     where hora_ini = '$horaInicial'
68                     and hora_fim = '$horaFinal'";
69         }
70         $retornoHorario = $this->db->query($sql);
71
72         //Verificar se a consulta ocorreu com sucesso
73         if($retornoHorario->num_rows() > 0){
74             $linha = $retornoHorario->row();
75             if (trim($linha->estatus) == "D") {
76                 $dados = array(
77                     'codigo' => 9,
78                     'msg' => 'Horário desativado no sistema, caso precise reativar o mesmo,
79                             fale com o administrador.'
80                 );
81             } else {
82                 $dados = array(
83                     'codigo' => 10,
84                     'msg' => 'Horário já cadastrado no sistema.'
85                 );
86             }
87         } else {
88             $dados = array(
89                 'codigo' => 98,
90                 'msg' => 'Horário não encontrado.'
91             );
92         }
93     } catch (Exception $e) {
94         $dados = array(
95             'codigo' => 0,
96             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
97         );
98     }
99
100 //Envia o array $dados com as informações tratadas
101 //acima pela estrutura de decisão if
102 return $dados;
103 }

```

```

104
105 public function consultar($codigo, $descricao, $horaInicial, $horaFinal){
106     try {
107         //Query para consultar dados de acordo com parâmetros passados
108         $sql = "select * from tbl_horario where status = ' ' ";
109
110         if (trim($codigo) != '') {
111             $sql = $sql . " and codigo = $codigo ";
112         }
113
114         if (trim($descricao) != '') {
115             $sql = $sql . " and descricao like '%$descricao%' ";
116         }
117
118         if (trim($horaInicial) != '') {
119             $sql = $sql . " and hora_ini = '$horaInicial' ";
120         }
121
122         if (trim($horaFinal) != '') {
123             $sql = $sql . " and hora_fim = '$horaFinal' ";
124         }
125
126         $sql = $sql . " order by codigo";
127
128         $retorno = $this->db->query($sql);
129
130         //Verificar se a consulta ocorreu com sucesso
131         if ($retorno->num_rows() > 0) {
132             $dados = array(
133                 'codigo' => 1,
134                 'msg' => 'Consulta efetuada com sucesso.',
135                 'dados' => $retorno->result()
136             );
137
138         } else {
139             $dados = array(
140                 'codigo' => 11,
141                 'msg' => 'Horário não encontrado.'
142             );
143         }
144     } catch (Exception $e) {
145         $dados = array(
146             'codigo' => 00,
147             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
148         );
149     }
150     //Envia o array $dados com as informações tratadas
151     //acima pela estrutura de decisão if
152     return $dados;
153 }
154
155 public function alterar($codigo, $descricao, $horaInicial, $horaFinal){
156     try {
157         //Verifico se o horário já está cadastrado
158         $retornoConsulta = $this->consultar($codigo, '', '', '');
159
160         if ($retornoConsulta['codigo'] == 10) {
161             //Inicio a query para atualização
162             $query = "update tbl_horario set ";
163
164             //Vamos comparar os itens
165             if ($descricao != '') {
166                 $query .= "descricao = '$descricao', ";
167             }
168
169             if ($horaInicial != '') {
170                 $query .= "hora_ini = '$horaInicial', ";
171             }

```

```

172
173         if ($horaFinal != '') {
174             $query .= "hora_fim = '$horaFinal', ";
175         }
176
177         //Termino a concatenação da query
178         $queryFinal = rtrim($query, ", ") . " where codigo = $codigo";
179
180         //Executo a Query de atualização dos dados
181         $this->db->query($queryFinal);
182
183         //Verificar se a atualização ocorreu com sucesso
184         if ($this->db->affected_rows() > 0) {
185             $dados = array(
186                 'codigo' => 1,
187                 'msg' => 'Horário atualizado corretamente.'
188             );
189
190         } else {
191             $dados = array(
192                 'codigo' => 8,
193                 'msg' => 'Houve algum problema na atualização na tabela de horário.'
194             );
195         }
196     } else {
197         $dados = array('codigo' => $retornoConsulta['codigo'],
198                        'msg' => $retornoConsulta['msg']);
199     }
200
201 } catch (Exception $e) {
202     $dados = array(
203         'codigo' => 00,
204         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
205     );
206 }

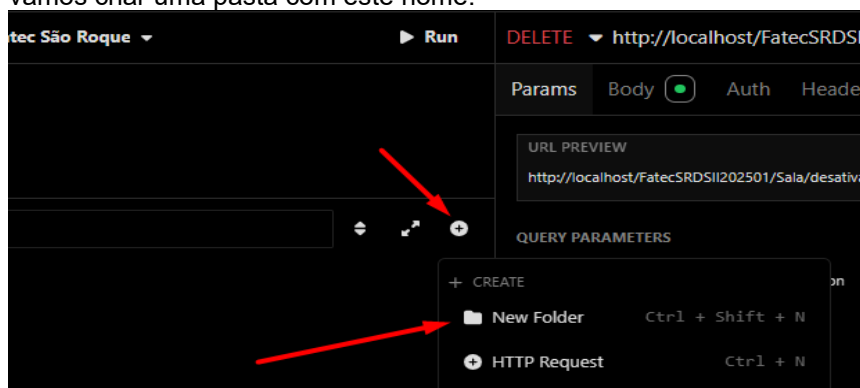
```

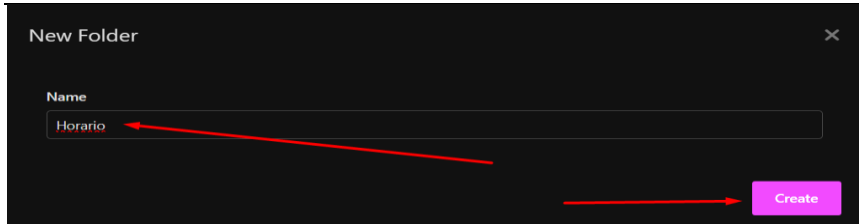
```
207 //Envia o array $dados com as informações tratadas
208 //acima pela estrutura de decisão if
209 return $dados;
210 }
211
212 public function desativar($codigo){
213     try {
214         //Verifico se o horário já está cadastrado
215         $retornoConsulta = $this->consultarHorario($codigo, '', '');
216
217         if ($retornoConsulta['codigo'] == 10) {
218
219             //Query de atualização dos dados
220             $this->db->query("update tbl_horario set status = 'D'
221                             where codigo = $codigo");
222
223             //Verificar se a atualização ocorreu com sucesso
224             if ($this->db->affected_rows() > 0) {
225                 $dados = array(
226                     'codigo' => 1,
227                     'msg' => 'Horário DESATIVADO corretamente.'
228                 );
229             } else {
230                 $dados = array(
231                     'codigo' => 8,
232                     'msg' => 'Houve algum problema na DESATIVAÇÃO do Horário.'
233                 );
234             }
235         } else {
236             $dados = array('codigo' => $retornoConsulta['codigo'],
237                             'msg' => $retornoConsulta['msg']);
238         }
239     } catch (Exception $e) {
240         $dados = array(
241             'codigo' => 00,
242             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
243         );
244     }
245
246     //Envia o array $dados com as informações tratadas
247     //acima pela estrutura de decisão if
248     return $dados;
249 }
250
251 }
252 ?>
253
```

Assim, terminamos a codificação da *Model* de nosso objeto Horário.

6.5.2.3 Objeto Horário no *Insomnia*

A ideia agora é a organização de nosso projeto, como vamos trabalhar com alguns objetos, a ideia é criar uma pasta para cada um, então como estamos trabalhando com o objeto Horário, vamos criar uma pasta com este nome:

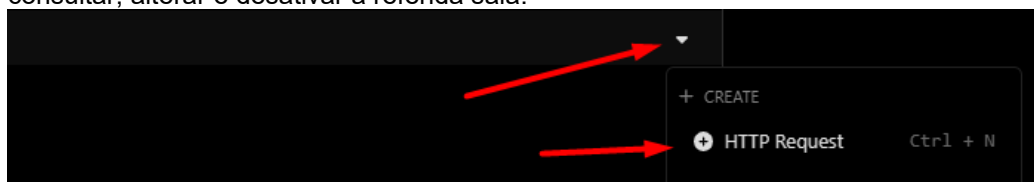




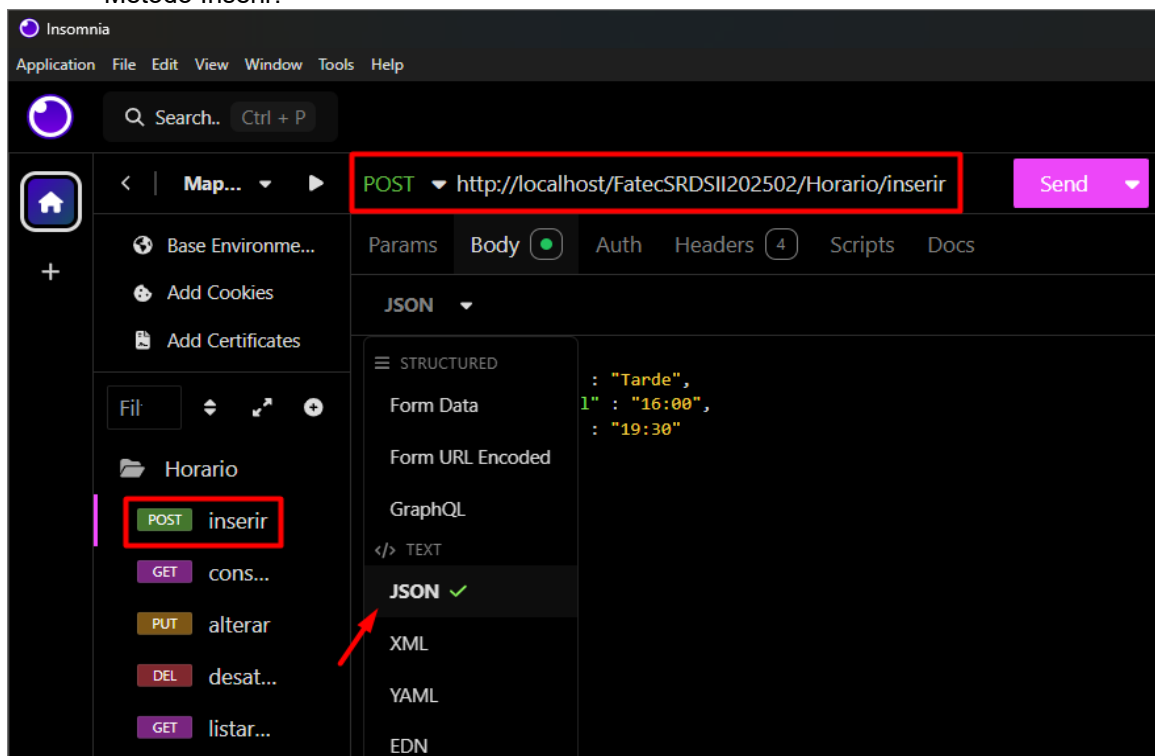
Ficando assim:



Feito isso, agora vamos efetivamente criar os *EndPoints* de nosso objeto, ou seja, incluir, consultar, alterar e desativar a referida sala.



Método Inserir:



POST http://localhost/FatecSRDSII202502/Horario/inserir Send 200 OK 72 ms 75 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0

JSON Preview

```
1 {
2   "descricao": "Manhã",
3   "horaInicial": "08:00",
4   "horaFinal": "12:30"
5 }
6
```

```
1 {
2   "sucesso": true,
3   "codigo": 1,
4   "msg": "Horário cadastrado
5   corretamente."
6 }
```

Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:

POST http://localhost/FatecSRDSII202502/Horario/inserir Send 200 OK 41 ms 253 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0

JSON Preview

```
1 {
2   "descricao": "",
3   "horaInicial": "",
4   "horaFinal": ""
5 }
6
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 2,
6       "campo": "Descrição",
7       "msg": "Conteúdo nulo ou vazio."
8     },
9     {
10      "codigo": 2,
11      "campo": "Hora Inicial",
12      "msg": "Conteúdo nulo ou vazio."
13     },
14     {
15      "codigo": 2,
16      "campo": "Hora Final",
17      "msg": "Conteúdo nulo ou vazio."
18     }
19   ]
20 }
```

POST http://localhost/FatecSRDSII202502/Horario/inserir Send 200 OK 39 ms 173 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0

JSON Preview

```
1 {
2   "descricao": "Manhã",
3   "horaInicial": "",
4   "horaFinal": ""
5 }
6
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 2,
6       "campo": "Hora Inicial",
7       "msg": "Conteúdo nulo ou vazio."
8     },
9     {
10      "codigo": 2,
11      "campo": "Hora Final",
12      "msg": "Conteúdo nulo ou vazio."
13     }
14   ]
15 }
```

POST http://localhost/FatecSRDSII202502/Horario/inserir Send 200 OK 51 ms 104 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0

JSON Preview

```
1 {
2   "descricao": "Manhã",
3   "horaInicial": "44:00",
4   "horaFinal": "12:00"
5 }
6
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 7,
6       "campo": "Hora Inicial",
7       "msg": "Hora em formato inválido."
8     }
9   ]
10 }
```

POST http://localhost/FatecSRDSII202502/Horario/inserir Send 200 OK 59 ms 102 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0

JSON Preview

```
1 {
2   "descricao": "Manhã",
3   "horaInicial": "08:00",
4   "horaFinal": "55:00"
5 }
6
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 7,
6       "campo": "Hora Final",
7       "msg": "Hora em formato inválido."
8     }
9   ]
10 }
```

POST http://localhost/FatecSRDSII202502/Horario/inserir Send 200 OK 37 ms 124 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0

JSON Preview

```
1 {
2   "descricao": "Manhã",
3   "horaInicial": "12:00",
4   "horaFinal": "08:00"
5 }
6
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 13,
6       "campo": "Hora Inicial e Hora
Final",
7       "msg": "Hora Final menor que a Hora
Inicial."
8     }
9   ]
10 }
```

POST http://localhost/FatecSRDSII202502/Horario/inserir Send 200 OK 56 ms 95 B Just Now

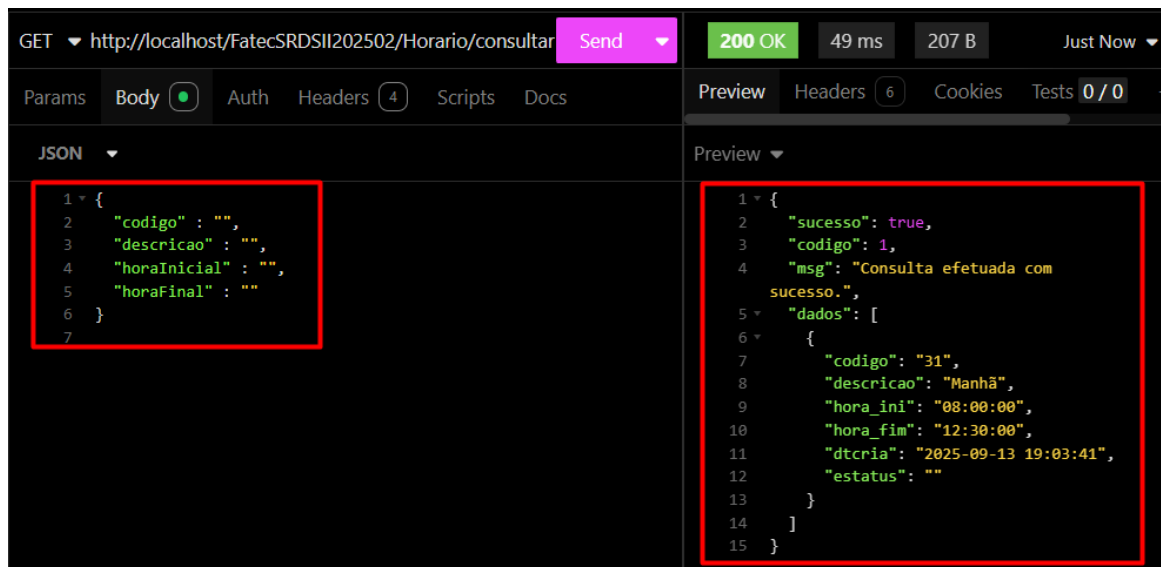
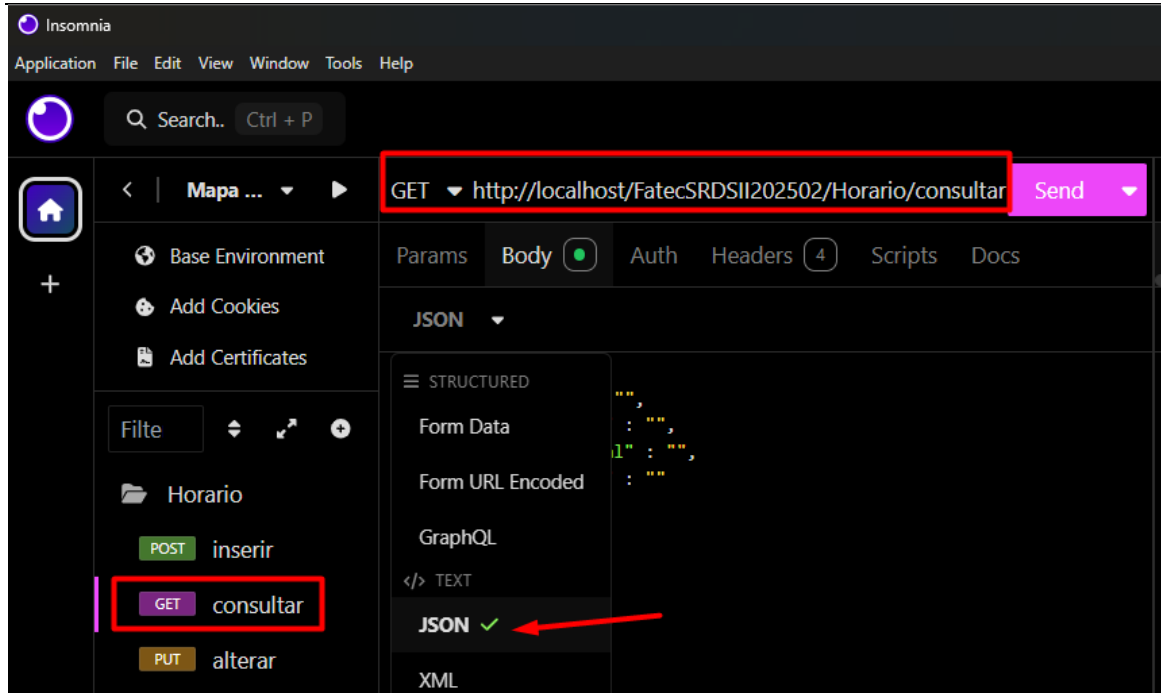
Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0

JSON Preview

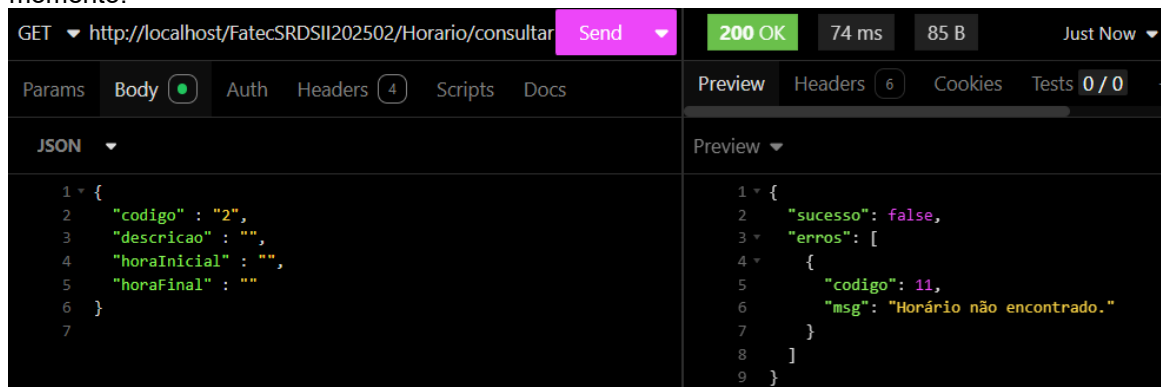
```
1 {
2   "descricao": "Manhã",
3   "horaInicial": "08:00",
4   "horaFinal": "12:00"
5 }
6
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 10,
6       "msg": "Horário já cadastrado no
sistema."
7     }
8   ]
9 }
```

Método consultar:



Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:



GET <http://localhost/FatecSRDSII202502/Horario/consultar> Send 200 OK 69 ms 99 B Just Now

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON

```

1 {
2   "codigo" : "dois",
3   "descricao" : "",
4   "horaInicial" : "",
5   "horaFinal" : ""
6 }
7

```

Preview

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Codigo",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }

```

GET <http://localhost/FatecSRDSII202502/Horario/consultar> Send 200 OK 39 ms 104 B Just Now

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON

```

1 {
2   "codigo" : "",
3   "descricao" : "",
4   "horaInicial" : "duas da tarde",
5   "horaFinal" : ""
6 }
7

```

Preview

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 7,
6       "campo": "Hora Inicial",
7       "msg": "Hora em formato inválido."
8     }
9   ]
10 }

```

GET <http://localhost/FatecSRDSII202502/Horario/consultar> Send 200 OK 47 ms 104 B Just Now

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON

```

1 {
2   "codigo" : "",
3   "descricao" : "",
4   "horaInicial" : "67:00",
5   "horaFinal" : ""
6 }
7

```

Preview

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 7,
6       "campo": "Hora Inicial",
7       "msg": "Hora em formato inválido."
8     }
9   ]
10 }

```

GET <http://localhost/FatecSRDSII202502/Horario/consultar> Send 200 OK 45 ms 102 B Just Now

Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON

```

1 {
2   "codigo" : "",
3   "descricao" : "",
4   "horaInicial" : "",
5   "horaFinal" : "três"
6 }
7

```

Preview

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 7,
6       "campo": "Hora Final",
7       "msg": "Hora em formato inválido."
8     }
9   ]
10 }

```

GET **http://localhost/FatecSRDSII202502/Horario/consultar** **Send** **200 OK** 40 ms 124 B Just Now

Params Body **Auth** Headers (4) Scripts Docs

JSON

```
1 {
2   "codigo" : "",
3   "descricao" : "",
4   "horaInicial" : "12:00",
5   "horaFinal" : "09:00"
6 }
7
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 13,
6       "campo": "Hora Inicial e Hora
7         Final",
8       "msg": "Hora Final menor que a Hora
9         Inicial."
10    }
11  ]
12 }
```

Método alterar:

Insomnia

Application File Edit View Window Tools Help

Search.. Ctrl + P

PUT **http://localhost/FatecSRDSII202502/Horario/alterar** **Send**

Base Environment
Add Cookies
Add Certificates

Filte

Horario

POST inserir
GET consultar
PUT alterar
DEL desativar

Params Body **Auth** Headers (4) Scripts Docs

JSON

STRUCTURED

Form Data
Form URL Encoded
GraphQL
TEXT
JSON ✓
XML
YAML

```
"1",
: "noite",
1" : "95:00",
: "88:00"
```

PUT **http://localhost/FatecSRDSII202502/Horario/alterar** **Send** **200 OK** 84 ms 75 B Just Now

Params Body **Auth** Headers (4) Scripts Docs

JSON

```
1 {
2   "codigo" : "31",
3   "descricao" : "Manhã",
4   "horaInicial" : "07:00",
5   "horaFinal" : "11:00"
6 }
7
```

Preview

```
1 {
2   "sucesso": true,
3   "codigo": 1,
4   "msg": "Horário atualizado
5     corretamente."
6 }
```

Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:

PUT ▼ http://localhost/FatecSRDSII202502/Horario/alterar Send ▼ 200 OK 88 ms 127 B Just Now ▼

Params Body ☒ Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON ▼ Preview ▼

```

1 {
2   "codigo" : "",
3   "descricao" : "",
4   "horaInicial" : "",
5   "horaFinal" : ""
6 }
7

```

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 12,
6       "msg": "Pelo menos um parâmetro
precisa ser passado para atualização"
7     }
8   ]
9 }

```

PUT ▼ http://localhost/FatecSRDSII202502/Horario/alterar Send ▼ 200 OK 57 ms 99 B Just Now ▼

Params Body ☒ Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON ▼ Preview ▼

```

1 {
2   "codigo" : "um",
3   "descricao" : "Teste",
4   "horaInicial" : "",
5   "horaFinal" : ""
6 }
7

```

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Codigo",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }

```

PUT ▼ http://localhost/FatecSRDSII202502/Horario/alterar Send ▼ 200 OK 49 ms 85 B Just Now ▼

Params Body ☒ Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON ▼ Preview ▼

```

1 {
2   "codigo" : "32",
3   "descricao" : "Teste",
4   "horaInicial" : "09:00",
5   "horaFinal" : ""
6 }
7

```

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 11,
6       "msg": "Horário não encontrado."
7     }
8   ]
9 }

```

PUT ▼ http://localhost/FatecSRDSII202502/Horario/alterar Send ▼ 200 OK 52 ms 104 B Just Now ▼

Params Body ☒ Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0

JSON ▼ Preview ▼

```

1 {
2   "codigo" : "32",
3   "descricao" : "Teste",
4   "horaInicial" : "67:00",
5   "horaFinal" : ""
6 }
7

```

```

1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 7,
6       "campo": "Hora Inicial",
7       "msg": "Hora em formato inválido."
8     }
9   ]
10 }

```

PUT http://localhost/FatecSRDSII202502/Horario/alterar Send 200 OK 82 ms 102 B Just Now

Params Body Auth Headers 4 Scripts Docs

JSON

```
1 {
2   "codigo" : "32",
3   "descricao" : "Teste",
4   "horaInicial" : "",
5   "horaFinal" : "99:00"
6 }
7
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 7,
6       "campo": "Hora Final",
7       "msg": "Hora em formato inválido."
8     }
9   ]
10 }
```

Método desativar:

Insomnia

Application File Edit View Window Tools Help

Search.. Ctrl + P

Mapa ... DELETE http://localhost/FatecSRDSII202502/Horario/desativar Send

Base Environment Add Cookies Add Certificates

Filte

Horário

POST inserir

GET consultar

PUT alterar

DEL desativar

Params Body Auth Headers 4 Scripts Docs

JSON

STRUCTURED

Form Data

Form URL Encoded

GraphQL

TEXT

JSON ✓

XML

YAML

DELETE http://localhost/FatecSRDSII202502/Horario/desativar Send 200 OK 58 ms 75 B Just Now

Params Body Auth Headers 4 Scripts Docs

JSON

```
1 {
2   "codigo" : "31"
3 }
4
```

Preview

```
1 {
2   "sucesso": true,
3   "codigo": 1,
4   "msg": "Horário DESATIVADO corretamente."
5 }
```

Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:

DELETE ▼ http://localhost/FatecSRDSII202502/Horario/desativar Send ▼ 200 OK 61 ms 96 B Just Now ▼

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0 / 0

JSON ▼ Preview ▼

```
1 {
2   "codigo" : ""
3 }
4
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 2,
6       "campo": "Codigo",
7       "msg": "Conteúdo nulo ou vazio."
8     }
9   ]
10 }
```

DELETE ▼ http://localhost/FatecSRDSII202502/Horario/desativar Send ▼ 200 OK 47 ms 99 B Just Now ▼

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0 / 0

JSON ▼ Preview ▼

```
1 {
2   "codigo" : "dois"
3 }
4
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Codigo",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```

DELETE ▼ http://localhost/FatecSRDSII202502/Horario/desativar Send ▼ 200 OK 49 ms 85 B Just Now ▼

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0 / 0

JSON ▼ Preview ▼

```
1 {
2   "codigo" : "32"
3 }
4
```

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 98,
6       "msg": "Horário não encontrado."
7     }
8   ]
9 }
```

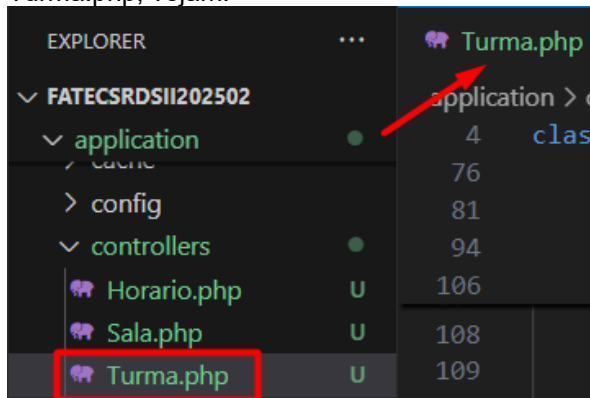

Horário:

- Pesquisa realizada com sucesso;
- Cadastro efetuado com sucesso;
- Alteração efetuada com sucesso;
- Desativação efetuada com sucesso.

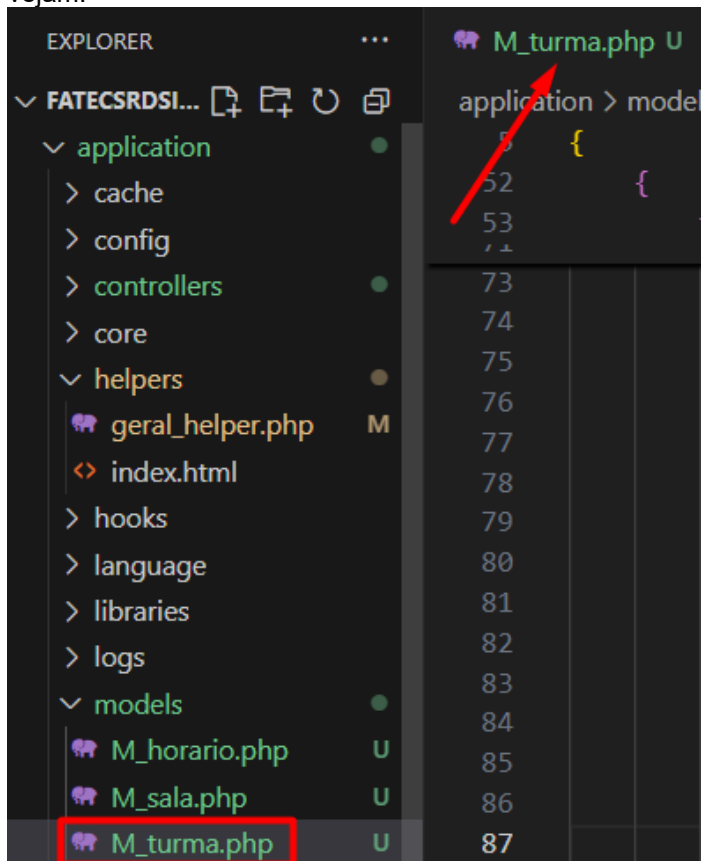
Observação: Irei validar os *endpoints* com testes funcionais, ou seja, irei “marretar” para ver se todas as funções foram codificadas como instruído em aula.

6.5.4 Objeto Turma

Neste objeto vamos tratar também o CRUD (*Create Read Update and Delete*) dos dados pertencentes, inicialmente vamos criar dentro da pasta *controllers* de nosso projeto base o arquivo *Turma.php*, vejam:



Agora vamos criar dentro da pasta *models* de nosso projeto o arquivo *M_turma.php*, vejam:



6.5.4.1 *Controller* Turma (*getters* e *setters* e os métodos *inserir()* e *consultar()*)

Vamos inicialmente documentar através de comentários, os retornos possíveis dentro de nossa classe, como fizemos em nossa classe anterior, no caso, a Turma.

Em seguida, vamos codificar a parte de encapsulamento de nossa classe Turma, onde colocaremos os *getters* e *setters*, após isso iremos já fazer os métodos de inserção e consulta dos dados de nossa *controller*, conforme a seguir:

```
Turma.php U X
application > controllers > Turma.php
1  <?php
2  defined('BASEPATH') OR exit('No direct script access allowed');
3
4  class Turma extends CI_Controller {
5      /*
6       * Validação dos tipos de retornos nas validações (Código de erro)
7       * 1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
8       * 2 - Conteúdo passado nulo ou vazio
9       * 3 - Conteúdo zerado
10      * 4 - Conteúdo não inteiro
11      * 5 - Conteúdo não é um texto
12      * 6 - Data em formato inválido
13      * 12 - Na atualização, pelo menos um atributo deve ser passado
14      * 99 - Parâmetros passados do front não correspondem ao método
15      */
16
17      //Atributos privados da classe
18      private $codigo;
19      private $descricao;
20      private $capacidade;
21      private $dataInicio;
22      private $status;
23
24      //Getters dos atributos
25      public function getCodigo()
26      {
27          return $this->codigo;
28      }
29
30      public function getDescricao()
31      {
32          return $this->descricao;
33      }
34
35      public function getCapacidade()
36      {
37          return $this->capacidade;
38      }
39
40      public function getDataInicio()
41      {
42          return $this->dataInicio;
43      }
44
45      public function getStatus()
46      {
47          return $this->status;
48      }
49
50      //Setters dos atributos
51      public function setCodigo($codigoFront)
52      {
53          $this->codigo = $codigoFront;
54      }
55
56      public function setDescricao($descricaoFront)
57      {
58          $this->descricao = $descricaoFront;
59      }
60  }
```

```

60
61 public function setCapacidade($capacidadeFront)
62 {
63     $this->capacidade = $capacidadeFront;
64 }
65
66 public function setDataInicio($dataInicioFront)
67 {
68     $this->dataInicio = $dataInicioFront;
69 }
70
71 public function setStatus($statusFront)
72 {
73     $this->tipoUsuario = $statusFront;
74 }
75
76 public function inserir() {
77     //Atributos para controlar o status de nosso método
78     $erros = [];
79     $sucesso = false;
80
81     try {
82
83         $json = file_get_contents('php://input');
84         $resultado = json_decode($json);
85         $lista = [
86             "descricao" => '0',
87             "capacidade" => '0',
88             "dataInicio" => '0'
89         ];
90
91         if (verificarParam($resultado, $lista) != 1) {
92             // Validar vindos de forma correta do frontend (Helper)
93             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no
94                 | | | | | FrontEnd.'];
95         }else{
96             // Validar campos quanto ao tipo de dado e tamanho (Helper)
97             $retornoDescricao = validarDados($resultado->descricao, 'string', true);
98             $retornoCapacidade = validarDados($resultado->capacidade, 'int', true);
99             $retornoDataInicio = validarDados($resultado->dataInicio, 'date', true);
100
101             if($retornoDescricao['codigoHelper'] != 0){
102                 $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
103                     | | | | | 'campo' => 'Descrição',
104                     | | | | | 'msg' => $retornoDescricao['msg']];
105             }
106
107             if($retornoCapacidade['codigoHelper'] != 0){
108                 $erros[] = ['codigo' => $retornoCapacidade['codigoHelper'],
109                     | | | | | 'campo' => 'Capacidade',
110                     | | | | | 'msg' => $retornoCapacidade['msg']];
111             }
112
113             if($retornoDataInicio['codigoHelper'] != 0){
114                 $erros[] = ['codigo' => $retornoDataInicio['codigoHelper'],
115                     | | | | | 'campo' => 'Andar',
116                     | | | | | 'msg' => $retornoDataInicio['msg']];
117             }
118
119             //Se não encontrar erros
120             if (empty($erros)) {
121                 $this->setDescricao($resultado->descricao);
122                 $this->setCapacidade($resultado->capacidade);
123                 $this->setDataInicio($resultado->dataInicio);
124
125                 $this->load->model('M_turma');
126                 $resBanco = $this->M_turma->inserir(
127                     $this->getDescricao(),

```

```

128         $this->getCapacidade(),
129         $this->getDataInicio()
130     );
131
132     if ($resBanco['codigo'] == 1) {
133         $sucesso = true;
134     } else {
135         // Captura erro do banco
136         $erros[] = [
137             'codigo' => $resBanco['codigo'],
138             'msg' => $resBanco['msg']
139         ];
140     }
141 }
142 }
143 } catch (Exception $e) {
144     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
145 }
146
147 // Monta retorno único
148 if ($sucesso == true) {
149     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
150                 'msg' => $resBanco['msg']];
151 } else {
152     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
153 }
154
155 // Transforma o array em JSON
156 echo json_encode($retorno);
157 }
158
159 public function consultar() {
160     //Atributos para controlar o status de nosso método
161     $erros = [];
162     $sucesso = false;
163
164     try {
165
166         $json = file_get_contents('php://input');
167         $resultado = json_decode($json);
168         $lista = [
169             "codigo" => '0',
170             "descricao" => '0',
171             "capacidade" => '0',
172             "dataInicio" => '0'
173         ];
174
175         if (verificarParam($resultado, $lista) != 1) {
176             // Validar vindos de forma correta do frontend (Helper)
177             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
178         } else {
179             // Validar campos quanto ao tipo de dado e tamanho (Helper)
180             $retornoCodigo = validarDadosConsulta($resultado->codigo, 'int');
181             $retornoDescricao = validarDadosConsulta($resultado->descricao, 'string');
182             $retornoCapacidade = validarDadosConsulta($resultado->capacidade, 'int');
183             $retornoDataInicio = validarDadosConsulta($resultado->dataInicio, 'date');
184
185             if ($retornoCodigo['codigoHelper'] != 0) {
186                 $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
187                             'campo' => 'Codigo',
188                             'msg' => $retornoCodigo['msg']];
189             }
190         }
191     }

```

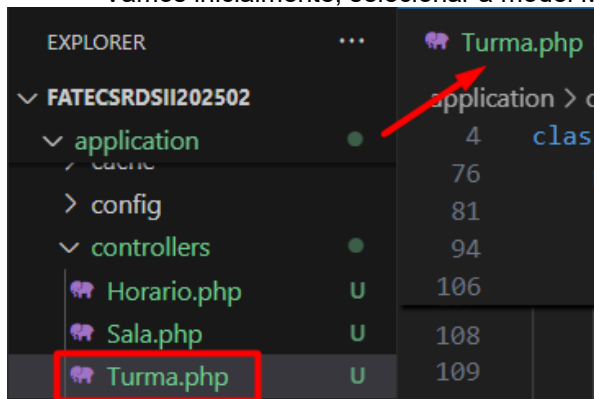
```

191
192         if($retornoDescricao['codigoHelper'] != 0){
193             $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
194                         'campo' => 'Descrição',
195                         'msg' => $retornoDescricao['msg']];
196         }
197
198         if($retornoCapacidade['codigoHelper'] != 0){
199             $erros[] = ['codigo' => $retornoCapacidade['codigoHelper'],
200                         'campo' => 'Capacidade',
201                         'msg' => $retornoCapacidade['msg']];
202         }
203
204         if($retornoDataInicio['codigoHelper'] != 0){
205             $erros[] = ['codigo' => $retornoDataInicio['codigoHelper'],
206                         'campo' => 'Andar',
207                         'msg' => $retornoDataInicio['msg']];
208         }
209
210         //Se não encontrar erros
211         if (empty($erros)) {
212             $this->setCodigo($resultado->codigo);
213             $this->setDescricao($resultado->descricao);
214             $this->setCapacidade($resultado->capacidade);
215             $this->setDataInicio($resultado->dataInicio);
216
217             $this->load->model('M_turma');
218             $resBanco = $this->M_turma->consultar($this->getCodigo(),
219                                                 $this->getDescricao(),
220                                                 $this->getCapacidade(),
221                                                 $this->getDataInicio());
222
223             if ($resBanco['codigo']== 1) {
224                 $sucesso = true;
225             } else {
226                 // Captura erro do banco
227                 $erros[] = [
228                     'codigo' => $resBanco['codigo'],
229                     'msg' => $resBanco['msg']
230                 ];
231             }
232         }
233     }
234 } catch (Exception $e) {
235     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
236 }
237
238 // Monta retorno único
239 if ($sucesso == true) {
240     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
241                 'msg' => $resBanco['msg'],
242                 'dados' => $resBanco['dados']];
243 } else {
244     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
245 }
246
247 // Transforma o array em JSON
248 echo json_encode($retorno);
249 }
250 }

```

6.5.4.2 Model M_horario (métodos **inserir()** e **consultar()**)

Vamos inicialmente, selecionar a model M_turma que já está criada.



Agora, iremos implementar os métodos inserir e consultar de nossa classe, observe:

```

M_turma.php U X
application > models > M_turma.php
1  <?php
2  defined('BASEPATH') or exit('No direct script access allowed');
3
4  class M_turma extends CI_Model
5  {
6      /*
7       * Validação dos tipos de retornos nas validações (Código de erro)
8       * 0 - Erro de exceção
9       * 1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
10      * 8 - Houve algum problema de inserção, atualização, consulta ou exclusão
11      * 9 - Turma desativada no sistema
12      * 10 - Turma já cadastrada
13      * 11 - Turma não encontrada pelo método publico
14      * 98 - Método auxiliar de consulta que não trouxe dados
15      */
16
17      public function inserir($descricao, $capacidade, $dataInicio)
18      {
19          try {
20
21              //Query de inserção dos dados
22              $this->db->query("insert into tbl_turma (descricao, capacidade, dataInicio)
23                  values ('$descricao', $capacidade, '$dataInicio')");
24
25              //Verificar se a inserção ocorreu com sucesso
26              if ($this->db->affected_rows() > 0) {
27                  $dados = array(
28                      'codigo' => 1,
29                      'msg' => 'Turma cadastrada corretamente.'
30                  );
31
32              } else {
33                  $dados = array(
34                      'codigo' => 8,
35                      'msg' => 'Houve algum problema na inserção na tabela de turma.'
36                  );
37
38              }
39
40          } catch (Exception $e) {
41              $dados = array(
42                  'codigo' => 0,
43                  'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
44              );
45          }
46      }
47  }

```

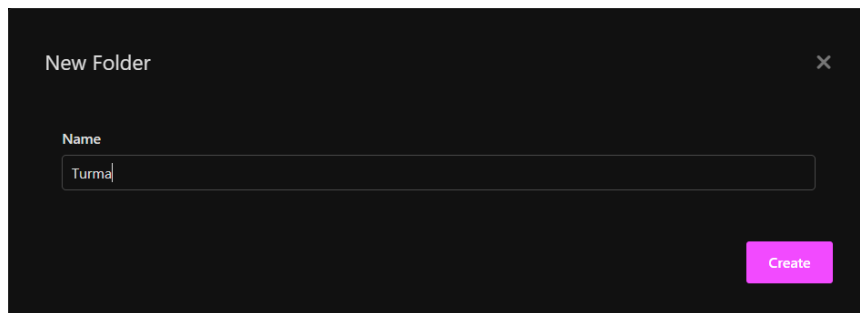
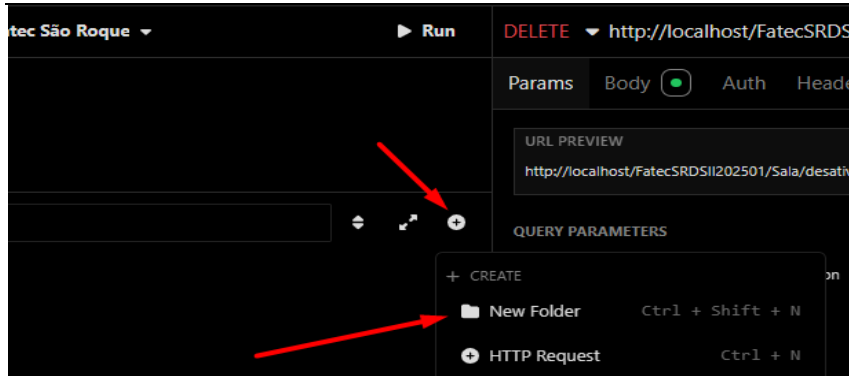
```

46         //Envia o array $dados com as informações tratadas
47         //acima pela estrutura de decisão if
48         return $dados;
49     }
50
51     public function consultar($codigo, $descricao, $capacidade, $dataInicio)
52     {
53         try {
54             //Query para consultar dados de acordo com parâmetros passados
55             $sql = "select codigo, descricao, capacidade, dataInicio,
56                 date_format(dataInicio,'%d-%m-%Y') dataIniciobra
57                 from tbl_turma where estatus = '' ";
58
59             if (trim($codigo) != '') {
60                 $sql = $sql . "and codigo = $codigo ";
61             }
62
63             if (trim($descricao) != '') {
64                 $sql = $sql . "and descricao like '%$descricao%' ";
65             }
66
67             if (trim($capacidade) != '') {
68                 $sql = $sql . "and capacidade = $capacidade ";
69             }
70
71             if (trim($dataInicio != '')) {
72                 $sql = $sql . "and dataInicio = '$dataInicio' ";
73             }
74
75             $retorno = $this->db->query($sql);
76
77             //Verificar se a consulta ocorreu com sucesso
78             if ($retorno->num_rows() > 0) {
79                 $dados = array(
80                     'codigo' => 1,
81                     'msg' => 'Consulta efetuada com sucesso',
82                     'dados' => $retorno->result()
83                 );
84             } else {
85                 $dados = array(
86                     'codigo' => 11,
87                     'msg' => 'Turma não encontrado.'
88                 );
89             }
90
91         } catch (Exception $e) {
92             $dados = array(
93                 'codigo' => 00,
94                 'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
95             );
96         }
97         //Envia o array $dados com as informações tratadas
98         //acima pela estrutura de decisão if
99         return $dados;
100     }
101 }

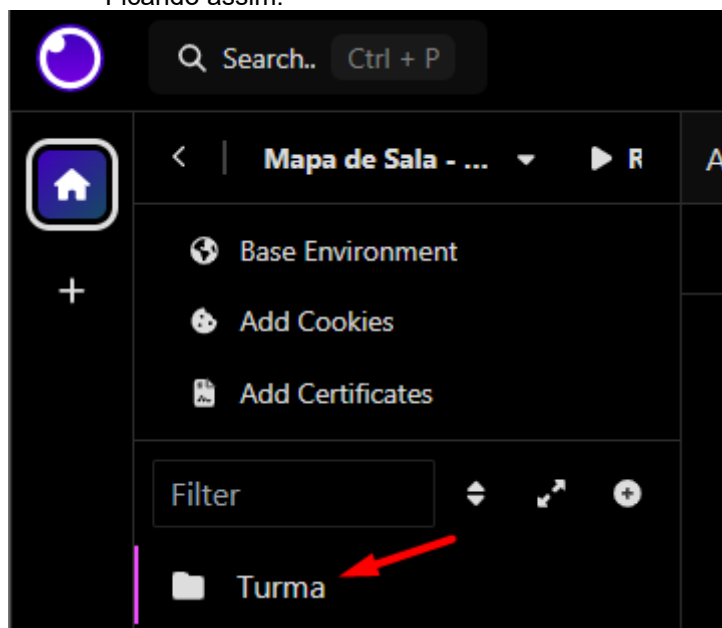
```

6.5.4.3 Objeto Turma no *Insomnia*

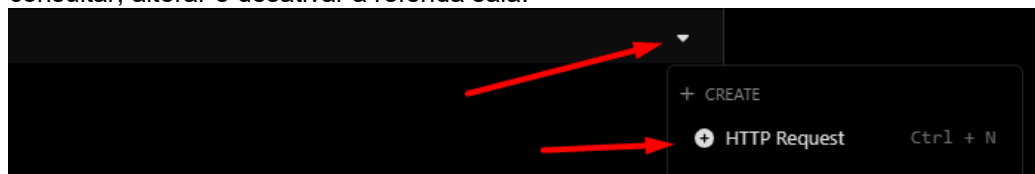
A ideia agora é a organização de nosso projeto, como vamos trabalhar com alguns objetos, a ideia é criar uma pasta para cada um, então como estamos trabalhando com o objeto Turma, vamos criar uma pasta com este nome:



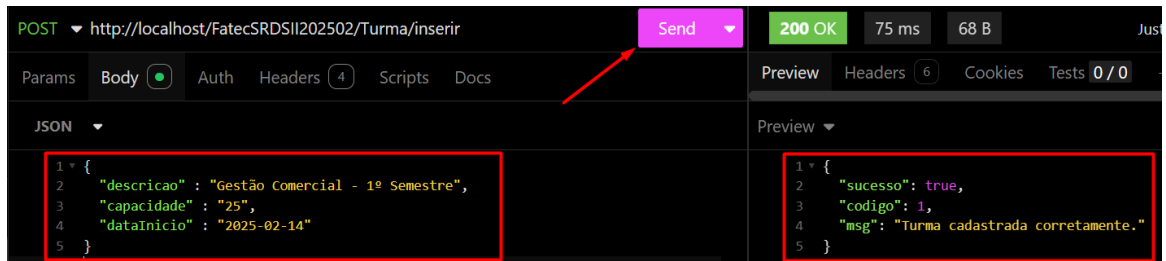
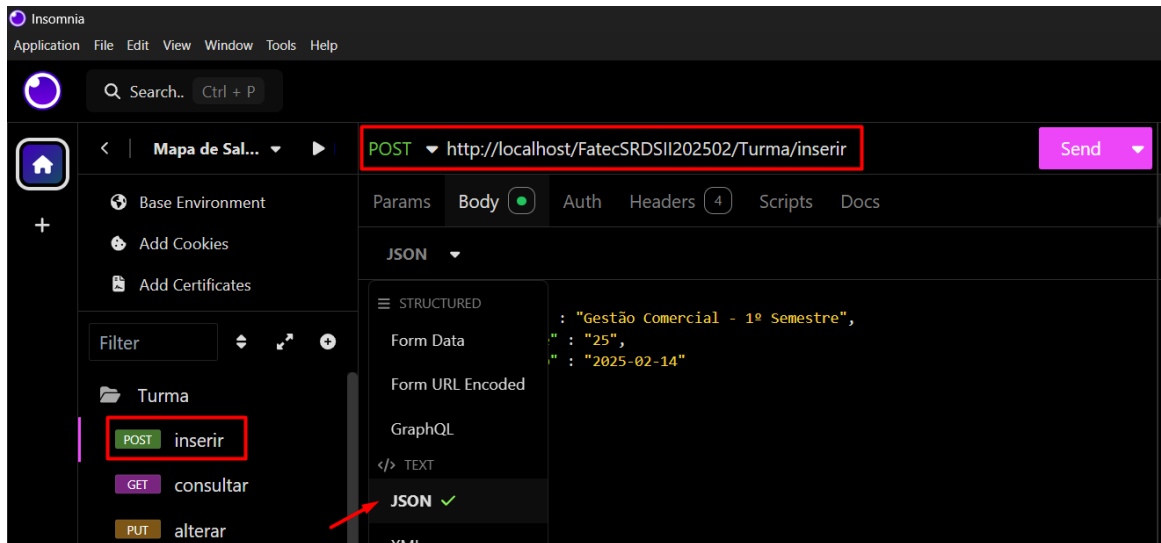
Ficando assim:



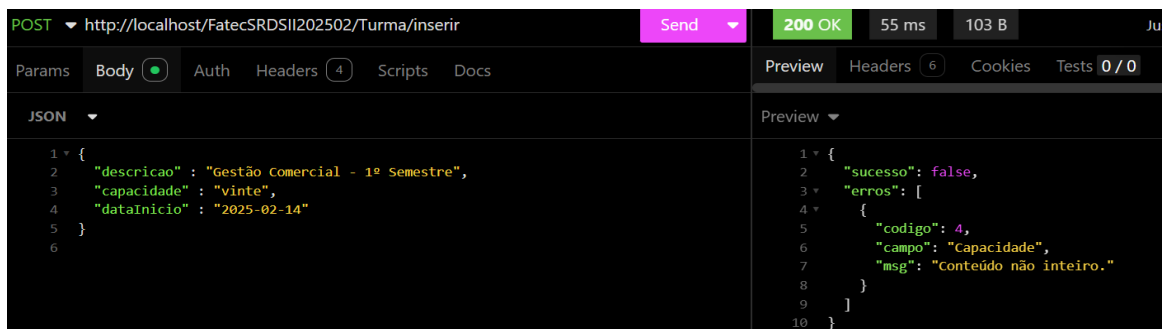
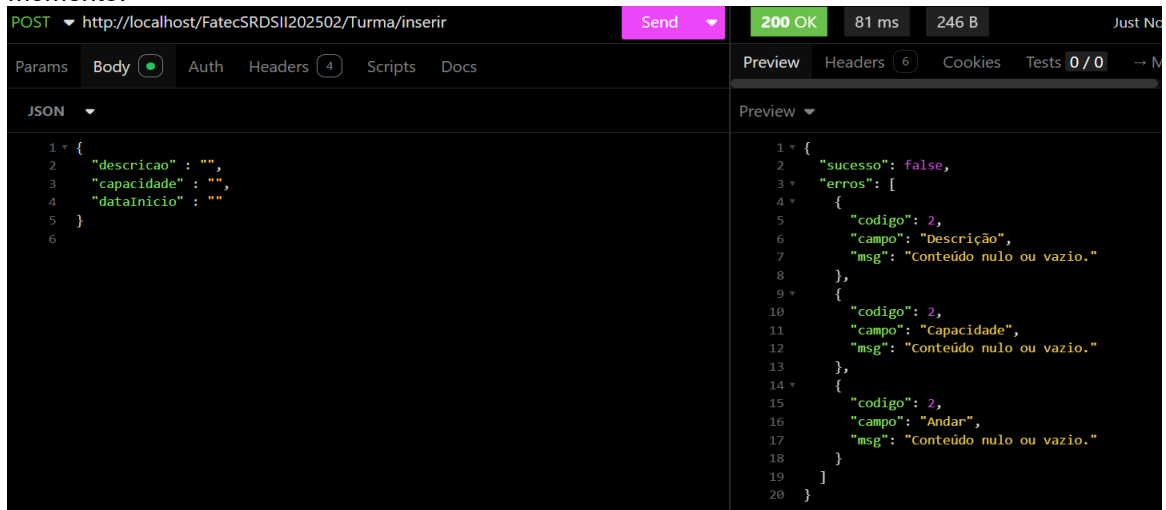
Feito isso, agora vamos efetivamente criar os EndPoints de nosso objeto, ou seja, incluir, consultar, alterar e desativar a referida sala.

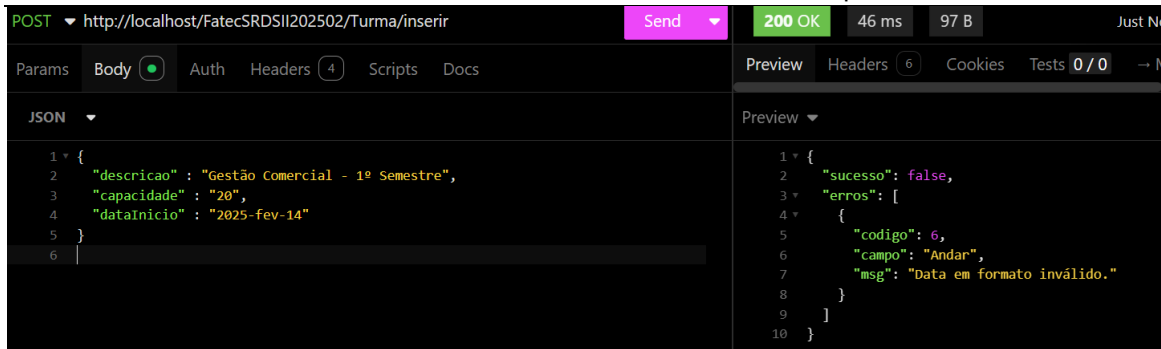


Método Inserir:

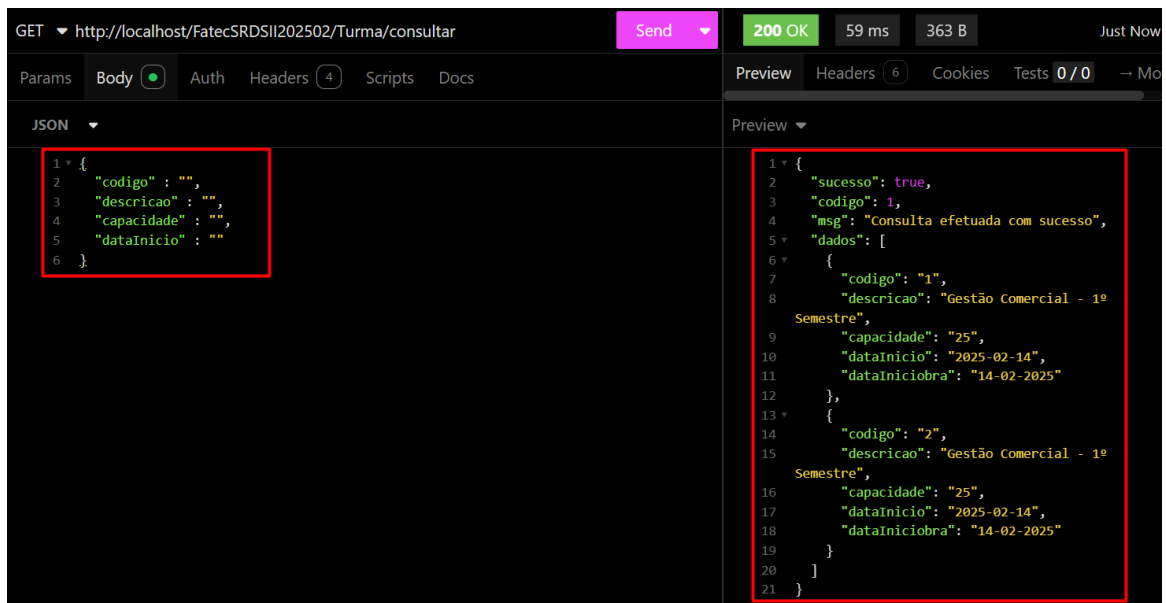
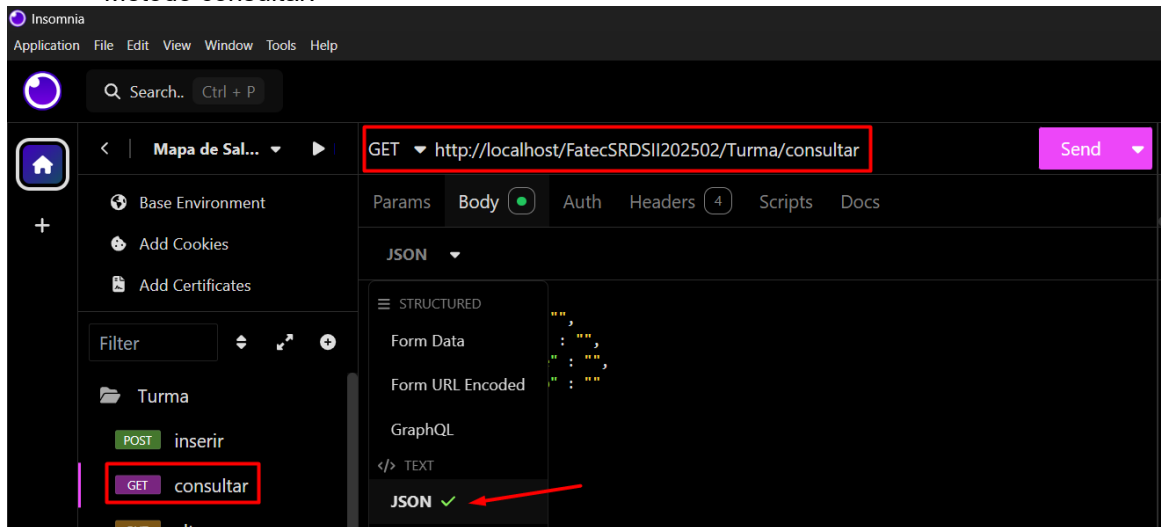


Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:





Método consultar:



Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:

```
GET http://localhost/FatecSRDSII202502/Turma/consultar 200 OK 61 ms 78 B
Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0
JSON
1 {
2   "codigo": "333",
3   "descricao": "",
4   "capacidade": "",
5   "dataInicio": ""
6 }
```

```
Preview
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 11,
6       "msg": "Turma não encontrado."
7     }
8   ]
9 }
```

```
GET http://localhost/FatecSRDSII202502/Turma/consultar 200 OK 56 ms 171 B
Params Body Auth Headers (4) Scripts Docs Preview Headers (6) Cookies Tests 0/0
JSON
1 {
2   "codigo": "",
3   "descricao": "",
4   "capacidade": "vinte",
5   "dataInicio": "fdf"
6 }
7
```

```
Preview
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Capacidade",
7       "msg": "Conteúdo não inteiro."
8     },
9     {
10      "codigo": 6,
11      "campo": "Andar",
12      "msg": "Data em formato inválido."
13    }
14  ]
15 }
```

6.5.4.4 Controller Turma (métodos **alterar()** e **desativar()**)

Vamos agora completar nosso CRUD, vamos codificar os dois métodos faltantes conforme a seguir:

```
251 public function alterar() {
252     //Atributos para controlar o status de nosso método
253     $erros = [];
254     $sucesso = false;
255
256     try {
257
258         $json = file_get_contents('php://input');
259         $resultado = json_decode($json);
260         $lista = [
261             "codigo" => '0',
262             "descricao" => '0',
263             "capacidade" => '0',
264             "dataInicio" => '0'
265         ];
266
267         if (verificarParam($resultado, $lista) != 1) {
268             // Validar vindos de forma correta do frontend (Helper)
269             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
270         }else{
271             //Pelo menos um dos três parâmetros precisam ter dados para acontecer a atualização
272             if(trim($resultado->descricao) == '' && trim($resultado->capacidade) == '' &&
273                 trim($resultado->dataInicio)==''){
274                 $erros[] = ['codigo' => 12,
275                     'msg' => 'Pelo menos um parâmetro precisa ser passado para atualização'];
276             }else{
277                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
278                 $retornoCodigo = validarDados($resultado->codigo,'int', true);
279                 $retornoDescricao = validarDadosConsulta($resultado->descricao,'string');
280                 $retornoCapacidade = validarDadosConsulta($resultado->capacidade,'int');
281                 $retornoDataInicio = validarDadosConsulta($resultado->dataInicio,'date');
282
283                 if($retornoCodigo['codigoHelper'] != 0){
```

```

284         $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
285                    'campo' => 'Codigo',
286                    'msg' => $retornoCodigo['msg']];
287     }
288
289     if($retornoDescricao['codigoHelper'] != 0){
290         $erros[] = ['codigo' => $retornoDescricao['codigoHelper'],
291                    'campo' => 'Descrição',
292                    'msg' => $retornoDescricao['msg']];
293     }
294
295     if($retornoCapacidade['codigoHelper'] != 0){
296         $erros[] = ['codigo' => $retornoCapacidade['codigoHelper'],
297                    'campo' => 'Andar',
298                    'msg' => $retornoCapacidade['msg']];
299     }
300
301     if($retornoDataInicio['codigoHelper'] != 0){
302         $erros[] = ['codigo' => $retornoDataInicio['codigoHelper'],
303                    'campo' => 'Data Inicio',
304                    'msg' => $retornoDataInicio['msg']];
305     }
306     //Se não encontrar erros
307     if (empty($erros)) {
308         $this->setCodigo($resultado->codigo);
309         $this->setDescricao($resultado->descricao);
310         $this->setCapacidade($resultado->capacidade);
311         $this->setDataInicio($resultado->dataInicio);
312
313         $this->load->model('M_turma');
314         $resBanco = $this->M_turma->alterar($this->getCodigo(),
315                                           $this->getDescricao(),
316                                           $this->getCapacidade(),
317                                           $this->getDataInicio());
318
319         if ($resBanco['codigo']== 1) {
320             $sucesso = true;
321         } else {
322             // Captura erro do banco
323             $erros[] = [
324                 'codigo' => $resBanco['codigo'],
325                 'msg' => $resBanco['msg']
326             ];
327         }
328     }
329 }
330
331 } catch (Exception $e) {
332     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
333 }
334
335 // Monta retorno único
336 if ($sucesso == true) {
337     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
338               'msg' => $resBanco['msg']];
339 } else {
340     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
341 }
342
343 // Transforma o array em JSON
344 echo json_encode($retorno);
345 }
346
347 public function desativar() {
348     //Atributos para controlar o status de nosso método
349     $erros = [];
350     $sucesso = false;
351
352     try {
353
354         $json = file_get_contents('php://input');
355         $resultado = json_decode($json);
356         $lista = [
357             "codigo" => '0'
358         ];

```

```

359
360     if (verificarParam($resultado, $lista) != 1) {
361         // Validar vindos de forma correta do frontend (Helper)
362         $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
363     }else{
364         // Validar código quanto ao tipo de dado e tamanho (Helper)
365         $retornoCodigo = validarDados($resultado->codigo,'int', true);
366
367         if($retornoCodigo['codigoHelper'] != 0){
368             $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
369                 'campo' => 'Codigo',
370                 'msg' => $retornoCodigo['msg']];
371         }
372
373         //Se não encontrar erros
374         if (empty($erros)) {
375             $this->setCodigo($resultado->codigo);
376
377             $this->load->model('M_turma');
378             $resBanco = $this->M_turma->desativar($this->getCodigo());
379
380             if ($resBanco['codigo']== 1) {
381                 $sucesso = true;
382             } else {
383                 // Captura erro do banco
384                 $erros[] = [
385                     'codigo' => $resBanco['codigo'],
386                     'msg' => $resBanco['msg']
387                 ];
388             }
389         }
390     }
391 }
392 } catch (Exception $e) {
393     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
394 }
395
396 // Monta retorno único
397 if ($sucesso == true) {
398     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
399                 'msg' => $resBanco['msg']];
400 } else {
401     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
402 }
403
404 // Transforma o array em JSON
405 echo json_encode($retorno);
406 }
407 }
408 ?>

```

6.5.4.5 Model/ M_turma (métodos **alterar()** e **desativar()**)

Vamos agora implementar a modelo, porém, temos um método privado de verificação da turma por código, iremos implementar mais este método conforme a seguir:

```

103     public function alterar($codigo, $descricao, $capacidade, $dataInicio)
104     {
105         try {
106             // Verifica se a turma já está cadastrada
107             $retornoConsulta = $this->consultaTurmaCod($codigo);
108
109             if ($retornoConsulta['codigo'] == 10) {
110                 // Monta a query dinâmica
111                 $query = "UPDATE tbl_turma SET ";
112                 $updates = [];
113
114                 if ($descricao != '') {
115                     $updates[] = "descricao = '$descricao'";
116                 }
117                 if ($capacidade != '') {
118                     $updates[] = "capacidade = $capacidade";
119                 }
120                 if ($dataInicio != '') {
121                     $updates[] = "dataInicio = '$dataInicio'";
122                 }

```

```

123
124         $query .= implode(" ", $updates) . " WHERE codigo = $codigo ";
125
126         // Prepara os valores para binding
127         $params = [];
128         if ($descricao !== '') {
129             $params[] = $descricao;
130         }
131         if ($capacidade !== '') {
132             $params[] = $capacidade;
133         }
134         if ($dataInicio !== '') {
135             $params[] = $dataInicio;
136         }
137         $params[] = $codigo;
138
139         // Executa a query
140         $this->db->query($query, $params);
141
142         // Verifica se a atualização foi bem-sucedida
143         if ($this->db->affected_rows() > 0) {
144             $dados = array(
145                 'codigo' => 1,
146                 'msg' => 'Turma atualizada corretamente.'
147             );
148         } else {
149             $dados = array(
150                 'codigo' => 8,
151                 'msg' => 'Houve algum problema na atualização na tabela de turma.'
152             );
153         }
154     } else {
155         $dados = array(
156             'codigo' => 5,
157             'msg' => 'Turma não cadastrada no sistema.'
158         );
159     }
160 } catch (Exception $e) {
161     $dados = array(
162         'codigo' => 00,
163         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
164     );
165 }
166
167 return $dados;
168 }
169
170 private function consultaTurmaCod($codigo)
171 {
172     try {
173         //Query para consultar dados de acordo com parâmetros passados
174         $sql = "select * from tbl_turma where codigo = $codigo ";
175
176         $retornoTurma = $this->db->query($sql);
177
178         //Verificar se a consulta ocorreu com sucesso
179         if ($retornoTurma->num_rows() > 0) {
180             $linha = $retornoTurma->row();
181             if (trim($linha->estatus) == "D") {
182                 $dados = array(
183                     'codigo' => 9,
184                     'msg' => 'Turma desativada no sistema.'
185                 );
186             } else {
187                 $dados = array(
188                     'codigo' => 10,
189                     'msg' => 'Consulta efetuada com sucesso.'
190                 );
191             }
192         } else {
193             $dados = array('codigo' => 12,
194                 'msg' => 'Turma não encontrada.');
```

```

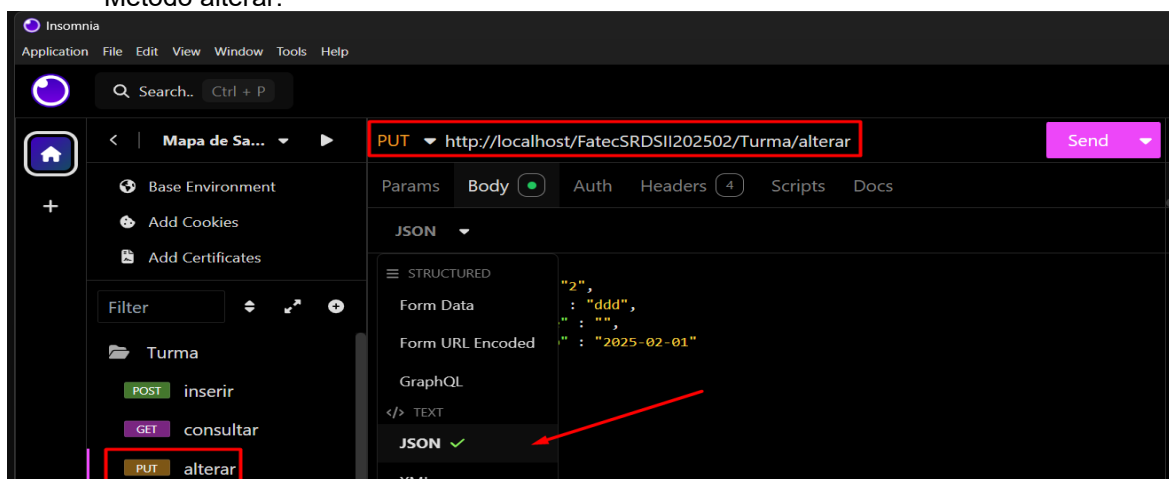
197     } catch (Exception $e) {
198         $dados = array(
199             'codigo' => 00,
200             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
201         );
202     }
203     //Envia o array $dados com as informações tratadas
204     //acima pela estrutura de decisão if
205     return $dados;
206 }
207
208 public function desativar($codigo)
209 {
210     try {
211         // Verifica se a turma já está cadastrada
212         $retornoConsulta = $this->consultaTurmaCod($codigo);
213
214         if ($retornoConsulta['codigo'] == 10) {
215             //Query de atualização dos dados
216             $this->db->query("update tbl_turma set status = 'D'
217                             where codigo = $codigo");
218
219             //Verificar se a atualização ocorreu com sucesso
220             if ($this->db->affected_rows() > 0) {
221                 $dados = array(
222                     'codigo' => 1,
223                     'msg' => 'Turma DESATIVADA corretamente.'
224                 );
225             } else {
226                 $dados = array(
227                     'codigo' => 8,
228                     'msg' => 'Houve algum problema na DESATIVAÇÃO da turma.'
229                 );
230             }
231         } else {
232             $dados = array('codigo' => $retornoConsulta['codigo'],
233                             'msg' => $retornoConsulta['msg']);
234         }
235     } catch (Exception $e) {
236         $dados = array(
237             'codigo' => 00,
238             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
239         );
240     }
241     //Envia o array $dados com as informações tratadas
242     //acima pela estrutura de decisão if
243     return $dados;
244 }
245 }
246 }
247
248

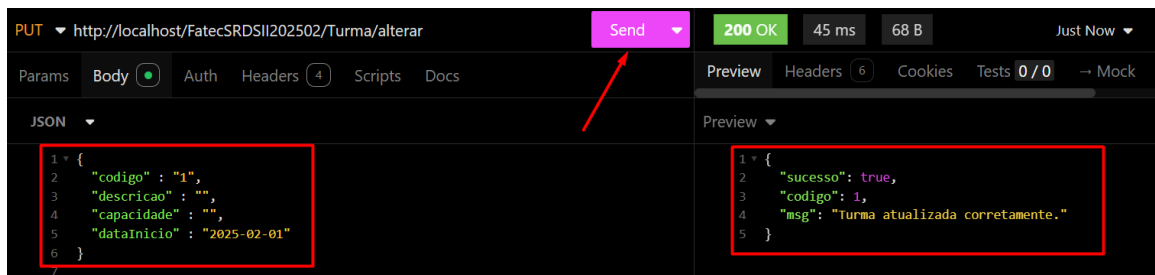
```

6.5.4.6 Objeto no Insomnia

Vamos agora terminar os métodos no Insomnia:

Método alterar:





PUT http://localhost/FatecSRDSII202502/Turma/alterar

Send 200 OK 45 ms 68 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0 → Mock

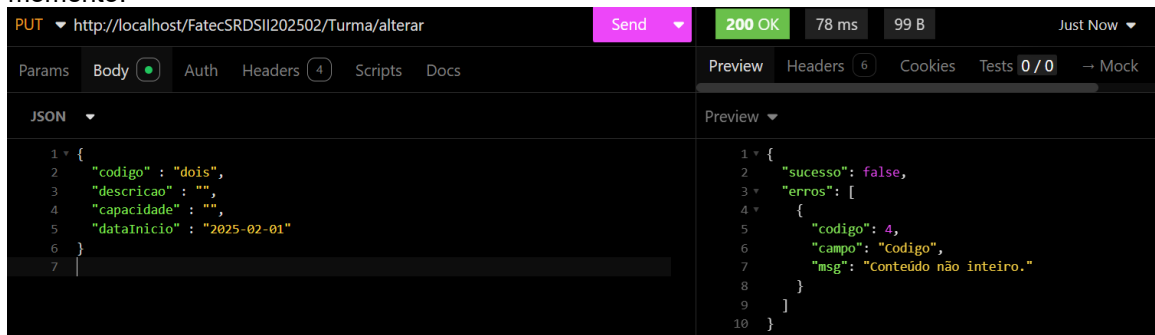
JSON

```
1 {
2   "codigo": "1",
3   "descricao": "",
4   "capacidade": "",
5   "dataInicio": "2025-02-01"
6 }
```

Preview

```
1 {
2   "sucesso": true,
3   "codigo": 1,
4   "msg": "Turma atualizada corretamente."
5 }
```

Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:



PUT http://localhost/FatecSRDSII202502/Turma/alterar

Send 200 OK 78 ms 99 B Just Now

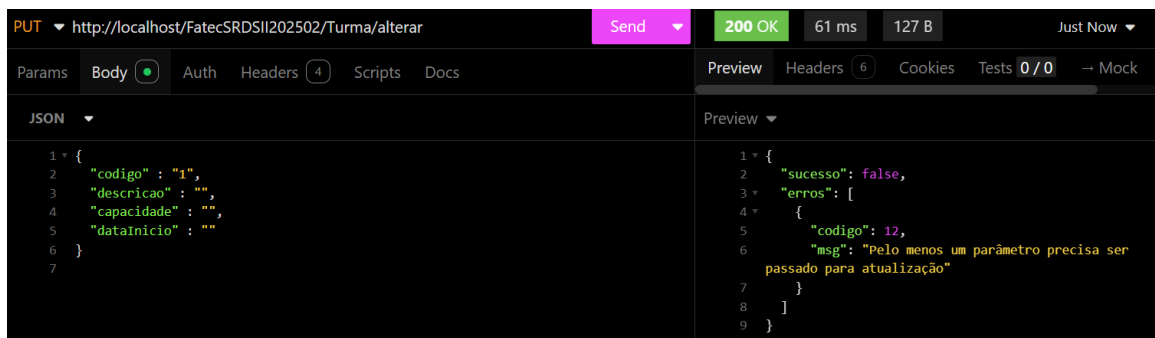
Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0 → Mock

JSON

```
1 {
2   "codigo": "dois",
3   "descricao": "",
4   "capacidade": "",
5   "dataInicio": "2025-02-01"
6 }
7
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Codigo",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```



PUT http://localhost/FatecSRDSII202502/Turma/alterar

Send 200 OK 61 ms 127 B Just Now

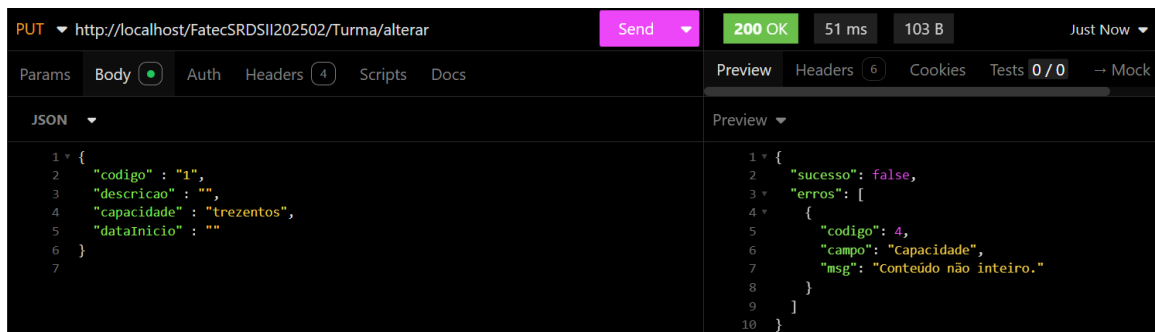
Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0 → Mock

JSON

```
1 {
2   "codigo": "1",
3   "descricao": "",
4   "capacidade": "",
5   "dataInicio": ""
6 }
7
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 12,
6       "msg": "Pelo menos um parâmetro precisa ser
passado para atualização"
7     }
8   ]
9 }
```



PUT http://localhost/FatecSRDSII202502/Turma/alterar

Send 200 OK 51 ms 103 B Just Now

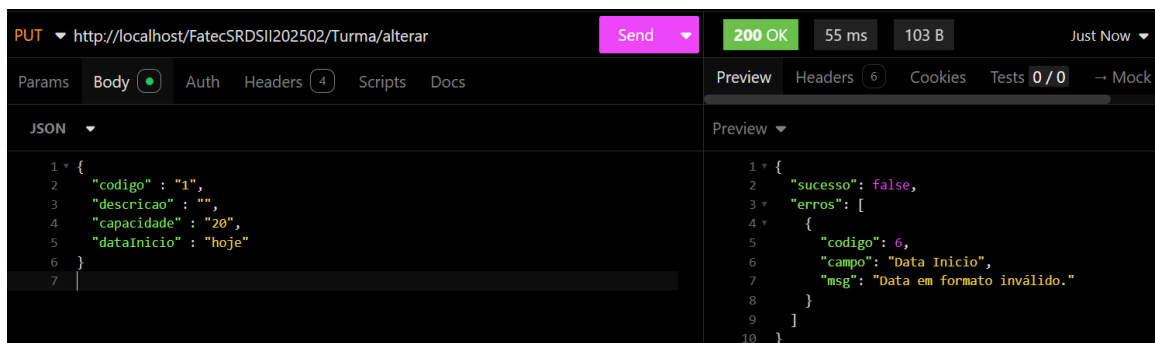
Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0 → Mock

JSON

```
1 {
2   "codigo": "1",
3   "descricao": "",
4   "capacidade": "trezentos",
5   "dataInicio": ""
6 }
7
```

Preview

```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 4,
6       "campo": "Capacidade",
7       "msg": "Conteúdo não inteiro."
8     }
9   ]
10 }
```



PUT http://localhost/FatecSRDSII202502/Turma/alterar

Send 200 OK 55 ms 103 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 6 Cookies Tests 0/0 → Mock

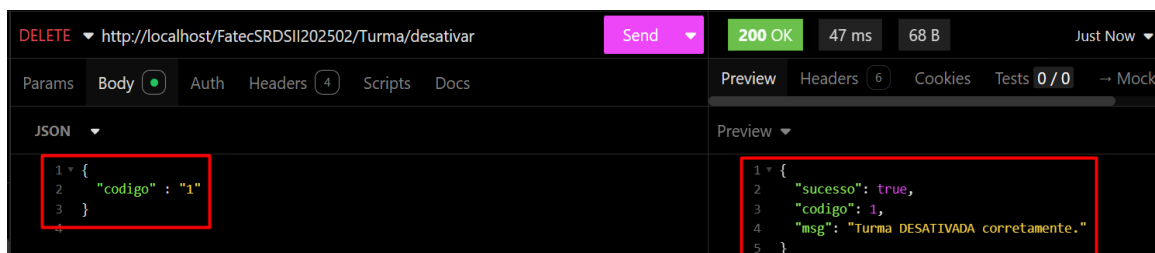
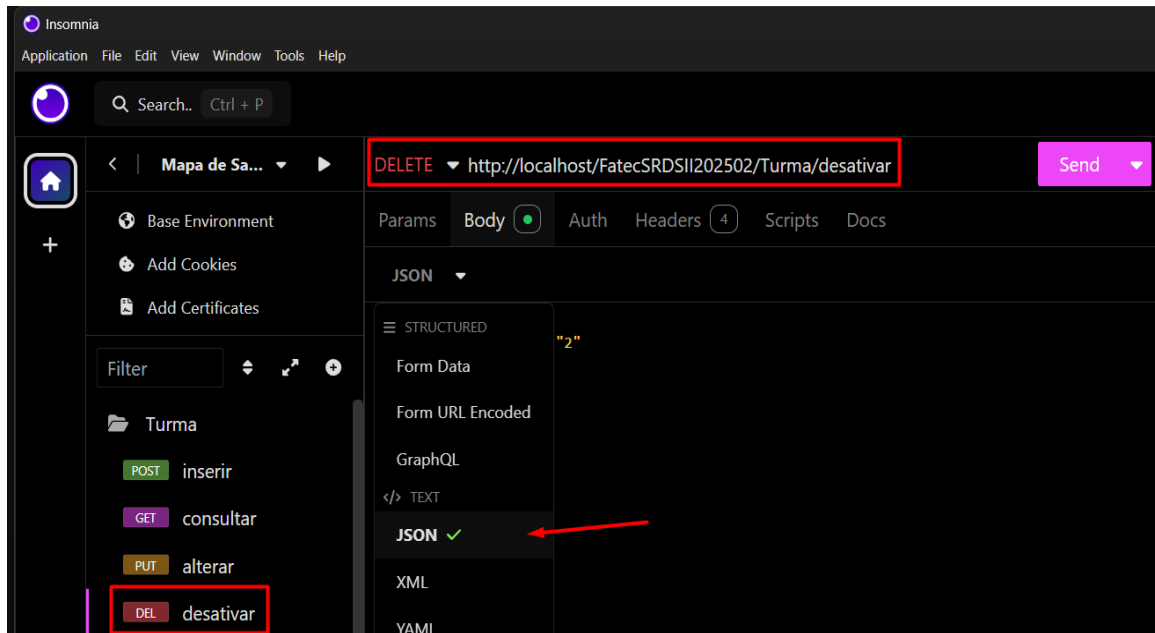
JSON

```
1 {
2   "codigo": "1",
3   "descricao": "",
4   "capacidade": "20",
5   "dataInicio": "hoje"
6 }
7
```

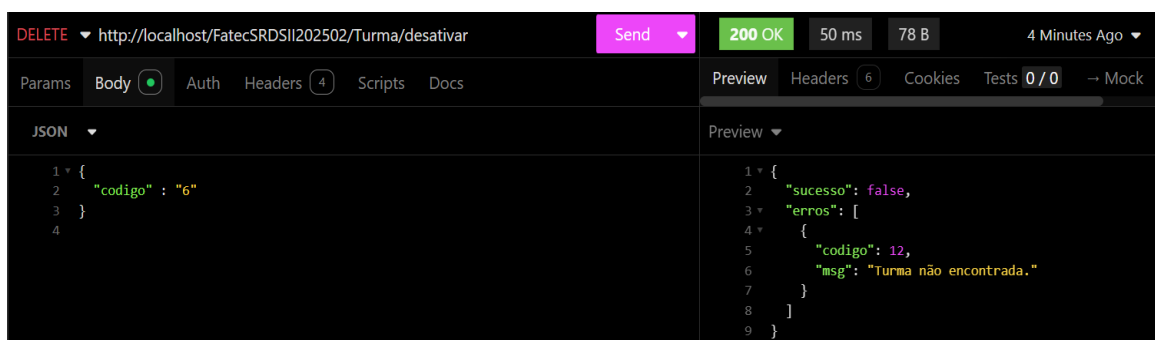
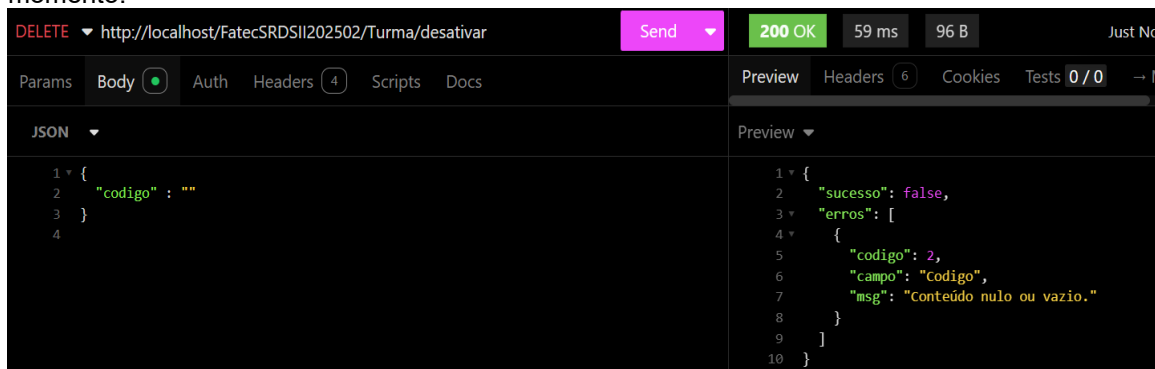
Preview

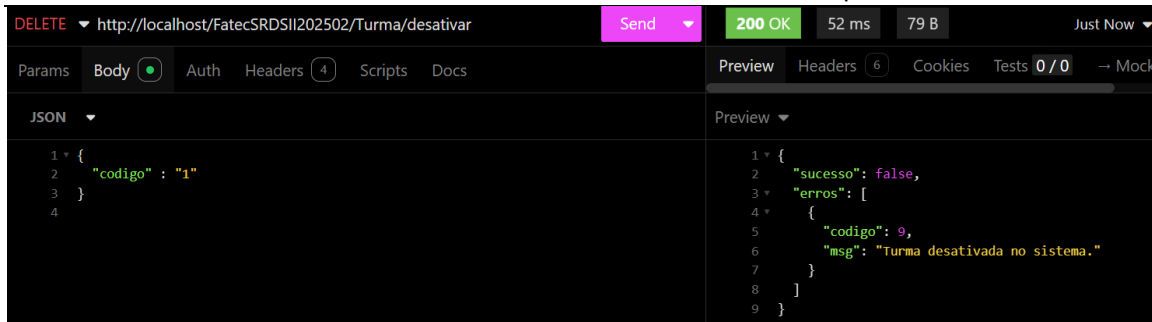
```
1 {
2   "sucesso": false,
3   "erros": [
4     {
5       "codigo": 6,
6       "campo": "Data Inicio",
7       "msg": "Data em formato inválido."
8     }
9   ]
10 }
```

Método desativar:

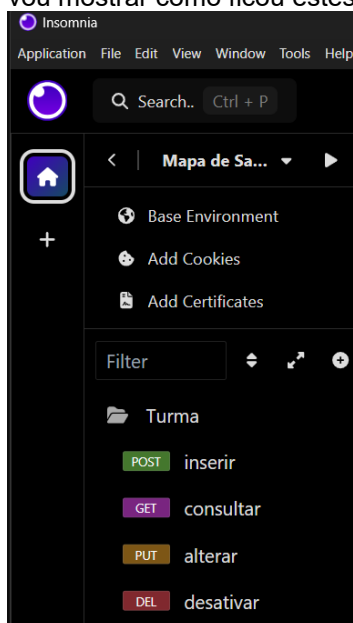


Fizemos o caminho “feliz”, mas claro que nossas críticas precisam aparecer neste momento:





Com isso, finalizamos a implementação de nosso terceiro objeto do sistema, os *endpoints* no *Insomnia* são importantes para o salvamento, recomendo que ao final de cada aula, façam a exportação do mesmo e guardem para que possam atualizar e utilizar durante as aulas, abaixo vou mostrar como ficou estes *endpoints* no final.



6.5.5 Objeto Professor

Neste objeto vamos tratar também o CRUD (*Create Read Update and Delete*) dos dados pertencentes ao professor, e acrescentar uma função na helper.

6.5.5.1 Implementação de mais um método na *Helper*

Novamente, vamos implementar mais uma função em nossa **Helper**, esta função será um validador de CPF, portanto vamos acrescentar em nossa *geral_helper.php* a função conforme a seguir:

```

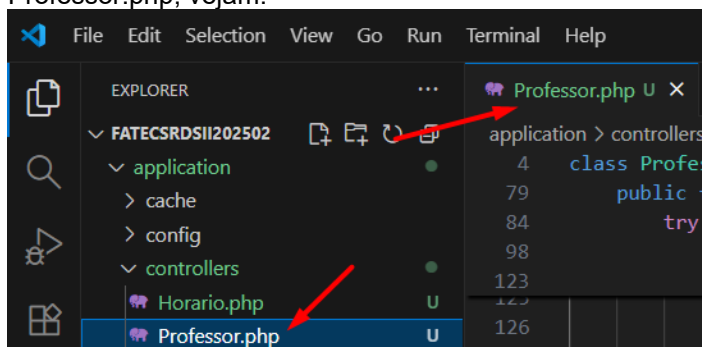
151  /*
152  Função para verificar se o CPF é válido quanto a sua estrutura
153  */
154  function validarCPF($cpf) {
155      // Remove tudo que não for número
156      $cpf = preg_replace('/[^0-9]/', '', $cpf);
157
158      // CPF deve ter 11 dígitos
159      if (strlen($cpf) != 11) {
160          return array('codigoHelper' => 15, 'msg' => 'CPF com menos de 11 dígitos.');
```

```

168 // Calcula os dígitos verificadores
169 for ($t = 9; $t < 11; $t++) {
170     $soma = 0;
171     for ($i = 0; $i < $t; $i++) {
172         $soma += $cpf[$i] * (($t + 1) - $i);
173     }
174     $digito = (10 * $soma) % 11;
175     $digito = ($digito == 10) ? 0 : $digito;
176
177     if ($cpf[$t] != $digito) {
178         return array('codigoHelper' => 17, 'msg' => 'CPF com dígitos verificadores incorretos.');
```

6.5.5.2 Controller Professor

Vamos inicialmente criar dentro da pasta *controller* de nosso projeto, o arquivo *Professor.php*, vejamos:



Assim, agora iremos codificar este arquivo conforme a seguir:

```

Professor.php U X
application > controllers > Professor.php
1 <?php
2 defined('BASEPATH') OR exit('No direct script access allowed');
3
4 class Professor extends CI_Controller {
5     /*
6     Validação dos tipos de retornos nas validações (Código de erro)
7     1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
8     2 - Conteúdo passado nulo ou vazio
9     3 - Conteúdo zerado
10    4 - Conteúdo não inteiro
11    5 - Conteúdo não é um texto
12    6 - Data em formato inválido
13    12 - Na atualização, pelo menos um atributo deve ser passado
14    15 - CPF com menos de 11 dígitos
15    16 - CPF com todos dígitos iguais
16    17 - CPF com dígitos verificadores incorretos
17    99 - Parâmetros passados do front não correspondem ao método
18    */
19
20    //Atributos privados da classe
21    private $codigo;
22    private $nome;
23    private $cpf;
24    private $tipo;
25    private $status;
26
27    //Getters dos atributos
28    public function getCodigo()
29    {
30        return $this->codigo;
31    }
32
33    public function getNome()
34    {
35        return $this->nome;
36    }
37

```

```

38 public function getCpf()
39 {
40     return $this->cpf;
41 }
42
43 public function getTipo()
44 {
45     return $this->tipo;
46 }
47
48 public function getEstatus()
49 {
50     return $this->estatus;
51 }
52
53 //Setters dos atributos
54 public function setCodigo($codigoFront)
55 {
56     $this->codigo = $codigoFront;
57 }
58
59 public function setNome($nomeFront)
60 {
61     $this->nome = $nomeFront;
62 }
63
64 public function setCpf($cpfFront)
65 {
66     $this->cpf = $cpfFront;
67 }
68
69 public function setTipo($tipoFront)
70 {
71     $this->tipo = $tipoFront;
72 }
73
74 public function setEstatus($estatusFront)
75 {
76     $this->estatus = $estatusFront;
77 }
78
79 public function inserir() {
80     //Atributos para controlar o status de nosso método
81     $erros = [];
82     $sucesso = false;
83
84     try {
85
86         $json = file_get_contents('php://input');
87         $resultado = json_decode($json);
88         $lista = [
89             "nome" => '0',
90             "cpf" => '0',
91             "tipo" => '0'
92         ];
93
94         if (verificarParam($resultado, $lista) != 1) {
95             // Validar vindos de forma correta do frontend (Helper)
96             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no
97                 FrontEnd.'];
98         }else{
99             // Validar campos quanto ao tipo de dado e tamanho (Helper)
100             $retornoNome = validarDados($resultado->nome,'string',true);
101             $retornoCPF = validarDados($resultado->cpf,'string',true);
102             $retornoCPFInvalido = validarCPF($resultado->cpf);
103             $retornoTipo = validarDados($resultado->tipo,'string',true);
104
105             if($retornoNome['codigoHelper'] != 0){
106                 $erros[] = ['codigo' => $retornoNome['codigoHelper'],
107                     'campo' => 'Nome',
108                     'msg' => $retornoNome['msg']];
109             }
110

```

```

111         if($retornoCPF['codigoHelper'] != 0){
112             $erros[] = ['codigo' => $retornoCPF['codigoHelper'],
113                       'campo' => 'CPF',
114                       'msg' => $retornoCPF['msg']];
115         }
116
117         if($retornoCPFNroValido['codigoHelper'] != 0){
118             $erros[] = ['codigo' => $retornoCPFNroValido['codigoHelper'],
119                       'campo' => 'CPF validação número',
120                       'msg' => $retornoCPFNroValido['msg']];
121         }
122
123         if($retornoTipo['codigoHelper'] != 0){
124             $erros[] = ['codigo' => $retornoTipo['codigoHelper'],
125                       'campo' => 'Tipo',
126                       'msg' => $retornoTipo['msg']];
127         }
128
129         //Se não encontrar erros
130         if (empty($erros)) {
131             $this->setNome($resultado->nome);
132             $this->setCpf($resultado->cpf);
133             $this->setTipo($resultado->tipo);
134
135             $this->load->model('M_professor');
136             $resBanco = $this->M_professor->inserir(
137                 $this->getNome(),
138                 $this->getCpf(),
139                 $this->getTipo()
140             );
141
142             if ($resBanco['codigo']== 1) {
143                 $sucesso = true;
144             } else {
145                 // Captura erro do banco
146                 $erros[] = [
147                     'codigo' => $resBanco['codigo'],
148                     'msg' => $resBanco['msg']
149                 ];
150             }
151         }
152     }
153 } catch (Exception $e) {
154     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
155 }
156
157 // Monta retorno único
158 if ($sucesso == true) {
159     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
160               'msg' => $resBanco['msg']];
161 } else {
162     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
163 }
164
165 // Transforma o array em JSON
166 echo json_encode($retorno);
167 }
168
169 public function consultar() {
170     //Atributos para controlar o status de nosso método
171     $erros = [];
172     $sucesso = false;
173
174     try {
175
176         $json = file_get_contents('php://input');
177         $resultado = json_decode($json);
178         $lista = [
179             "codigo" => '0',
180             "nome" => '0',
181             "cpf" => '0',
182             "tipo" => '0'
183         ];
184
185         if (verificarParam($resultado, $lista) != 1) {
186             // Validar vindos de forma correta do frontend (Helper)
187             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no

```

```

188 |                                     FrontEnd.'];
189 |     }else{
190 |         // Validar campos quanto ao tipo de dado e tamanho (Helper)
191 |         $retornoCodigo = validarDadosConsulta($resultado->codigo,'int');
192 |         $retornoNome = validarDadosConsulta($resultado->nome,'string');
193 |         $retornoCPF = validarDadosConsulta($resultado->cpf,'string');
194 |         $retornoTipo = validarDadosConsulta($resultado->tipo,'string');
195 |
196 |         if($retornoCodigo['codigoHelper'] != 0){
197 |             $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
198 |                         'campo' => 'Codigo',
199 |                         'msg' => $retornoCodigo['msg']];
200 |         }
201 |
202 |         if($retornoNome['codigoHelper'] != 0){
203 |             $erros[] = ['codigo' => $retornoNome['codigoHelper'],
204 |                         'campo' => 'Nome',
205 |                         'msg' => $retornoNome['msg']];
206 |         }
207 |
208 |         if($retornoCPF['codigoHelper'] != 0){
209 |             $erros[] = ['codigo' => $retornoCPF['codigoHelper'],
210 |                         'campo' => 'CPF',
211 |                         'msg' => $retornoCPF['msg']];
212 |         }
213 |
214 |         if($retornoTipo['codigoHelper'] != 0){
215 |             $erros[] = ['codigo' => $retornoTipo['codigoHelper'],
216 |                         'campo' => 'Tipo',
217 |                         'msg' => $retornoTipo['msg']];
218 |         }
219 |
220 |         if($resultado->cpf != ''){
221 |             //CPF informado, verificar se é número válido
222 |             $retornoCPFInvalido = validarCPF($resultado->cpf);
223 |             if($retornoCPFInvalido['codigoHelper'] != 0){
224 |                 $erros[] = ['codigo' => $retornoCPFInvalido['codigoHelper'],
225 |                             'campo' => 'CPF validação número',
226 |                             'msg' => $retornoCPFInvalido['msg']];
227 |             }
228 |         }
229 |         //Se não encontrar erros
230 |         if (empty($erros)) {
231 |             $this->setCodigo($resultado->codigo);
232 |             $this->setNome($resultado->nome);
233 |             $this->setCpf($resultado->cpf);
234 |             $this->setTipo($resultado->tipo);
235 |
236 |             $this->load->model('M_professor');
237 |             $resBanco = $this->M_professor->consultar($this->getCodigo(),
238 |                                                         $this->getNome(),
239 |                                                         $this->getCpf(),
240 |                                                         $this->getTipo());
241 |
242 |             if ($resBanco['codigo']== 1) {
243 |                 $sucesso = true;
244 |             } else {
245 |                 // Captura erro do banco
246 |                 $erros[] = [
247 |                     'codigo' => $resBanco['codigo'],
248 |                     'msg' => $resBanco['msg']
249 |                 ];
250 |             }
251 |         }
252 |     }
253 | } catch (Exception $e) {
254 |     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
255 | }
256 |
257 | // Monta retorno único
258 | if ($sucesso == true) {
259 |     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
260 |                 'msg' => $resBanco['msg'],
261 |                 'dados' => $resBanco['dados']];
262 | } else {
263 |     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
264 | }
265 |

```

```

266 // Transforma o array em JSON
267 echo json_encode($retorno);
268 }
269
270 public function alterar() {
271     //Atributos para controlar o status de nosso método
272     $erros = [];
273     $sucesso = false;
274
275     try {
276
277         $json = file_get_contents('php://input');
278         $resultado = json_decode($json);
279         $lista = [
280             "codigo" => '0',
281             "nome" => '0',
282             "cpf" => '0',
283             "tipo" => '0'
284         ];
285
286         if (verificarParam($resultado, $lista) != 1) {
287             // Validar vindos de forma correta do frontend (Helper)
288             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
289         }else{
290             //Pelo menos um dos três parâmetros precisam ter dados para acontecer a atualização
291             if(trim($resultado->nome) == '' && trim($resultado->cpf) == '' &&
292                 trim($resultado->tipo) == ''){
293                 $erros[] = ['codigo' => 12,
294                     'msg' => 'Pelo menos um parâmetro precisa ser passado para atualização'];
295             }else{
296                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
297                 $retornoCodigo = validarDados($resultado->codigo,'int', true);
298                 $retornoNome = validarDadosConsulta($resultado->nome,'string');
299                 $retornoCPF = validarDadosConsulta($resultado->cpf,'string');
300                 $retornoTipo = validarDadosConsulta($resultado->tipo,'string');
301
302                 if($retornoCodigo['codigoHelper'] != 0){
303                     $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
304                         'campo' => 'Codigo',
305                         'msg' => $retornoCodigo['msg']];
306                 }
307
308                 if($retornoNome['codigoHelper'] != 0){
309                     $erros[] = ['codigo' => $retornoNome['codigoHelper'],
310                         'campo' => 'Nome',
311                         'msg' => $retornoNome['msg']];
312                 }
313
314                 if($retornoCPF['codigoHelper'] != 0){
315                     $erros[] = ['codigo' => $retornoCPF['codigoHelper'],
316                         'campo' => 'CPF',
317                         'msg' => $retornoCPF['msg']];
318                 }
319
320                 if($retornoTipo['codigoHelper'] != 0){
321                     $erros[] = ['codigo' => $retornoTipo['codigoHelper'],
322                         'campo' => 'Tipo',
323                         'msg' => $retornoTipo['msg']];
324                 }
325
326                 if($resultado->cpf != ''){
327                     //CPF informado, verificar se é número válido
328                     $retornoCPFInvalido = validarCPF($resultado->cpf);
329                     if($retornoCPFInvalido['codigoHelper'] != 0){
330                         $erros[] = ['codigo' => $retornoCPFInvalido['codigoHelper'],
331                             'campo' => 'CPF validação número',
332                             'msg' => $retornoCPFInvalido['msg']];
333                     }
334                 }
335             }
336         }
337     }
338 }

```

```

335 //Se não encontrar erros
336 if (empty($erros)) {
337     $this->setCodigo($resultado->codigo);
338     $this->setNome($resultado->nome);
339     $this->setCpf($resultado->cpf);
340     $this->setTipo($resultado->tipo);
341
342     $this->load->model('M_professor');
343     $resBanco = $this->M_professor->alterar($this->getCodigo(),
344                                             $this->getNome(),
345                                             $this->getCpf(),
346                                             $this->getTipo());
347
348     if ($resBanco['codigo']== 1) {
349         $sucesso = true;
350     } else {
351         // Captura erro do banco
352         $erros[] = [
353             'codigo' => $resBanco['codigo'],
354             'msg' => $resBanco['msg']
355         ];
356     }
357 }
358 }
359 }
360 } catch (Exception $e) {
361     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
362 }
363
364 // Monta retorno único
365 if ($sucesso == true) {
366     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
367                 'msg' => $resBanco['msg']];
368 } else {
369     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
370 }
371
372 // Transforma o array em JSON
373 echo json_encode($retorno);
374 }
375
376 public function desativar() {
377     //Atributos para controlar o status de nosso método
378     $erros = [];
379     $sucesso = false;
380
381     try {
382
383         $json = file_get_contents('php://input');
384         $resultado = json_decode($json);
385         $lista = [
386             "codigo" => '0'
387         ];
388
389         if (verificarParam($resultado, $lista) != 1) {
390             // Validar vindos de forma correta do frontend (Helper)
391             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
392         } else {
393             // Validar código quanto ao tipo de dado e tamanho (Helper)
394             $retornoCodigo = validarDados($resultado->codigo, 'int', true);
395
396             if ($retornoCodigo['codigoHelper'] != 0) {
397                 $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
398                             'campo' => 'Codigo',
399                             'msg' => $retornoCodigo['msg']];
400             }
401
402             //Se não encontrar erros
403             if (empty($erros)) {
404                 $this->setCodigo($resultado->codigo);
405
406                 $this->load->model('M_professor');
407                 $resBanco = $this->M_professor->desativar($this->getCodigo());
408
409                 if ($resBanco['codigo']== 1) {
410                     $sucesso = true;
411                 } else {

```

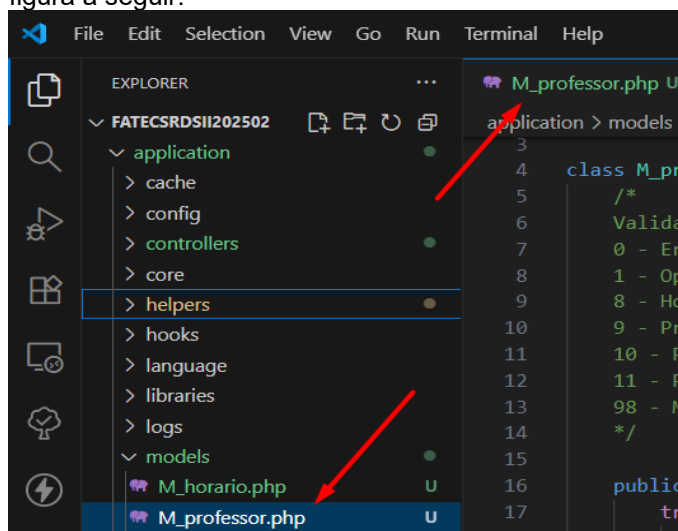
```

412 // Captura erro do banco
413 $erros[] = [
414     'codigo' => $resBanco['codigo'],
415     'msg' => $resBanco['msg']
416 ];
417 }
418 }
419
420 } catch (Exception $e) {
421     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
422 }
423
424 // Monta retorno único
425 if ($sucesso == true) {
426     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
427                 'msg' => $resBanco['msg']];
428 } else {
429     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
430 }
431
432 // Transforma o array em JSON
433 echo json_encode($retorno);
434 }
435 }
436 }
437 ?>

```

6.5.5.3 Model/ Professor

Agora vamos implementar a camada de modelo, ou seja, nossa model, assim sendo, vamos criar um arquivo na pasta models de nossa estrutura, o arquivo M_professor.php como a figura a seguir:



E vamos codificar da forma a seguir:

```

M_professor.php
application > models > M_professor.php
1 <?php
2 defined('BASEPATH') or exit('No direct script access allowed');
3
4 class M_professor extends CI_Model {
5     /*
6     Validação dos tipos de retornos nas validações (Código de erro)
7     0 - Erro de exceção
8     1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
9     8 - Houve algum problema de inserção, atualização, consulta ou exclusão
10    9 - Professor desativado no sistema
11    10 - Professor já cadastrado
12    11 - Professor não encontrado pelo método publico
13    98 - Método auxiliar de consulta que não trouxe dados
14    */
15
16    public function inserir($nome, $cpf, $tipo){
17        try{
18            //Verifico se o professor já está cadastrado
19            $retornoConsulta = $this->consultaProfessorCpf($cpf);
20
21            if($retornoConsulta['codigo'] != 9 &&

```



```

22     $retornoConsulta['codigo'] != 10){
23         //Query de inserção dos dados
24         $this->db->query("insert into tbl_professor (nome, tipo, cpf)
25             values ('$nome', '$tipo', '$cpf')");
26
27         //Verificar se a inserção ocorreu com sucesso
28         if($this->db->affected_rows() > 0){
29             $dados = array('codigo' => 1,
30                 'msg' => 'Professor cadastrado corretamente.');
```

```

31         }else{
32             $dados = array('codigo' => 8,
33                 'msg' => 'Houve algum problema na inserção na tabela de professor.');
```

```

34         }
35     }else{
36         $dados = array('codigo' => $retornoConsulta['codigo'],
37             'msg' => $retornoConsulta['msg']);
38     }
39 } catch (Exception $e) {
40     $dados = array(
41         'codigo' => 00,
42         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
43     );
44 }
45 }
46 //Envia o array $dados com as informações tratadas
47 //acima pela estrutura de decisão if
48 return $dados;
49 }
50
51 private function consultaProfessorCpf($cpf){
52     try{
53         //Query para consultar dados de acordo com parâmetros passados
54         $sql = "select * from tbl_professor where cpf = '$cpf'";
55
56         $retornoProfessor = $this->db->query($sql);
57
58         //Verificar se a consulta ocorreu com sucesso
59         if($retornoProfessor->num_rows() > 0){
60             $linha = $retornoProfessor->row();
61             if (trim($linha->estatus) == "D") {
62                 $dados = array(
63                     'codigo' => 9,
64                     'msg' => 'Professor desativado no sistema, caso precise reativar a mesma,
65                         fale com o administrador.'
66                 );
67             }else{
68                 $dados = array('codigo' => 10,
69                     'msg' => 'Professor já cadastrado no sistema.');
```

```

70             }
71         }else{
72             $dados = array('codigo' => 98,
73                 'msg' => 'Professor não encontrado.');
```

```

74         }
75     }catch (Exception $e) {
76         $dados = array(
77             'codigo' => 00,
78             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
79         );
80     }
81     //Envia o array $dados com as informações tratadas
82     //acima pela estrutura de decisão if
83     return $dados;
84 }
85
86 private function consultaProfessorCod($codigo){
87     try{
88         //Query para consultar dados de acordo com parâmetros passados
89         $sql = "select * from tbl_professor where codigo = '$codigo' and estatus = ''";
90
91         $retornoProfessor = $this->db->query($sql);

```

```

92
93     //Verificar se a consulta ocorreu com sucesso
94     if($retornoProfessor->num_rows() > 0){
95         $dados = array('codigo' => 1,
96             'msg' => 'Consulta efetuada com sucesso.');
```

```

97     }else{
98         $dados = array('codigo' => 98,
99             'msg' => 'Professor não encontrado.');
```

```

100     }
101 }catch (Exception $e) {
102     $dados = array(
103         'codigo' => 00,
104         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
105     );
106 }
107 //Envia o array $dados com as informações tratadas
108 //acima pela estrutura de decisão if
109 return $dados;
110 }
111
112
113 public function consultar($codigo, $nome, $cpf, $tipo){
114     try{
115         //Query para consultar dados de acordo com parâmetros passados
116         $sql = "select * from tbl_professor where status = ' ' ";
117
118         if(trim($codigo) != '') {
119             $sql = $sql . "and codigo = $codigo ";
120         }
121
122         if(trim($cpf) != ''){
123             $sql = $sql . "and cpf = '$cpf' ";
124         }
125
126         if(trim($nome) != ''){
127             $sql = $sql . "and nome like '%$nome%' ";
128         }
129
130         if(trim($tipo) != ''){
131             $sql = $sql . "and tipo = '$tipo' ";
132         }
133
134         $sql = $sql . " order by nome ";
135
136         $retorno = $this->db->query($sql);
137
138         //Verificar se a consulta ocorreu com sucesso
139         if($retorno->num_rows() > 0){
140             $dados = array('codigo' => 1,
141                 'msg' => 'Consulta efetuada com sucesso.',
142                 'dados' => $retorno->result());
143
144         }else{
145             $dados = array('codigo' => 11,
146                 'msg' => 'Professor não encontrado.');
```

```

147         }
148     }catch (Exception $e) {
149         $dados = array(
150             'codigo' => 00,
151             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
152         );
153     }
154     //Envia o array $dados com as informações tratadas
155     //acima pela estrutura de decisão if
156     return $dados;
157 }
158
159 public function alterar($codigo, $nome, $cpf, $tipo){
160     try{
161         //Verifico se o professor já está cadastrado
162         $retornoConsulta = $this->consultaProfessorCod($codigo);

```

```

163
164         if($retornoConsulta['codigo'] == 1){
165             //Inicio a query para atualização
166             $query = "update tbl_professor set ";
167
168             //Vamos comparar os itens
169             if($nome != ''){
170                 $query .= "nome = '$nome', ";
171             }
172
173             if($cpf != ''){
174                 $query .= "cpf = '$cpf', ";
175             }
176
177             if($tipo != ''){
178                 $query .= "tipo = '$tipo' ";
179             }
180             //Termino a concatenação da query
181             $queryFinal = rtrim($query, ", ") . " where codigo = $codigo";
182
183             //Executo a Query de atualização dos dados
184             $this->db->query($queryFinal);
185
186             //Verificar se a atualização ocorreu com sucesso
187             if($this->db->affected_rows() > 0){
188                 $dados = array('codigo' => 1,
189                     'msg' => 'Professor atualizado corretamente.');
```

```

190             }else{
191                 $dados = array('codigo' => 8,
192                     'msg' => 'Houve algum problema na atualização na tabela de professor.');
```

```

193             }
194         }else{
195             $dados = array('codigo' => $retornoConsulta['codigo'],
196                 'msg' => $retornoConsulta['msg']);
197         }
198     }
199
200 }catch (Exception $e) {
201     $dados = array(
202         'codigo' => 00,
203         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
204     );
205 }
206 //Envia o array $dados com as informações tratadas
207 //acima pela estrutura de decisão if
208 return $dados;
209 }
210
211 public function desativar($codigo){
212     try{
213         //Verifico se o professor já está cadastrado
214         $retornoConsulta = $this->consultaProfessorCod($codigo);
215
216         if($retornoConsulta['codigo'] == 1){
217
218             //Query de atualização dos dados
219             $this->db->query("update tbl_professor set status = 'D'
220                 where codigo = $codigo");
221
222             //Verificar se a atualização ocorreu com sucesso
223             if($this->db->affected_rows() > 0){
224                 $dados = array('codigo' => 1,
225                     'msg' => 'Professor DESATIVADO corretamente.');
```

```

226             }else{
227                 $dados = array('codigo' => 5,
228                     'msg' => 'Houve algum problema na DESATIVAÇÃO do Professor.');
```

```

229             }
230         }else{
231             $dados = array('codigo' => 6,
232                 'msg' => 'Professor não cadastrado no Sistema, não pode excluir.');
```

```

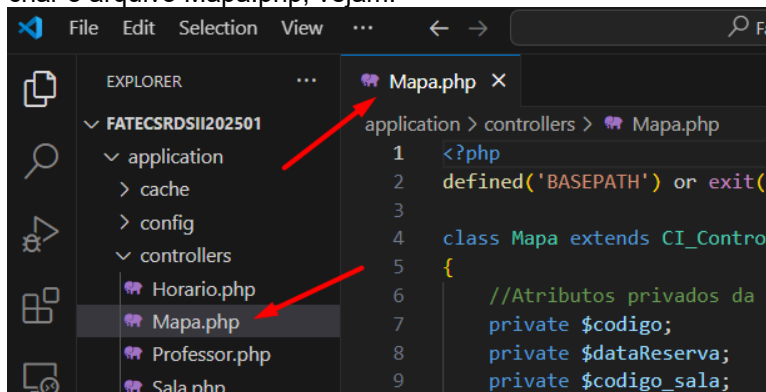
235     }
236   }catch (Exception $e) {
237     $dados = array(
238       'codigo' => 00,
239       'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
240     );
241   }
242   //Envia o array $dados com as informações tratadas
243   //acima pela estrutura de decisão if
244   return $dados;
245 }
246 }

```

Assim, agora temos 4 objetos criados em nosso sistema, ou seja, temos a base dos CRUDs para fazermos o mapeamento em si, que será nossa próxima etapa do projeto.

6.5.6 Objeto Mapa

Neste momento, temos todos os objetos base de nosso projeto, ou seja, Sala, Horário, Turma e Professor, agora nossa ideia é montar efetivamente o mapa de Sala, para isso vamos criar o arquivo Mapa.php, vejam:



Assim, agora iremos codificar este arquivo conforme a seguir:

```

Mapa.php
application > controllers > Mapa.php
1  <?php
2  defined('BASEPATH') OR exit('No direct script access allowed');
3
4  class Mapa extends CI_Controller {
5      /*
6       * Validação dos tipos de retornos nas validações (Código de erro)
7       * 1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
8       * 2 - Conteúdo passado nulo ou vazio
9       * 3 - Conteúdo zerado
10      * 4 - Conteúdo não inteiro
11      * 5 - Conteúdo não é um texto
12      * 6 - Data em formato inválido
13      * 12 - Na atualização, pelo menos um atributo deve ser passado
14      * 99 - Parâmetros passados do front não correspondem ao método
15      */
16
17      //Atributos privados da classe
18      private $codigo;
19      private $dataReserva;
20      private $codigo_sala;
21      private $codigo_horario;
22      private $codigo_turma;
23      private $codigo_professor;
24      private $status;
25      //Atributos para os mapeamentos
26      private $dataInicio;
27      private $dataFim;
28      private $diaSemana;
29

```

```

30 //Getters dos atributos
31 public function getCodigo()
32 {
33     return $this->codigo;
34 }
35
36 public function getDataReserva()
37 {
38     return $this->dataReserva;
39 }
40
41 public function getCodigoSala()
42 {
43     return $this->codigo_sala;
44 }
45
46 public function getCodigoHorario()
47 {
48     return $this->codigo_horario;
49 }
50
51 public function getCodigoTurma()
52 {
53     return $this->codigo_turma;
54 }
55
56 public function getProfessor()
57 {
58     return $this->codigo_professor;
59 }
60
61 public function getEstatus()
62 {
63     return $this->estatus;
64 }
65
66 public function getDataInicio()
67 {
68     return $this->dataInicio;
69 }
70
71 public function getDataFim()
72 {
73     return $this->dataFim;
74 }
75
76 public function getDiaSemana()
77 {
78     return $this->diaSemana;
79 }
80
81 //Setters dos atributos
82 public function setCodigo($codigoFront)
83 {
84     $this->codigo = $codigoFront;
85 }
86
87 public function setDataReserva($dataReservaFront)
88 {
89     $this->dataReserva = $dataReservaFront;
90 }
91
92 public function setCodigoSala($codigo_salaFront)
93 {
94     $this->codigo_sala = $codigo_salaFront;
95 }
96
97 public function setCodigoHorario($codigo_horarioFront)
98 {
99     $this->codigo_horario = $codigo_horarioFront;
100 }
101

```

```

102     public function setCodigoTurma($codigo_turmaFront)
103     {
104         $this->codigo_turma = $codigo_turmaFront;
105     }
106
107     public function setProfessor($professorFront)
108     {
109         $this->codigo_professor = $professorFront;
110     }
111
112     public function setEstatus($estatusFront)
113     {
114         $this->estatus = $estatusFront;
115     }
116
117     public function setDataInicio($dataInicioFront)
118     {
119         $this->dataInicio = $dataInicioFront;
120     }
121
122     public function setDataFim($dataFimFront)
123     {
124         $this->dataFim = $dataFimFront;
125     }
126
127     public function setDiaSemana($diaSemanaFront)
128     {
129         $this->diaSemana = $diaSemanaFront;
130     }
131
132     public function inserir() {
133         //Atributos para controlar o status de nosso método
134         $erros = [];
135         $sucesso = false;
136
137         try {
138
139             $json = file_get_contents('php://input');
140             $resultado = json_decode($json);
141             $lista = [
142                 "dataReserva" => '0',
143                 "codSala" => '0',
144                 "codHorario" => '0',
145                 "codTurma" => '0',
146                 "codProfessor" => '0'
147             ];
148
149             if (verificarParam($resultado, $lista) != 1) {
150                 // Validar vindos de forma correta do frontend (Helper)
151                 $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no
152                                     FrontEnd.'];
153             }else{
154                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
155                 $retornoDataReserva = validarDados($resultado->dataReserva,'date',true);
156                 $retornoCodSala = validarDados($resultado->codSala,'int',true);
157                 $retornoCodHorario = validarDados($resultado->codHorario,'int',true);
158                 $retornoCodTurma = validarDados($resultado->codTurma,'int',true);
159                 $retornoCodProfessor = validarDados($resultado->codProfessor,'int',true);
160
161                 if($retornoDataReserva['codigoHelper'] != 0){
162                     $erros[] = ['codigo' => $retornoDataReserva['codigoHelper'],
163                                     'campo' => 'Data de Reserva',
164                                     'msg' => $retornoDataReserva['msg']];
165                 }
166
167                 if($retornoCodSala['codigoHelper'] != 0){
168                     $erros[] = ['codigo' => $retornoCodSala['codigoHelper'],
169                                     'campo' => 'Código da Sala',
170                                     'msg' => $retornoCodSala['msg']];
171                 }
172

```

```

173         if($retornoCodHorario['codigoHelper'] != 0){
174             $erros[] = ['codigo' => $retornoCodHorario['codigoHelper'],
175                       'campo' => 'Código do horário',
176                       'msg' => $retornoCodHorario['msg']];
177         }
178
179         if($retornoCodTurma['codigoHelper'] != 0){
180             $erros[] = ['codigo' => $retornoCodTurma['codigoHelper'],
181                       'campo' => 'Código da Turma',
182                       'msg' => $retornoCodTurma['msg']];
183         }
184
185         if($retornoCodProfessor['codigoHelper'] != 0){
186             $erros[] = ['codigo' => $retornoCodProfessor['codigoHelper'],
187                       'campo' => 'Código do professor',
188                       'msg' => $retornoCodProfessor['msg']];
189         }
190
191         //Se não encontrar erros
192         if (empty($erros)) {
193             $this->setDataReserva($resultado->dataReserva);
194             $this->setCodigoSala($resultado->codSala);
195             $this->setCodigoHorario($resultado->codHorario);
196             $this->setCodigoTurma($resultado->codTurma);
197             $this->setProfessor($resultado->codProfessor);
198
199             $this->load->model('M_mapa');
200             $resBanco = $this->M_mapa->inserir(
201                 $this->getDataReserva(),
202                 $this->getCodigoSala(),
203                 $this->getCodigoHorario(),
204                 $this->getCodigoTurma(),
205                 $this->getProfessor()
206             );
207
208             if ($resBanco['codigo']== 1) {
209                 $sucesso = true;
210             } else {
211                 // Captura erro do banco
212                 $erros[] = [
213                     'codigo' => $resBanco['codigo'],
214                     'msg' => $resBanco['msg']
215                 ];
216             }
217         }
218     }
219     catch (Exception $e) {
220         $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
221     }
222
223     // Monta retorno único
224     if ($sucesso == true) {
225         $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
226                   'msg' => $resBanco['msg']];
227     } else {
228         $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
229     }
230
231     // Transforma o array em JSON
232     echo json_encode($retorno);
233 }
234
235 public function consultar() {
236     //Atributos para controlar o status de nosso método
237     $erros = [];
238     $sucesso = false;
239

```

```

240     try {
241
242         $json = file_get_contents('php://input');
243         $resultado = json_decode($json);
244         $lista = [
245             "codigo" => '0',
246             "dataReserva" => '0',
247             "codSala" => '0',
248             "codHorario" => '0',
249             "codTurma" => '0',
250             "codProfessor" => '0'
251         ];
252
253         if (verificarParam($resultado, $lista) != 1) {
254             // Validar vindos de forma correta do frontend (Helper)
255             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no
256                 FrontEnd.'];
257         }else{
258             // Validar campos quanto ao tipo de dado e tamanho (Helper)
259             $retornoCodigo = validarDadosConsulta($resultado->codigo, 'int');
260             $retornoDataReserva = validarDadosConsulta($resultado->dataReserva, 'date');
261             $retornoCodSala = validarDadosConsulta($resultado->codSala, 'int');
262             $retornoCodHorario = validarDadosConsulta($resultado->codHorario, 'int');
263             $retornoCodTurma = validarDadosConsulta($resultado->codTurma, 'int');
264             $retornoCodProfessor = validarDadosConsulta($resultado->codProfessor, 'int');
265
266             if($retornoCodigo['codigoHelper'] != 0){
267                 $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
268                     'campo' => 'Código',
269                     'msg' => $retornoCodigo['msg']];
270             }
271
272             if($retornoDataReserva['codigoHelper'] != 0){
273                 $erros[] = ['codigo' => $retornoDataReserva['codigoHelper'],
274                     'campo' => 'Data de Reserva',
275                     'msg' => $retornoDataReserva['msg']];
276             }
277
278             if($retornoCodSala['codigoHelper'] != 0){
279                 $erros[] = ['codigo' => $retornoCodSala['codigoHelper'],
280                     'campo' => 'Código da Sala',
281                     'msg' => $retornoCodSala['msg']];
282             }
283
284             if($retornoCodHorario['codigoHelper'] != 0){
285                 $erros[] = ['codigo' => $retornoCodHorario['codigoHelper'],
286                     'campo' => 'Código do horário',
287                     'msg' => $retornoCodHorario['msg']];
288             }
289
290             if($retornoCodTurma['codigoHelper'] != 0){
291                 $erros[] = ['codigo' => $retornoCodTurma['codigoHelper'],
292                     'campo' => 'Código da Turma',
293                     'msg' => $retornoCodTurma['msg']];
294             }
295
296             if($retornoCodProfessor['codigoHelper'] != 0){
297                 $erros[] = ['codigo' => $retornoCodProfessor['codigoHelper'],
298                     'campo' => 'Código do professor',
299                     'msg' => $retornoCodProfessor['msg']];
300             }
301
302             //Se não encontrar erros
303             if (empty($erros)) {
304                 $this->setCodigo($resultado->codigo);
305                 $this->setDataReserva($resultado->dataReserva);

```



```

306     $this->setCodigoSala($resultado->codSala);
307     $this->setCodigoHorario($resultado->codHorario);
308     $this->setCodigoTurma($resultado->codTurma);
309     $this->setProfessor($resultado->codProfessor);
310
311     $this->load->model('M_mapa');
312     $resBanco = $this->M_mapa->consultar($this->getCodigo(),
313                                     $this->getDataReserva(),
314                                     $this->getCodigoSala(),
315                                     $this->getCodigoHorario(),
316                                     $this->getCodigoTurma(),
317                                     $this->getProfessor()
318                                     );
319
320     if ($resBanco['codigo']== 1) {
321         $sucesso = true;
322     } else {
323         // Captura erro do banco
324         $erros[] = [
325             'codigo' => $resBanco['codigo'],
326             'msg'     => $resBanco['msg']
327         ];
328     }
329 }
330
331 } catch (Exception $e) {
332     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
333 }
334
335 // Monta retorno único
336 if ($sucesso == true) {
337     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
338               'msg' => $resBanco['msg'],
339               'dados' => $resBanco['dados']];
340 } else {
341     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
342 }
343
344 // Transforma o array em JSON
345 echo json_encode($retorno);
346 }
347
348 public function alterar() {
349     //Atributos para controlar o status de nosso método
350     $erros = [];
351     $sucesso = false;
352
353     try {
354
355         $json = file_get_contents('php://input');
356         $resultado = json_decode($json);
357         $lista = [
358             "codigo" => '0',
359             "dataReserva" => '0',
360             "codSala" => '0',
361             "codHorario" => '0',
362             "codTurma" => '0',
363             "codProfessor" => '0'
364         ];
365
366         if (verificarParam($resultado, $lista) != 1) {
367             // Validar vindos de forma correta do frontend (Helper)
368             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
369         } else {
370             //Pelo menos um dos três parâmetros precisam ter dados para acontecer a atualização
371             if(trim($resultado->dataReserva) == '' && trim($resultado->codSala) == '' &&
372                trim($resultado->codHorario) == '' && trim($resultado->codTurma) == '' &&

```

```

373 trim($resultado->codProfessor) == ''){
374     $erros[] = ['codigo' => 12,
375               'msg' => 'Pelo menos um parâmetro precisa ser passado para atualização'];
376 }else{
377     // Validar campos quanto ao tipo de dado e tamanho (Helper)
378     $retornoCodigo = validarDados($resultado->codigo,'int', true);
379     $retornoDataReserva = validarDadosConsulta($resultado->dataReserva,'date');
380     $retornoCodSala = validarDadosConsulta($resultado->codSala,'int');
381     $retornoCodHorario = validarDadosConsulta($resultado->codHorario,'int');
382     $retornoCodTurma = validarDadosConsulta($resultado->codTurma,'int');
383     $retornoCodProfessor = validarDadosConsulta($resultado->codProfessor,'int');
384
385     if($retornoCodigo['codigoHelper'] != 0){
386         $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
387                   'campo' => 'Codigo',
388                   'msg' => $retornoCodigo['msg']];
389     }
390
391     if($retornoDataReserva['codigoHelper'] != 0){
392         $erros[] = ['codigo' => $retornoDataReserva['codigoHelper'],
393                   'campo' => 'Data de Reserva',
394                   'msg' => $retornoDataReserva['msg']];
395     }
396
397     if($retornoCodSala['codigoHelper'] != 0){
398         $erros[] = ['codigo' => $retornoCodSala['codigoHelper'],
399                   'campo' => 'Código da Sala',
400                   'msg' => $retornoCodSala['msg']];
401     }
402
403     if($retornoCodHorario['codigoHelper'] != 0){
404         $erros[] = ['codigo' => $retornoCodHorario['codigoHelper'],
405                   'campo' => 'Código do horário',
406                   'msg' => $retornoCodHorario['msg']];
407     }
408
409     if($retornoCodTurma['codigoHelper'] != 0){
410         $erros[] = ['codigo' => $retornoCodTurma['codigoHelper'],
411                   'campo' => 'Código da Turma',
412                   'msg' => $retornoCodTurma['msg']];
413     }
414
415     if($retornoCodProfessor['codigoHelper'] != 0){
416         $erros[] = ['codigo' => $retornoCodProfessor['codigoHelper'],
417                   'campo' => 'Código do professor',
418                   'msg' => $retornoCodProfessor['msg']];
419     }
420     //Se não encontrar erros
421     if (empty($erros)) {
422         $this->setCodigo($resultado->codigo);
423         $this->setDataReserva($resultado->dataReserva);
424         $this->setCodigoSala($resultado->codSala);
425         $this->setCodigoHorario($resultado->codHorario);
426         $this->setCodigoTurma($resultado->codTurma);
427         $this->setProfessor($resultado->codProfessor);
428
429         $this->load->model('M_mapa');
430         $resBanco = $this->M_mapa->alterar($this->getCodigo(),
431                                           $this->getDataReserva(),
432                                           $this->getCodigoSala(),
433                                           $this->getCodigoHorario(),
434                                           $this->getCodigoTurma(),
435                                           $this->getProfessor());
436
437         if ($resBanco['codigo']== 1) {
438             $sucesso = true;
439         } else {

```

```

440         // Captura erro do banco
441         $erros[] = [
442             'codigo' => $resBanco['codigo'],
443             'msg' => $resBanco['msg']
444         ];
445     }
446 }
447 }
448 }
449 } catch (Exception $e) {
450     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
451 }
452
453 // Monta retorno único
454 if ($sucesso == true) {
455     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
456                 'msg' => $resBanco['msg']];
457 } else {
458     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
459 }
460
461 // Transforma o array em JSON
462 echo json_encode($retorno);
463 }
464
465 public function desativar() {
466     //Atributos para controlar o status de nosso método
467     $erros = [];
468     $sucesso = false;
469
470     try {
471
472         $json = file_get_contents('php://input');
473         $resultado = json_decode($json);
474
475         $lista = [
476             "codigo" => '0'
477         ];
478
479         if (verificarParam($resultado, $lista) != 1) {
480             // Validar vindos de forma correta do frontend (Helper)
481             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
482         } else {
483             // Validar código quanto ao tipo de dado e tamanho (Helper)
484             $retornoCodigo = validarDados($resultado->codigo, 'int', true);
485
486             if ($retornoCodigo['codigoHelper'] != 0) {
487                 $erros[] = ['codigo' => $retornoCodigo['codigoHelper'],
488                             'campo' => 'Codigo',
489                             'msg' => $retornoCodigo['msg']];
490             }
491
492             //Se não encontrar erros
493             if (empty($erros)) {
494                 $this->setCodigo($resultado->codigo);
495
496                 $this->load->model('M_mapa');
497                 $resBanco = $this->M_mapa->desativar($this->getCodigo());
498
499                 if ($resBanco['codigo'] == 1) {
500                     $sucesso = true;
501                 } else {
502                     // Captura erro do banco
503                     $erros[] = [
504                         'codigo' => $resBanco['codigo'],
505                         'msg' => $resBanco['msg']
506                     ];
507                 }
508             }
509         }
510     }
511 }

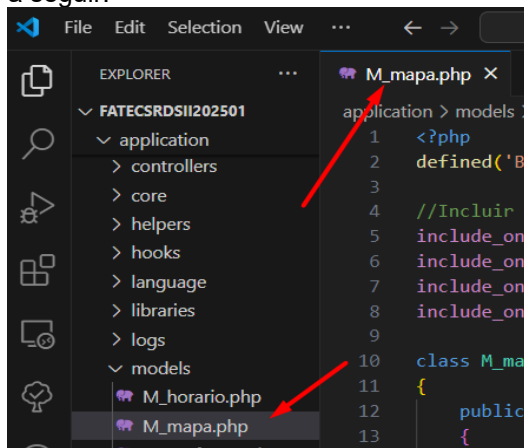
```

```

509     }
510 } catch (Exception $e) {
511     $erros[] = ['codigo' => 0, 'msg' => 'Erro inesperado: ' . $e->getMessage()];
512 }
513
514 // Monta retorno único
515 if ($sucesso == true) {
516     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
517                 'msg' => $resBanco['msg']];
518 } else {
519     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
520 }
521
522 // Transforma o array em JSON
523 echo json_encode($retorno);
524 }
525 }
526 ?>

```

Agora vamos implementar a camada de modelo, ou seja, nossa model, assim sendo, vamos criar um arquivo na pasta models de nossa estrutura, o arquivo M_mapa.php como a figura a seguir:



Percebam no começo desta classe que faremos menção aos outros objetos criados, para que possamos fazer a instância dos objetos, e vamos codificar da forma a seguir:

```

M_mapa.php U X
application > models > M_mapa.php
1 <?php
2 defined('BASEPATH') or exit('No direct script access allowed');
3
4 //Incluir a classe que precisaremos instanciar
5 include_once("M_sala.php");
6 include_once("M_horario.php");
7 include_once("M_turma.php");
8 include_once("M_professor.php");
9
10 class M_mapa extends CI_Model{
11     /*
12     Validação dos tipos de retornos nas validações (Código de erro)
13     0 - Erro de exceção
14     1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
15     7 - Reserva desativada no sistema
16     8 - A data está ocupada para esta sala
17     9 - Houve algum problema de inserção, atualização, consulta ou exclusão
18     10 - Reserva já cadastrada
19     11 - Reserva não encontrada
20     98 - Método auxiliar de consulta que não trouxe dados
21     */
22
23     public function inserir($dataReserva, $codSala, $codHorario, $codTurma, $codProfessor){
24         try {
25             //Verifico se a reserva já está cadastrada
26             $retornoConsultaReservaTotal = $this->consultaReservaTotal($dataReserva, $codSala, $codHorario,
27                 $codTurma, $codProfessor);
28

```

```

29         if ($retornoConsultaReservaTotal['codigo'] == 11 ||
30             $retornoConsultaReservaTotal['codigo'] == 7) {
31             //Chamo o objeto sala para validação
32             $salaObj = new M_sala();
33
34             //chamar o método de verificação
35             $retornoConsultaSala = $salaObj->consultar($codSala, '', '', '');
36
37             if ($retornoConsultaSala['codigo'] == 1) {
38                 //Chamo o objeto sala para validação
39                 $horarioObj = new M_horario();
40
41                 //chamar o método de verificação
42                 $retornoConsultaHorario = $horarioObj->consultarHorario($codHorario, '', '');
43
44                 if ($retornoConsultaHorario['codigo'] == 10) {
45                     //Chamo o objeto sala para validação
46                     $turmaObj = new M_turma();
47
48                     //chamar o método de verificação
49                     $retornoConsultaTurma = $turmaObj->consultaTurmaCod($codTurma);
50
51                     if ($retornoConsultaTurma['codigo'] == 10) {
52                         //Chamo o objeto sala para validação
53                         $professorObj = new M_professor();
54
55                         //chamar o método de verificação
56                         $retornoConsultaProfessor = $professorObj->consultar($codProfessor, '', '', '');
57
58                         if ($retornoConsultaProfessor['codigo'] == 1) {
59                             //Query de inserção dos dados
60                             $this->db->query("insert into tbl_mapa (datareserva, sala, codigo_horario,
61                                     codigo_turma, codigo_professor)
62                                     values ('" . $dataReserva . "', $codSala, $codHorario,
63                                     $codTurma, $codProfessor)");
64
65                             //Verificar se a inserção ocorreu com sucesso
66                             if ($this->db->affected_rows() > 0) {
67                                 $dados = array(
68                                     'codigo' => 1,
69                                     'msg' => 'Agendamento cadastrado corretamente.'
70                                 );
71                             } else {
72                                 $dados = array(
73                                     'codigo' => 9,
74                                     'msg' => 'Houve algum problema na inserção na tabela de
75                                     agendamento.'
76                                 );
77                             }
78                         } else {
79                             $dados = array(
80                                 'codigo' => $retornoConsultaProfessor['codigo'],
81                                 'msg' => $retornoConsultaProfessor['msg']
82                             );
83                         }
84                     } else {
85                         $dados = array(
86                             'codigo' => $retornoConsultaTurma['codigo'],
87                             'msg' => $retornoConsultaTurma['msg']
88                         );
89                     }
90                 } else {
91
92                     $dados = array(
93                         'codigo' => $retornoConsultaHorario['codigo'],
94                         'msg' => $retornoConsultaHorario['msg']
95                     );
96                 }

```

```

97         } else {
98
99             $dados = array(
100                 'codigo' => $retornoConsultaSala['codigo'],
101                 'msg' => $retornoConsultaSala['msg']
102             );
103         }
104     } else {
105
106         $dados = array(
107             'codigo' => $retornoConsultaReservaTotal['codigo'],
108             'msg' => $retornoConsultaReservaTotal['msg']
109         );
110     }
111 } catch (Exception $e) {
112     $dados = array(
113         'codigo' => 00,
114         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
115     );
116 }
117 //Envia o array $dados com as informações tratadas
118 //acima pela estrutura de decisão if
119 return $dados;
120 }
121
122 private function consultaReservaTotal($dataReserva, $codSala, $codHorario)
123 {
124     try {
125         //Query para verificar a hora inicial e final daquele determinado horário
126         $sql = "select * from tbl_horario
127             where codigo = $codHorario";
128
129         $retornoHorario = $this->db->query($sql);
130
131         if ($retornoHorario->num_rows() > 0) {
132             $linhaHr = $retornoHorario->row();
133             $horaInicial = $linhaHr->hora_ini;
134             $horaFinal = $linhaHr->hora_fim;
135
136             //Query para consultar dados de acordo com parâmetros passados
137             $sql = "select * from tbl_mapa m, tbl_horario h
138                 where m.datareserva = ' . $dataReserva . '
139                     and m.sala = $codSala
140                     and m.codigo_horario = h.codigo
141                     and (h.hora_fim >= ' . $horaInicial . '
142                     and h.hora_ini <= ' . $horaFinal . ')";
143             $retornoMapa = $this->db->query($sql);
144
145             //Verificar se a consulta ocorreu com sucesso
146             if ($retornoMapa->num_rows() > 0) {
147                 $linha = $retornoMapa->row();
148
149                 if (trim($linha->estatus) == "D") {
150                     $dados = array(
151                         'codigo' => 7,
152                         'msg' => 'Agendamento desativado no sistema.'
153                     );
154                 } else {
155                     $dados = array(
156                         'codigo' => 8,
157                         'msg' => 'A data de ' . $dataReserva . ' está ocupada para esta sala'
158                     );
159                 }
160             } else {
161                 $dados = array(
162                     'codigo' => 11,
163                     'msg' => 'Reserva não encontrada.'
164                 );
165             }
166         }
167     }
168 }

```

```

166         }else{
167             $dados = array(
168                 'codigo' => 11,
169                 'msg' => 'Reserva não encontrada.'
170             );
171         }
172     }
173     } catch (Exception $e) {
174         $dados = array(
175             'codigo' => 00,
176             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
177         );
178     }
179     //Envia o array $dados com as informações tratadas
180     //acima pela estrutura de decisão if
181     return $dados;
182 }
183
184 public function consultar($codigo, $dataReserva, $codSala, $codHorario, $codTurma, $codProfessor)
185 {
186     try {
187         //Query para consultar dados de acordo com parâmetros passados
188         $sql = "select m.codigo, date_format(m.datareserva,'%d-%m-%Y') datareservabra, datareserva,
189             m.sala, s.descricao descsala, m.codigo_horario,
190             h.descricao deshorario, m.codigo_turma, t.descricao desc turma, m.codigo_professor,
191             p.nome nome_professor
192             from tbl_mapa m, tbl_professor p, tbl_horario h, tbl_turma t, tbl_sala s
193             where m.estatus = ''
194                 and m.codigo_professor = p.codigo
195                 and m.codigo_horario = h.codigo
196                 and m.codigo_turma = t.codigo
197                 and m.sala = s.codigo ";
198
199         if (trim($codigo) != '') {
200             $sql = $sql . "and m.codigo = $codigo ";
201         }
202
203         if (trim($dataReserva) != '') {
204             $sql = $sql . "and m.datareserva = '" . $dataReserva . "' ";
205         }
206
207         if (trim($codSala) != '') {
208             $sql = $sql . "and m.sala = $codSala ";
209         }
210
211         if (trim($codHorario) != '') {
212             $sql = $sql . "and m.codigo_horario = $codHorario ";
213         }
214
215         if (trim($codTurma) != '') {
216             $sql = $sql . "and m.codigo_turma = $codTurma ";
217         }
218
219         if (trim($codProfessor) != '') {
220             $sql = $sql . "and m.codigo_professor = $codProfessor ";
221         }
222
223         $sql = $sql . " order by m.datareserva, h.hora_ini, m.codigo_horario, m.sala ";
224
225         $retorno = $this->db->query($sql);
226
227         //Verificar se a consulta ocorreu com sucesso
228         if ($retorno->num_rows() > 0) {
229             $dados = array(
230                 'codigo' => 1,
231                 'msg' => 'Consulta efetuada com sucesso.',
232                 'dados' => $retorno->result()
233             );
234         } else {

```

```

235         $dados = array(
236             'codigo' => 11,
237             'msg' => 'Agendamento(s) não encontrado(s).'
238         );
239     }
240 } catch (Exception $e) {
241     $dados = array(
242         'codigo' => 00,
243         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
244     );
245 }
246 //Envia o array $dados com as informações tratadas
247 //acima pela estrutura de decisão if
248 return $dados;
249 }
250
251 public function alterar($codigo, $dataReserva, $codSala, $codHorario, $codTurma, $codProfessor)
252 {
253     try {
254         //Verifico se o professor já está cadastrado
255
256         $retornoConsultaCodigo = $this->consultar(
257             $codigo,
258             "",
259             "",
260             "",
261             "",
262             ""
263         );
264
265         if ($retornoConsultaCodigo['codigo'] == 1) {
266             //Inicio a query para atualização
267             $query = "update tbl_mapa set ";
268
269             if ($dataReserva != "") {
270                 $query .= "datareserva = '$dataReserva', ";
271             }
272
273             if ($codSala != "") {
274                 //Chamo o objeto sala para validação
275                 $salaObj = new M_sala();
276
277                 //chamar o método de verificação
278                 $retornoConsultaSala = $salaObj->consultar($codSala, '', '', '');
279
280                 if ($retornoConsultaSala['codigo'] == 1) {
281                     $query .= "sala = $codSala, ";
282                 } else {
283                     $dados = array(
284                         'codigo' => $retornoConsultaSala['codigo'],
285                         'msg' => $retornoConsultaSala['msg']
286                     );
287                 }
288             }
289
290             if ($codHorario != "") {
291                 //Chamo o objeto sala para validação
292                 $horarioObj = new M_horario();
293
294                 //chamar o método de verificação
295                 $retornoConsultaHorario = $horarioObj->consultarHorario($codHorario, '', '');
296
297                 if ($retornoConsultaHorario['codigo'] == 1) {
298                     $query .= "codigo_horario = $codHorario, ";
299                 } else {
300                     $dados = array(
301                         'codigo' => $retornoConsultaHorario['codigo'],
302                         'msg' => $retornoConsultaHorario['msg']

```



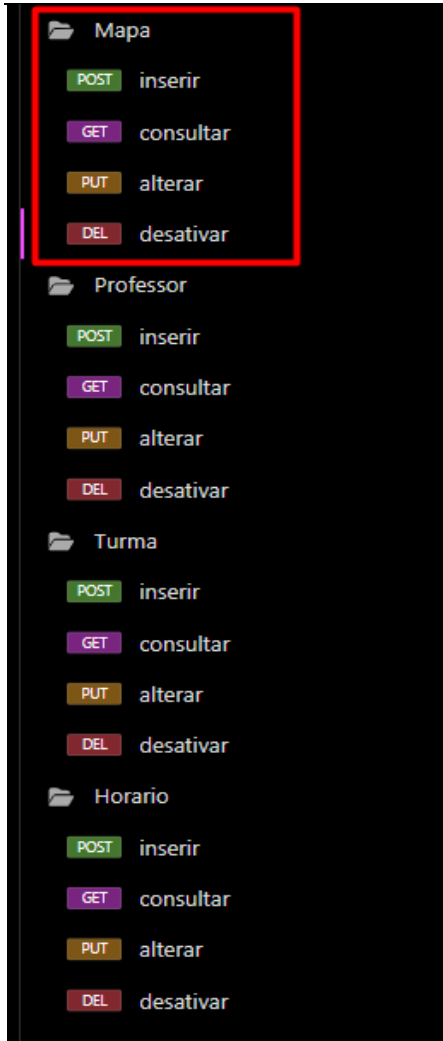
```

303         );
304     }
305 }
306
307 if ($codTurma != "") {
308     //Chamo o objeto sala para validação
309     $turmaObj = new M_turma();
310
311     //chamar o método de verificação
312     $retornoConsultaTurma = $turmaObj->consultaTurmaCod($codTurma);
313
314     if ($retornoConsultaTurma['codigo'] == 1) {
315         $query .= "codigo_turma = $codTurma, ";
316     } else {
317         $dados = array(
318             'codigo' => $retornoConsultaTurma['codigo'],
319             'msg' => $retornoConsultaTurma['msg']
320         );
321     }
322 }
323
324 if ($codProfessor != "") {
325     //Chamo o objeto sala para validação
326     $professorObj = new M_professor();
327
328     //chamar o método de verificação
329     $retornoConsultaProfessor = $professorObj->consultar($codProfessor, ',', ',');
330
331     if ($retornoConsultaProfessor['codigo'] == 1) {
332         $query .= "codigo_professor = $codProfessor, ";
333     } else {
334         $dados = array(
335             'codigo' => $retornoConsultaProfessor['codigo'],
336             'msg' => $retornoConsultaProfessor['msg']
337         );
338     }
339 }
340
341 //Termino a concatenação da query
342 $queryFinal = rtrim($query, ", ") . " where codigo = $codigo";
343
344 //Executo a Query de atualização dos dados
345 $this->db->query($queryFinal);
346
347 //Verificar se a atualização ocorreu com sucesso
348 if ($this->db->affected_rows() > 0) {
349     $dados = array(
350         'codigo' => 1,
351         'msg' => 'Agendamento alterado corretamente.'
352     );
353 } else {
354     $dados = array(
355         'codigo' => 9,
356         'msg' => 'Houve algum problema na alteração na tabela de agendamento.'
357     );
358 }
359 } else {
360
361     $dados = array(
362         'codigo' => $retornoConsultaCodigo['codigo'],
363         'msg' => $retornoConsultaCodigo['msg']
364     );
365 }
366 } catch (Exception $e) {
367     $dados = array(
368         'codigo' => 00,

```

```
369         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
370     );
371 }
372 //Envia o array $dados com as informações tratadas
373 //acima pela estrutura de decisão if
374 return $dados;
375 }
376
377 public function desativar($codigo)
378 {
379     try {
380         //Verifico se o agendamento já está cadastrado
381         $retornoConsulta = $this->consultar(
382             $codigo,
383             "",
384             "",
385             "",
386             "",
387             ""
388         );
389
390         if ($retornoConsulta['codigo'] == 1) {
391
392             //Query de atualização dos dados
393             $this->db->query("delete from tbl_mapa
394                 where codigo = $codigo");
395
396             //Verificar se a atualização ocorreu com sucesso
397             if ($this->db->affected_rows() > 0) {
398                 $dados = array(
399                     'codigo' => 1,
400                     'msg' => 'Agendamento DESATIVADO corretamente.'
401                 );
402             } else {
403                 $dados = array(
404                     'codigo' => 9,
405                     'msg' => 'Houve algum problema na DESATIVAÇÃO do Agendamento.'
406                 );
407             }
408         } else {
409             $dados = array(
410                 'codigo' => $retornoConsulta['codigo'],
411                 'msg' => $retornoConsulta['msg']
412             );
413         }
414     } catch (Exception $e) {
415         $dados = array(
416             'codigo' => 00,
417             'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ' . $e->getMessage()
418         );
419     }
420     //Envia o array $dados com as informações tratadas
421     //acima pela estrutura de decisão if
422     return $dados;
423 }
424 }
425 ?>
```

Com isso, finalizamos a implementação de nosso quinto objeto do sistema, os *endpoints* no *Insomnia* e que vocês devem implementar e são importantes para o salvamento, recomeço que ao final de cada aula, façam a exportação do mesmo e guardem para que possam atualizar e utilizar durante as aulas, abaixo vou mostrar como ficou estes *endpoints* no final.



Assim, agora temos 5 objetos criados em nosso sistema, ou seja, temos a base dos CRUDs e a realização do Mapeamento.

6.5.7 Objeto Usuario

Já temos a base de nosso projeto, agora para partimos para a efetiva realização do front, temos que fazer o usuário, para isso teremos que confeccionar mais um CRUD, a única coisa a salientar é que colocaremos um método login, onde utilizaremos na tela de login, para isso vamos ter que acrescentar mais uma tabela em nosso banco de dados, a tbl_usuario, para isso teremos que codificar a mesma, vejam:

```

75      #CRIAR TABELA DE USUÁRIOS
76      • create table tbl_usuario(
77          id_usuario integer auto_increment primary key,
78          nome          varchar(30),
79          email          varchar(40),
80          usuario        varchar(20),
81          senha          char(32),
82          dtcria          datetime default now(),
83          estatus         char(01) default ''
84      );
    
```

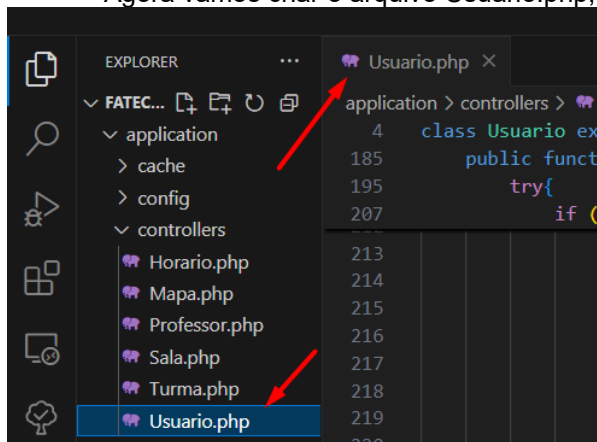
É recomendável que codifiquem a mesma dando continuidade no arquivo do banco que já tenham, pois, fica todo documentado em um único local, lembre-se que temos este arquivo em nosso repositório do GitHub, complete ele.

Antes de darmos continuidade com a confecção da *Controller*, vamos colocar mais uma função em nossa *Helper*, vamos validar o e-mail informado, através da função **filter_var()**, a mesma é responsável por validar a formatação do e-mail informado, para isso, vamos acrescentar mais um atributo em nosso case das funções da *Helper* **validarDados()** e **validarDadosConsulta()**, veja:

```
71 case 'email':
72     //Verifico se tem padrão de hora
73     if(!filter_var($valor, FILTER_VALIDATE_EMAIL)){
74         return array('codigoHelper' => 8, 'msg' => 'E-mail em formato inválido.');
```

Só acrescentem esta verificação nas duas funções.

Agora vamos criar o arquivo *Usuario.php*, em nossa *Controller*, vejamos:



E codificando assim:

```
Usuario.php U X
application > controllers > Usuario.php
1 <?php
2 defined('BASEPATH') OR exit('No direct script access allowed');
3
4 class Usuario extends CI_Controller {
5     /*
6     Validação dos tipos de retornos nas validações (Código de erro)
7     1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
8     2 - Conteúdo passado nulo ou vazio
9     3 - Conteúdo zerado
10    4 - Conteúdo não inteiro
11    5 - Conteúdo não é um texto
12    8 - E-mail em formato inválido
13    12 - Na atualização, pelo menos um atributo deve ser passado
14    99 - Parâmetros passados do front não correspondem ao método
15    */
16
17    //Atributos privados da classe
18    private $idUserio;
19    private $nome;
20    private $email;
21    private $usuario;
22    private $senha;
23
24    //Getters dos atributos
25    public function getIdUsuario()
26    {
27        return $this->idUserio;
28    }
29
```

```

30     public function getNome()
31     {
32         return $this->nome;
33     }
34
35     public function getEmail()
36     {
37         return $this->email;
38     }
39
40     public function getUsuario()
41     {
42         return $this->usuario;
43     }
44
45     public function getSenha()
46     {
47         return $this->senha;
48     }
49
50     //Setters dos atributos
51     public function setIdUsuario($idUserioFront)
52     {
53         $this->idUserio = $idUserioFront;
54     }
55
56     public function setName($nomeFront)
57     {
58         $this->nome = $nomeFront;
59     }
60
61     public function setEmail($emailFront)
62     {
63         $this->email = $emailFront;
64     }
65
66     public function setUsuario($usuarioFront)
67     {
68         $this->usuario = $usuarioFront;
69     }
70
71     public function setSenha($senhaFront)
72     {
73         $this->senha = $senhaFront;
74     }
75
76     public function inserir(){
77         //Atributos para controlar o status de nosso método
78         $erros = [];
79         $sucesso = false;
80
81         try {
82             $json = file_get_contents('php://input');
83             $resultado = json_decode($json);
84
85             //Array com os dados que deverão vir do Front
86             $lista = array(
87                 "nome" => '0',
88                 "email" => '0',
89                 "usuario" => '0',
90                 "senha" => '0'
91             );
92
93             if (verificarParam($resultado, $lista) != 1) {
94                 // Validar vindos de forma correta do frontend (Helper)
95                 $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no
96                                     FrontEnd.'];
97             }else{
98                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
99                 $retornoNome = validarDados($resultado->nome,'string',true);
100                 $retornoEmail = validarDados($resultado->email,'email',true);
101                 $retornoUsuario = validarDados($resultado->usuario,'string',true);
102                 $retornoSenha = validarDados($resultado->senha,'string',true);
103

```

```

104         if($retornoNome['codigoHelper'] != 0){
105             $erros[] = ['codigo' => $retornoNome['codigoHelper'],
106                 'campo' => 'Nome',
107                 'msg' => $retornoNome['msg']];
108         }
109
110         if($retornoEmail['codigoHelper'] != 0){
111             $erros[] = ['codigo' => $retornoEmail['codigoHelper'],
112                 'campo' => 'E-mail',
113                 'msg' => $retornoEmail['msg']];
114         }
115
116         if($retornoUsuario['codigoHelper'] != 0){
117             $erros[] = ['codigo' => $retornoUsuario['codigoHelper'],
118                 'campo' => 'Usuário',
119                 'msg' => $retornoUsuario['msg']];
120         }
121
122         if($retornoSenha['codigoHelper'] != 0){
123             $erros[] = ['codigo' => $retornoSenha['codigoHelper'],
124                 'campo' => 'Data Inicio',
125                 'msg' => $retornoSenha['msg']];
126         }
127
128         //Se não encontrar erros
129         if (empty($erros)) {
130             //Fazendo os setters
131             $this->setNome($resultado->nome);
132             $this->setEmail($resultado->email);
133             $this->setUsuario($resultado->usuario);
134             $this->setSenha($resultado->senha);
135
136
137             //Realizo a instância da Model
138             $this->load->model('M_usuario');
139
140             //Atributo $retorno recebe array com informações
141             //da validação do acesso
142             $resBanco = $this->M_usuario->inserir($this->getNome(),
143                 $this->getEmail(),
144                 $this->getUsuario(),
145                 $this->getSenha());
146
147             if ($resBanco['codigo']== 1) {
148                 $sucesso = true;
149             } else {
150                 // Captura erro do banco
151                 $erros[] = [
152                     'codigo' => $resBanco['codigo'],
153                     'msg' => $resBanco['msg']
154                 ];
155             }
156         }
157     }
158 }catch (Exception $e) {
159     $retorno = array('codigo' => 0,
160         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
161         $e->getMessage());
162 }
163
164 // Monta retorno único
165 if ($sucesso == true) {
166     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
167         'msg' => $resBanco['msg']];
168 } else {
169     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
170 }
171
172 //Retorno no formato JSON

```

```

173         echo json_encode($retorno);
174     }
175
176     public function consultar(){
177         //Atributos para controlar o status de nosso método
178         $erros = [];
179         $sucesso = false;
180
181         try {
182             $json = file_get_contents('php://input');
183             $resultado = json_decode($json);
184
185             //Array com os dados que deverão vir do Front
186             $lista = array(
187                 "nome" => '0',
188                 "email" => '0',
189                 "usuario" => '0'
190             );
191
192             if (verificarParam($resultado, $lista) != 1) {
193                 // Validar vindos de forma correta do frontend (Helper)
194                 $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no
195                             FrontEnd.'];
196             }else{
197                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
198                 $retornoNome = validarDadosConsulta($resultado->nome, 'string');
199                 $retornoEmail = validarDadosConsulta($resultado->email, 'email');
200                 $retornoUsuario = validarDadosConsulta($resultado->usuario, 'string');
201
202                 if($retornoNome['codigoHelper'] != 0){
203                     $erros[] = ['codigo' => $retornoNome['codigoHelper'],
204                                 'campo' => 'Nome',
205                                 'msg' => $retornoNome['msg']];
206                 }
207
208                 if($retornoEmail['codigoHelper'] != 0){
209                     $erros[] = ['codigo' => $retornoEmail['codigoHelper'],
210                                 'campo' => 'E-mail',
211                                 'msg' => $retornoEmail['msg']];
212                 }
213
214                 if($retornoUsuario['codigoHelper'] != 0){
215                     $erros[] = ['codigo' => $retornoUsuario['codigoHelper'],
216                                 'campo' => 'Usuário',
217                                 'msg' => $retornoUsuario['msg']];
218                 }
219
220                 //Se não encontrar erros
221                 if (empty($erros)) {
222                     //Fazendo os setters
223                     $this->setNome($resultado->nome);
224                     $this->setEmail($resultado->email);
225                     $this->setUsuario($resultado->usuario);
226
227                     //Realizo a instância da Model
228                     $this->load->model('M_usuario');
229
230                     //Atributo $retorno recebe array com informações
231                     //da consulta dos dados
232                     $resBanco = $this->M_usuario->consultar($this->getNome(),
233                                                         $this->getEmail(),
234                                                         $this->getUsuario());
235                     if ($resBanco['codigo']== 1) {
236                         $sucesso = true;
237                     } else {
238                         // Captura erro do banco
239                         $erros[] = [
240                             'codigo' => $resBanco['codigo'],
241                             'msg' => $resBanco['msg']
242                         ];
243                     }
244                 }
245             }
246         } catch (Exception $e) {
247             $erros[] = ['codigo' => 500, 'msg' => $e->getMessage()];
248         }
249     }
250 }

```

```

244     }
245 }
246 }catch (Exception $e) {
247     $retorno = array('codigo' => 0,
248                     'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
249                     $e->getMessage());
250 }
251
252 // Monta retorno único
253 if ($sucesso == true) {
254     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
255               'msg' => $resBanco['msg'],
256               'dados' => $resBanco['dados']];
257 } else {
258     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
259 }
260
261 //Retorno no formato JSON
262 echo json_encode($retorno);
263 }
264
265 public function alterar(){
266     //Atributos para controlar o status de nosso método
267     $erros = [];
268     $sucesso = false;
269
270     try {
271         $json = file_get_contents('php://input');
272         $resultado = json_decode($json);
273
274         //Array com os dados que deverão vir do Front
275         $lista = array(
276             "idUsuario" => '0',
277             "nome" => '0',
278             "email" => '0',
279             "senha" => '0'
280         );
281
282         if (verificarParam($resultado, $lista) != 1) {
283             // Validar vindos de forma correta do frontend (Helper)
284             $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
285         }else{
286             //Pelo menos um dos três parâmetros precisam ter dados para acontecer a atualização
287             if(trim($resultado->nome) == '' && trim($resultado->email) == '' &&
288               trim($resultado->senha) == ''){
289                 $erros[] = ['codigo' => 12,
290                           'msg' => 'Pelo menos um parâmetro precisa ser passado para atualização'];
291             }else{
292                 // Validar campos quanto ao tipo de dado e tamanho (Helper)
293                 $retornoIdUsuario = validarDados($resultado->idUsuario, 'int');
294                 $retornoNome = validarDadosConsulta($resultado->nome, 'string');
295                 $retornoEmail = validarDadosConsulta($resultado->email, 'string');
296                 $retornoSenha = validarDadosConsulta($resultado->senha, 'string');
297
298                 if($retornoIdUsuario['codigoHelper'] != 0){
299                     $erros[] = ['codigo' => $retornoIdUsuario['codigoHelper'],
300                               'campo' => 'ID Usuário',
301                               'msg' => $retornoIdUsuario['msg']];
302                 }
303
304                 if($retornoNome['codigoHelper'] != 0){
305                     $erros[] = ['codigo' => $retornoNome['codigoHelper'],
306                               'campo' => 'Nome',
307                               'msg' => $retornoNome['msg']];
308                 }
309
310                 if($retornoEmail['codigoHelper'] != 0){
311                     $erros[] = ['codigo' => $retornoEmail['codigoHelper'],
312                               'campo' => 'E-mail',

```



```

313         'msg' => $retornoEmail['msg']);
314     }
315
316     if($retornoSenha['codigoHelper'] != 0){
317         $erros[] = ['codigo' => $retornoSenha['codigoHelper'],
318                 'campo' => 'Senha',
319                 'msg' => $retornoSenha['msg']];
320     }
321
322     //Se não encontrar erros
323     if (empty($erros)) {
324
325         //Fazendo os setters
326         $this->setIdUsuario($resultado->idUsuario);
327         $this->setNome($resultado->nome);
328         $this->setEmail($resultado->email);
329         $this->setSenha($resultado->senha);
330
331         $this->load->model('M_usuario');
332
333         //Atributo $retorno recebe array com informações
334         //da alteração dos dados
335         $resBanco = $this->M_usuario->alterar($this->getIdUsuario(),
336                                           $this->getNome(),
337                                           $this->getEmail(),
338                                           $this->getSenha());
339
340         if ($resBanco['codigo'] == 1) {
341             $sucesso = true;
342         } else {
343             // Captura erro do banco
344             $erros[] = [
345                 'codigo' => $resBanco['codigo'],
346                 'msg' => $resBanco['msg']
347             ];
348         }
349     }
350 }
351 }
352 } catch (Exception $e) {
353     $retorno = array('codigo' => 0,
354                     'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
355                     $e->getMessage());
356 }
357
358 // Monta retorno único
359 if ($sucesso == true) {
360     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
361               'msg' => $resBanco['msg']];
362 } else {
363     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
364 }
365
366 //Retorno no formato JSON
367 echo json_encode($retorno);
368 }
369
370 public function desativar(){
371     //Atributos para controlar o status de nosso método
372     $erros = [];
373     $sucesso = false;
374
375     try {
376         $json = file_get_contents('php://input');
377         $resultado = json_decode($json);
378
379         //Array com os dados que deverão vir do Front
380         $lista = array(

```

```

381         "idUserario" => '0'
382     );
383
384     if (verificarParam($resultado, $lista) != 1) {
385         // Validar vindos de forma correta do frontend (Helper)
386         $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
387     }else{
388         // Validar campos quanto ao tipo de dado e tamanho (Helper)
389         $retornoIdUsuario = validarDados($resultado->idUserario,'int');
390
391         if($retornoIdUsuario['codigoHelper'] != 0){
392             $erros[] = ['codigo' => $retornoIdUsuario['codigoHelper'],
393                 'campo' => 'ID Usuário',
394                 'msg' => $retornoIdUsuario['msg']];
395         }
396         //Se não encontrar erros
397         if (empty($erros)) {
398
399             //Fazendo os setters
400             $this->setIdUsuario($resultado->idUserario);
401
402             //Realizo a instância da Model
403             $this->load->model('M_usuario');
404
405             //Atributo $retorno recebe array com informações
406             $resBanco = $this->M_usuario->desativar($this->getIdUsuario());
407
408             if ($resBanco['codigo']== 1) {
409                 $sucesso = true;
410             } else {
411                 // Captura erro do banco
412                 $erros[] = [
413                     'codigo' => $resBanco['codigo'],
414                     'msg' => $resBanco['msg']
415                 ];
416             }
417         }
418     }
419 } catch (Exception $e) {
420     $retorno = array('codigo' => 0,
421         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
422         $e->getMessage());
423 }
424 // Monta retorno único
425 if ($sucesso == true) {
426     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
427         'msg' => $resBanco['msg']];
428 } else {
429     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
430 }
431
432 //Retorno no formato JSON
433 echo json_encode($retorno);
434 }
435
436 public function login() {
437     //Atributos para controlar o status de nosso método
438     $erros = [];
439     $sucesso = false;
440
441     try {
442         //Usuário e senha recebidos via JSON
443         //e colocados em atributos
444         $json = file_get_contents('php://input');
445         $resultado = json_decode($json);
446
447         //Array com os dados que deverão vir do Front
448         $lista = array(
449             "usuario" => '0',

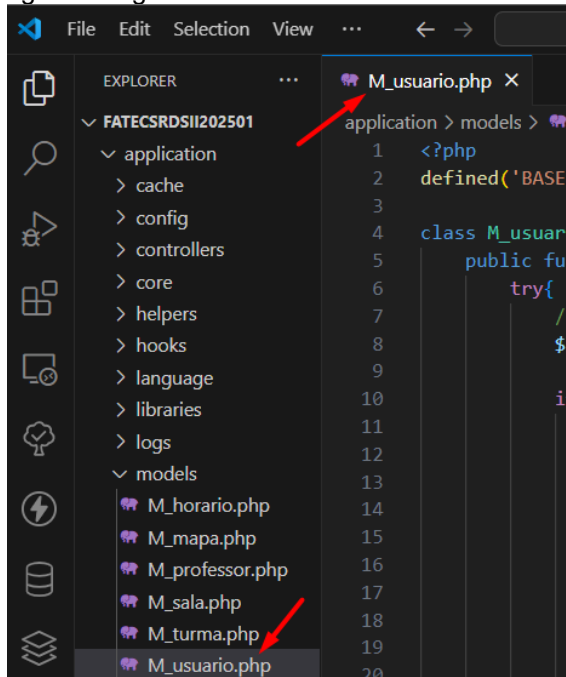
```

```

450     "senha" => '0'
451 );
452
453 if (verificarParam($resultado, $lista) != 1) {
454     // Validar vindos de forma correta do frontend (Helper)
455     $erros[] = ['codigo' => 99, 'msg' => 'Campos inexistentes ou incorretos no FrontEnd.'];
456 }else{
457     // Validar campos quanto ao tipo de dado e tamanho (Helper)
458     $retornoUsuario = validarDados($resultado->usuario, 'string', true);
459     $retornoSenha = validarDados($resultado->senha, 'string', true);
460
461     if($retornoUsuario['codigoHelper'] != 0){
462         $erros[] = ['codigo' => $retornoUsuario['codigoHelper'],
463                     'campo' => 'Usuário',
464                     'msg' => $retornoUsuario['msg']];
465     }
466
467     if($retornoSenha['codigoHelper'] != 0){
468         $erros[] = ['codigo' => $retornoSenha['codigoHelper'],
469                     'campo' => 'Senha',
470                     'msg' => $retornoSenha['msg']];
471     }
472
473     //Se não encontrar erros
474     if (empty($erros)) {
475         //Fazendo os setters
476         $this->setUsuario($resultado->usuario);
477         $this->setSenha($resultado->senha);
478
479         //Realizo a instância da Model
480         $this->load->model('M_usuario');
481
482         //Atributo $retorno recebe array com informações
483         //da validação do acesso
484         $resBanco = $this->M_usuario->validaLogin($this->getUsuario(),
485                                                 $this->getSenha());
486
487         if ($resBanco['codigo']!= 1) {
488             $sucesso = true;
489         } else {
490             // Captura erro do banco
491             $erros[] = [
492                 'codigo' => $resBanco['codigo'],
493                 'msg' => $resBanco['msg']
494             ];
495         }
496     }
497
498 }
499 } catch (Exception $e) {
500     $retorno = array('codigo' => 0,
501                     'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
502                     $e->getMessage());
503 }
504
505 // Monta retorno único
506 if ($sucesso == true) {
507     $retorno = ['sucesso' => $sucesso, 'codigo' => $resBanco['codigo'],
508                 'msg' => $resBanco['msg']];
509 } else {
510     $retorno = ['sucesso' => $sucesso, 'erros' => $erros];
511 }
512
513 //Retorno no formato JSON
514 echo json_encode($retorno);
515 }
516 }

```

Agora vamos implementar a camada de modelo, ou seja, nossa model, assim sendo, vamos criar um arquivo na pasta models de nossa estrutura, o arquivo M_usuario.php como a figura a seguir:



Vamos codificar da forma a seguir:

```
M_usuario.php U X
application > models > M_usuario.php
1  <?php
2  defined('BASEPATH') or exit('No direct script access allowed');
3
4  class M_usuario extends CI_Model {
5      /*
6       * Validação dos tipos de retornos nas validações (Código de erro)
7       * 0 - Erro de exceção
8       * 1 - Operação realizada no banco de dados com sucesso (Inserção, Alteração, Consulta ou Exclusão)
9       * 4 - Usuário não validado no sistema
10      * 5 - Usuário desativado no sistema
11      * 6 - Usuário não cadastrado no sistema
12      * 8 - Houve algum problema de inserção, atualização, consulta ou exclusão
13      * 9 - Horário desativado no sistema
14      * 10 - Horário já cadastrado
15      * 11 - Horário não encontrado pelo método publico
16      * 98 - Método auxiliar de consulta que não trouxe dados
17      */
18
19      public function inserir($nome, $email, $usuario, $senha){
20          try{
21              //Verificar o status do usuário antes de fazer o insert
22              $retornoUsuario = $this->validaUsuario($usuario);
23
24              if($retornoUsuario['codigo'] == 4){
25                  //Query de inserção dos dados
26                  $this->db->query("insert into tbl_usuario (nome, email, usuario, senha)
27                      values ('$nome', '$email', '$usuario', md5('$senha'))");
28
29                  //Verificar se a inserção ocorreu com sucesso
30                  if($this->db->affected_rows() > 0){
31                      $dados = array('codigo' => 1,
32                      'msg' => 'Usuário cadastrado corretamente.');

```

```

39         }else{
40             $dados = array('codigo' => $retornoUsuario['codigo'],
41                             'msg' => $retornoUsuario['msg']);
42         }
43     } catch (Exception $e) {
44         $dados = array('codigo' => 00,
45                         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
46                         $e->getMessage());
47     }
48     //Envia o array $dados com as informações tratadas
49     //acima pela estrutura de decisão if
50     return $dados;
51 }
52
53 public function consultar($nome, $email, $usuario){
54     //-----
55     //Função que servirá para três tipos de consulta:
56     // * Para todos os usuários;
57     // * Para um determinado usuário;
58     // * Para nomes de usuários;
59     //-----
60
61     try{
62         //Query para consultar dados de acordo com parâmetros passados
63         $sql = "select id_usuario, nome, usuario, email
64                 from tbl_usuario
65                 where estatus != 'D'";
66
67         if(trim($nome) != ''){
68             $sql = $sql . "and nome like '%$nome%' ";
69         }
70
71         if(trim($email) != ''){
72             $sql = $sql . "and email = '$email' ";
73         }
74
75         if(trim($usuario) != '') {
76             $sql = $sql . "and usuario like '%$usuario%' ";
77         }
78
79         $retorno = $this->db->query($sql);
80
81         //Verificar se a consulta ocorreu com sucesso
82         if($retorno->num_rows() > 0){
83             $dados = array('codigo' => 1,
84                             'msg' => 'Consulta efetuada com sucesso.',
85                             'dados' => $retorno->result());
86         }else{
87             $dados = array('codigo' => 6,
88                             'msg' => 'Dados não encontrados.');
```

```

110
111 //Vamos comparar os items
112 if($nome !== ''){
113     $query .= "nome = '$nome', ";
114 }
115
116 if($email !== ''){
117     $query .= "email = '$email', ";
118 }
119
120 if($senha !== ''){
121     $query .= "senha = md5('$senha'), ";
122 }
123
124 //Termino a concatenação da query
125 $queryFinal = rtrim($query, ", ") . " where id_usuario = $idUserario";
126
127 //Executo a Query de atualização dos dados
128 $this->db->query($queryFinal);
129
130 //Verificar se a atualização ocorreu com sucesso
131 if($this->db->affected_rows() > 0){
132     $dados = array('codigo' => 1,
133         'msg' => 'Usuário atualizado corretamente');
134 }
135 }else{
136     $dados = array('codigo' => 8,
137         'msg' => 'Houve algum problema na atualização na
138         tabela de usuários');
139 }
140 }else{
141     $dados = array('codigo' => $retornoUsuario['codigo'],
142         'msg' => $retornoUsuario['msg']);
143 }
144 }
145
146 }catch (Exception $e) {
147     $dados = array('codigo' => 00,
148         'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
149         $e->getMessage());
150 }
151 //Envia o array $dados com as informações tratadas
152 //acima pela estrutura de decisão if
153 return $dados;
154 }
155
156 public function desativar($idUserario){
157     try{
158         //Verificar o status do usuário antes de fazer o update
159         $retornoUsuario = $this->validaIdUsuario($idUserario);
160
161         if($retornoUsuario['codigo'] == 1){
162             //Query de atualização dos dados
163             $this->db->query("update tbl_usuario set status = 'D'
164                 where id_usuario = $idUserario");
165
166             //Verificar se a atualização ocorreu com sucesso
167             if($this->db->affected_rows() > 0){
168                 $dados = array('codigo' => 1,
169                     'msg' => 'Usuário DESATIVADO corretamente');
170             }
171             }else{
172                 $dados = array('codigo' => 8,
173                     'msg' => 'Houve algum problema na DESATIVACÃO
174                     do usuário');
175             }
176         }else{
177             $dados = array('codigo' => $retornoUsuario['codigo'],
178                 'msg' => $retornoUsuario['msg']);
179         }
180     }

```

```

181     } catch (Exception $e) {
182         $dados = array('codigo' => 00,
183                        'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
184                        $e->getMessage());
185     }
186     //Envia o array $dados com as informações tratadas
187     //acima pela estrutura de decisão if
188     return $dados;
189 }
190
191 private function validaUsuario($usuario){
192     try{
193         //Atributo retorno recebe o resultado do SELECT
194         //Sem status pois teremos que validar
195         //para verificar se está deletado virtualmente ou não.
196         $retorno = $this->db->query("select * from tbl_usuario
197                                   where usuario = '$usuario'");
198
199         //Verifica se a quantidade de linhas trazidas na consulta é superior a 0
200         //Vinculamos o resultado da query para tratarmos o resultado do status
201         $linha = $retorno->row();
202
203         if($retorno->num_rows() == 0){
204             $dados = array('codigo' => 4,
205                            'msg' => 'Usuário não existe na base de dados.');
```

```

206         }else{
207             if(trim($linha->estatus) == "D"){
208                 $dados = array('codigo' => 5,
209                                'msg' => 'Usuário DESATIVADO NA BASE DE DADOS,
210                                não pode ser utilizado!');
211             }else{
212                 $dados = array('codigo' => 1,
213                                'msg' => 'Usuário correto');
```

```

214             }
215         }
216     } catch (Exception $e) {
217         $dados = array('codigo' => 00,
218                        'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
219                        $e->getMessage());
220     }
221
222     return $dados;
223 }
224
225 private function validaIdUsuario($idUsuario){
226     try{
227         //Atributo retorno recebe o resultado do SELECT
228         //Sem status pois teremos que validar
229         //para verificar se está deletado virtualmente ou não.
230         $retorno = $this->db->query("select * from tbl_usuario
231                                   where id_usuario = $idUsuario");
232
233         //Verifica se a quantidade de linhas trazidas na consulta é superior a 0,
234         //Vinculamos o resultado da query para tratarmos o resultado do status
235         $linha = $retorno->row();
236
237         if($retorno->num_rows() == 0){
238             $dados = array('codigo' => 4,
239                            'msg' => 'Usuário não existe na base de dados.');
```

```

240         }else{
241             if(trim($linha->estatus) == "D"){
242                 $dados = array('codigo' => 5,
243                                'msg' => 'Usuário JÁ DESATIVADO NA BASE DE DADOS!');
244             }else{
245                 $dados = array('codigo' => 1,
246                                'msg' => 'Usuário correto');
```

```

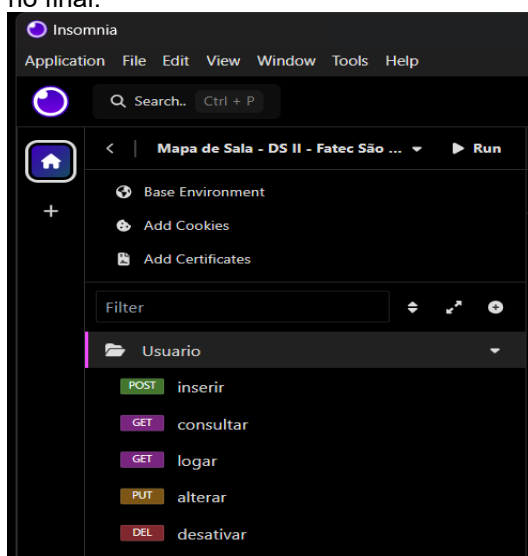
247             }
248         }
249     } catch (Exception $e) {
250         $dados = array('codigo' => 00,
251                        'msg' => 'ATENÇÃO: O seguinte erro aconteceu -> ',
252                        $e->getMessage());
253     }

```

```

254         $e->getMessage();
255     }
256
257     return $dados;
258 }
259
260 public function validaLogin($usuario, $senha){
261     try{
262         //Atributo retorno recebe o resultado do SELECT
263         //realizado na tabela de usuários lembrando da função MD5()
264         //por causa da criptografia, e sem status pois teremos que validar
265         //para verificar se está deletado virtualmente ou não.
266         $retorno = $this->db->query("select * from tbl_usuario
267                                     where usuario = '$usuario'
268                                     and senha = md5('$senha')");
269
270         //Verifica se a quantidade de linhas trazidas na consulta é superior a 0,
271         //Vinculamos o resultado da query para tratarmos o resultado do status
272         $linha = $retorno->row();
273
274         if($retorno->num_rows() == 0){
275             $dados = array('codigo' => 4,
276                           'msg' => 'Usuário ou senha inválidos.');
```

Com isso, finalizamos a implementação de nosso sexto objeto do sistema, os *endpoints* no *Insomnia* vocês devem montar conforme falamos no objeto anterior, são importantes para o salvamento, recomento que ao final de cada aula, façam a exportação do mesmo e guardem para que possam atualizar e utilizar durante as aulas, abaixo vou mostrar como ficou estes *endpoints* no final.



Assim, agora temos 6 objetos criados em nosso sistema, ou seja, agora sim, podemos pensar nas telas, ou seja, nossas *Views*. Mas antes, vamos pensar em nossa P2, que deverá ser a implementação de tudo o que vimos até aqui, com todos os *Endpoints*.

Portanto, vocês terão até o dia 19/05, para apresentar de forma presencial no dia de nossa aula, o checklist das atividades que fizemos até o momento, a ideia é termos nosso projeto todo funcional, rodando no *Insomnia*, de acordo com o seguinte *checklist*:

Validações:**Helpers:**

- As função `validarCPF()`, e a validação de e-mail, ou seja, alterações nos métodos existentes como explicado em aula;

Professor:

- Pesquisa realizada com sucesso;
- Cadastro efetuado com sucesso;
- Alteração efetuada com sucesso;
- Desativação efetuada com sucesso.

Turma:

- Pesquisa realizada com sucesso;
- Cadastro efetuado com sucesso;
- Alteração efetuada com sucesso;
- Desativação efetuada com sucesso.

Mapa:

- Pesquisa realizada com sucesso;
- Cadastro efetuado com sucesso;
- Alteração efetuada com sucesso;
- Desativação efetuada com sucesso.

Usuário:

- Pesquisa realizada com sucesso;
- Cadastro efetuado com sucesso;
- Alteração efetuada com sucesso;
- Desativação efetuada com sucesso;
- Logar efetuado com sucesso.

Observação: Irei validar os *endpoints* com testes funcionais, ou seja, irei “marretar” para ver se todas as funções foram codificadas como instruído em aula.